DEITEL® DEVELOPER SERIES

python™

Python®

for Programmers

with introductory AI case studies

- Natural Language Processing
- Data Mining Twitter®
- IBM® Watson™
- Machine Learning with scikit-learn®
- Deep Learning with Keras
- Big Data with Hadoop®, Spark™, NoSQL and the Cloud
- Internet of Things (IoT)
- Python Standard Library
- Data Science Libraries: NumPy, Pandas, SciPy, NLTK, TextBlob, Tweepy, Matplotlib, Seaborn, Folium and more

PAUL DEITEL • HARVEY DEITEL

Python® for Programmers

Deitel® Developer Series

Python for Programmers

Paul Deitel
Harvey Deitel

Visit us on the Web: informit.com

Library of Congress Control Number: 2019933267

Cover design by Paul Deitel, Harvey Deitel, and Chuti Prasertsith
Cover art by Agsandrew/Shutterstock

ISBN-13: 978-0-13-522433-5
ISBN-10: 0-13-522433-0

1 19

D

reface

"There's gold in them thar hills!" [1]

[1] Source unknown, frequently misattributed to Mark Twain.

Welcome to *Python for Programmers*! In this book, you'll learn hands-on with today's most compelling, leading-edge computing technologies, and you'll program in Python—one of the world's most popular languages and the fastest growing among them.

Developers often quickly discover that they like Python. They appreciate its expressive power, readability, conciseness and interactivity. They like the world of open-source software development that's generating a rapidly growing base of reusable software for an enormous range of application areas.

For many decades, some powerful trends have been in place. Computer hardware has rapidly been getting faster, cheaper and smaller. Internet bandwidth has rapidly been getting larger and cheaper. And quality computer software has become ever more abundant and essentially free or nearly free through the "open source" movement. Soon, the "Internet of Things" will connect tens of billions of devices of every imaginable type. These will generate enormous volumes of data at rapidly increasing speeds and quantities.

In computing today, the latest innovations are "all about the data"—*data* science, *data* analytics, big *data*, relational *data*bases (SQL), and NoSQL and NewSQL *data*bases, each of which we address along with an innovative treatment of Python programming.

JOBS REQUIRING DATA SCIENCE SKILLS

In 2011, McKinsey Global Institute produced their report, "Big data: The next frontier for innovation, competition and productivity." In it, they said, "The United States alone faces a shortage of 140,000 to 190,000 people with deep analytical skills as well as 1.5 million managers and analysts to analyze big data and make decisions based on their findings." [2] This continues to be the case. The August 2018 "LinkedIn Workforce Report" says the United States has a shortage of over 150,000 people with data science skills. [3] A 2017 report from IBM, Burning Glass Technologies and the Business-Higher Education Forum, says that by 2020 in the United States there will be hundreds of thousands of new jobs requiring data science skills. [4]

[2] ttps://www.mckinsey.com/~/media/McKinsey/Business%20Functions/McKinsey%20Digital/Our%20I page 3).

[3] ttps://economicgraph.linkedin.com/resources/linkedin-workforce-eport-august-2018.

[4] ttps://www.burning-glass.com/wp-ontent/uploads/The_Quant_Crunch.pdf (page 3).

MODULAR ARCHITECTURE

The book's **modular architecture** (please see the **Table of Contents graphic** on the book's inside front cover) helps us meet the diverse needs of various professional audiences.

hapters 1– 0 cover Python programming. These chapters each include a brief **Intro to Data Science** section introducing artificial intelligence, basic descriptive statistics, measures of central tendency and dispersion, simulation, static and dynamic visualization, working with CSV files, pandas for data exploration and data wrangling, time series and

imple linear regression. These help you prepare for the data science, AI, big data and cloud case studies in hapters 11– 6, which present opportunities for you to use **real-world datasets** in complete case studies.

After covering Python hapters 1– 5 and a few key parts of hapters 6– 7 , you'll be able to handle significant portions of the case studies in hapters 11– 6. The "Chapter Dependencies" section of this Preface will help trainers plan their professional courses in the context of the book's unique architecture.

hapters 11– 6 are loaded with cool, powerful, contemporary examples. They present hands-on implementation case studies on topics such as **natural language processing**, **data mining Twitter**, **cognitive computing with IBM's Watson**, **supervised machine learning with classification and regression**, **unsupervised machine learning with clustering**, **deep learning with convolutional neural networks**, **deep learning with recurrent neural networks**, **big data with Hadoop, Spark and NoSQL databases**, the **Internet of Things** and more. Along the way, you'll acquire a **broad literacy** of data science terms and concepts, ranging from brief definitions to using concepts in small, medium and large programs. Browsing the book's detailed **Table of Contents** and Index will give you a sense of the breadth of coverage.

KEY FEATURES

KIS (Keep It Simple), KIS (Keep it Small), KIT (Keep it Topical)

- **Keep it simple—**In every aspect of the book, we strive for **simplicity and clarity**. For example, when we present natural language processing, we use the simple and intuitive **TextBlob** library rather than the more complex NLTK. In our deep learning presentation, we prefer **Keras** to TensorFlow. In general, when multiple libraries could be used to perform similar tasks, we use the simplest one.
- **Keep it small—**Most of the book's 538 examples are small—often just a few lines of code, with immediate interactive **IPython** feedback. We also include 40 larger scripts and in-depth case studies.
- **Keep it topical—**We read scores of recent Python-programming and data science books, and browsed, read or watched about 15,000 current articles, research papers, white papers, videos, blog posts, forum posts and documentation pieces. This enabled us to "take the pulse" of the Python, computer science, data science, AI, big data and cloud communities.

Immediate-Feedback: Exploring, Discovering and Experimenting with IPython

- The ideal way to learn from this book is to read it and run the code examples in parallel. Throughout the book, we use the **IPython interpreter**, which provides a friendly, immediate-feedback interactive mode for quickly exploring, discovering and experimenting with Python and its extensive libraries.
- Most of the code is presented in **small, interactive IPython sessions**. For each code snippet you write, IPython immediately reads it, evaluates it and prints the results. This **instant feedback** keeps your attention, boosts learning, facilitates rapid prototyping and speeds the software-development process.
- Our books always emphasize the **live-code approach**, focusing on complete, working programs with live inputs and outputs. IPython's "magic" is that it turns even snippets into code that "comes alive" as you enter each line. This promotes learning and encourages experimentation.

Python Programming Fundamentals

- First and foremost, this book provides rich Python coverage.
- We discuss Python's programming models—**procedural programming**, **functional-**

tyle programming and **object-oriented programming**.

- We use best practices, emphasizing current idiom.
- **Functional-style programming** is used throughout the book as appropriate. A chart in hapter 4 lists most of Python's key functional-style programming capabilities and the chapters in which we initially cover most of them.

538 Code Examples

- You'll get an engaging, challenging and entertaining introduction to Python with **538 real-world examples** ranging from individual snippets to substantial computer science, data science, artificial intelligence and big data case studies.
- You'll attack significant tasks with **AI, big data and cloud** technologies like **natural language processing**, **data mining Twitter**, **machine learning**, **deep learning**, **Hadoop**, **MapReduce-**, **Spark**, **IBM Watson**, key data science libraries (**NumPy**, **pandas**, **SciPy**, **NLTK**, **TextBlob**, **spaCy**, **Textatistic**, **Tweepy**, **Scikit-learn**, **Keras**), key visualization libraries (**Matplotlib**, **Seaborn**, **Folium**) and more.

Avoid Heavy Math in Favor of English Explanations

- We capture the conceptual essence of the mathematics and put it to work in our examples. We do this by using libraries such as **statistics**, **NumPy**, **SciPy**, **pandas** and many others, which hide the mathematical complexity. So, it's straightforward for you to get many of the benefits of mathematical techniques like **linear regression** without having to know the mathematics behind them. In the **machine-learning** and **deep-learning** examples, we focus on creating objects that do the math for you "behind the scenes."

Visualizations

- 67 **static, dynamic, animated and interactive visualizations** (charts, graphs, pictures, animations etc.) help you understand concepts.
- Rather than including a treatment of low-level graphics programming, we focus on high-level visualizations produced by **Matplotlib**, **Seaborn**, **pandas** and **Folium** (for **interactive maps**).
- We use visualizations as a pedagogic tool. For example, we make the **law of large numbers** "come alive" in a dynamic **die-rolling simulation** and bar chart. As the number of rolls increases, you'll see each face's percentage of the total rolls gradually approach 16.667% (1/6th) and the sizes of the bars representing the percentages equalize.
- Visualizations are crucial in big data for **data exploration** and **communicating reproducible research results**, where the data items can number in the millions, billions or more. A common saying is that a picture is worth a thousand words [5] —in **big data**, a visualization could be worth billions, trillions or even more items in a database. Visualizations enable you to "fly 40,000 feet above the data" to see it "in the large" and to get to know your data. **Descriptive statistics** help but can be misleading. For example, Anscombe's quartet [6] demonstrates through visualizations that *significantly different* datasets can have *nearly identical* descriptive statistics.

[5] ttps://en.wikipedia.org/wiki/A_picture_is_worth_a_thousand_words.

[6] ttps://en.wikipedia.org/wiki/Anscombe%27s_quartet.

- We show the visualization and animation code so you can implement your own. We also provide the animations in source-code files and as **Jupyter Notebooks**, so you can conveniently customize the code and animation parameters, re-execute the animations and see the effects of the changes.

Data Experiences

- Our **Intro to Data Science sections** and case studies in hapters 11– 6 provide rich data experiences.
- You'll work with many **real-world datasets and data sources**. There's an enormous variety of **free open datasets** available online for you to experiment with. Some of the sites we reference list hundreds or thousands of datasets.
- Many libraries you'll use come bundled with popular datasets for experimentation.
- You'll learn the steps required to obtain data and prepare it for analysis, analyze that data using many techniques, tune your models and communicate your results effectively, especially through visualization.

GitHub

- **GitHub** is an excellent venue for finding open-source code to incorporate into your projects (and to contribute your code to the open-source community). It's also a crucial element of the software developer's arsenal with version control tools that help teams of developers manage open-source (and private) projects.
- You'll use an extraordinary range of free and open-source Python and data science **libraries**, and **free**, **free-trial** and **freemium** offerings of software and cloud services. Many of the libraries are hosted on GitHub.

Hands-On Cloud Computing

- Much of big data analytics occurs in the cloud, where it's easy to scale *dynamically* the amount of hardware and software your applications need. You'll work with various cloud-based services (some directly and some indirectly), including **Twitter**, **Google Translate**, **IBM Watson**, **Microsoft Azure**, **OpenMapQuest**, **geopy**, **Dweet.io** and **PubNub**.
- • We encourage you to use free, free trial or freemium cloud services. We prefer those that don't require a credit card because you don't want to risk accidentally running up big bills. **If you decide to use a service that requires a credit card, ensure that the tier you're using for free will not automatically jump to a paid tier.**

Database, Big Data and Big Data Infrastructure

- According to IBM (Nov. 2016), 90% of the world's data was created in the last two years. [7] Evidence indicates that the speed of data creation is rapidly accelerating.

[7] ttps://public.dhe.ibm.com/common/ssi/ecm/wr/en/wrl12345usen/watson-customer-engagement--watson-marketing-wr-other-papers-and-reports-rl12345usen-20170719.pdf.

- According to a March 2016 *AnalyticsWeek* article, within five years there will be over 50 billion devices connected to the Internet and by 2020 we'll be producing 1.7 megabytes of new data every second *for every person on the planet*! [8]

[8] ttps://analyticsweek.com/content/big-data-facts/.

- We include a treatment of **relational databases** and **SQL** with **SQLite**.
- Databases are critical **big data infrastructure** for storing and manipulating the massive amounts of data you'll process. Relational databases process *structured data*—they're not geared to the *unstructured* and *semi-structured data* in big data applications. So, as big data evolved, **NoSQL and NewSQL databases** were created to handle such data efficiently. We include a NoSQL and NewSQL overview and a hands-on case study with a **MongoDB JSON document database**. MongoDB is the most popular NoSQL database.
- We discuss **big data hardware and software infrastructure** in hapter 16, " ig

ata: Hadoop, Spark, NoSQL and IoT (Internet of Things).”

Artificial Intelligence Case Studies

- In case study hapters 11– 5, we present **artificial intelligence** topics, including **natural language processing**, **data mining Twitter to perform sentiment analysis**, **cognitive computing with IBM Watson**, **supervised machine learning**, **unsupervised machine learning** and **deep learning**. hapter 16 presents the big data hardware and software infrastructure that enables computer scientists and data scientists to implement leading-edge AI-based solutions.

Built-In Collections: Lists, Tuples, Sets, Dictionaries

- There’s little reason today for most application developers to build *custom* data structures. The book features a rich **two-chapter treatment of Python’s built-in data structures—lists**, **tuples**, **dictionaries** and **sets—**with which most data-structuring tasks can be accomplished.

Array-Oriented Programming with NumPy Arrays and Pandas Series/DataFrames

- We also focus on three key data structures from open-source libraries—**NumPy arrays**, **pandas `Series`** and **pandas `DataFrames`**. These are used extensively in data science, computer science, artificial intelligence and big data. NumPy offers as much as two orders of magnitude higher performance than built-in Python lists.

- We include in hapter 7 a rich treatment of NumPy arrays. Many libraries, such as pandas, are built on NumPy. The **Intro to Data Science sections** in hapters 7– 9 introduce pandas `Series` and `DataFrame`s, which along with NumPy arrays are then used throughout the remaining chapters.

File Processing and Serialization

- hapter 9 presents **text-file processing**, then demonstrates how to serialize objects using the popular **JSON (JavaScript Object Notation)** format. JSON is used frequently in the data science chapters.

- Many data science libraries provide built-in file-processing capabilities for loading datasets into your Python programs. In addition to plain text files, we process files in the popular **CSV (comma-separated values) format** using the Python Standard Library’s `csv` module and capabilities of the pandas data science library.

Object-Based Programming

- We emphasize using the huge number of valuable classes that the **Python open-source community** has packaged into industry standard class libraries. You’ll focus on knowing what libraries are out there, choosing the ones you’ll need for your apps, creating objects from existing classes (usually in one or two lines of code) and making them “jump, dance and sing.” This **object-*based* programming** enables you to build impressive applications quickly and concisely, which is a significant part of Python’s appeal.

- With this approach, you’ll be able to use machine learning, deep learning and other AI technologies to quickly solve a wide range of intriguing problems, including **cognitive computing** challenges like **speech recognition** and **computer vision**.

Object-Oriented Programming

- Developing *custom* classes is a crucial **object-oriented- programming** skill, along with inheritance, polymorphism and duck typing. We discuss these in hapter 10.

- hapter 10 includes a discussion of **unit testing with `doctest`** and a fun card-shuffling-and-dealing simulation.

- Chapters 11–16 require only a few straightforward custom class definitions. In Python, you'll probably use more of an object-based programming approach than full-out object-oriented programming.

Reproducibility

- In the sciences in general, and data science in particular, there's a need to reproduce the results of experiments and studies, and to communicate those results effectively. **Jupyter Notebooks** are a preferred means for doing this.

- We discuss reproducibility throughout the book in the context of programming techniques and software such as Jupyter Notebooks and **Docker**.

Performance

- We use the `%timeit` **profiling tool** in several examples to compare the performance of different approaches to performing the same tasks. Other performance-related discussions include generator expressions, NumPy arrays vs. Python lists, performance of machine-learning and deep-learning models, and Hadoop and Spark distributed-computing performance.

Big Data and Parallelism

- In this book, rather than writing your own parallelization code, you'll let libraries like Keras running over TensorFlow, and big data tools like Hadoop and Spark parallelize operations for you. In this big data/AI era, the sheer processing requirements of massive data applications demand taking advantage of true parallelism provided by **multicore processors**, **graphics processing units (GPUs)**, **tensor processing units (TPUs)** and huge ***clusters* of computers in the cloud**. Some big data tasks could have thousands of processors working in parallel to analyze massive amounts of data expeditiously.

CHAPTER DEPENDENCIES

If you're a trainer planning your syllabus for a professional training course or a developer deciding which chapters to read, this section will help you make the best decisions. Please read the one-page color **Table of Contents** on the book's inside front cover—this will quickly familiarize you with the book's unique architecture. Teaching or reading the chapters in order is easiest. However, much of the content in the Intro to Data Science sections at the ends of Chapters 1–10 and the case studies in Chapters 11–16 requires only Chapters 1–5 and small portions of Chapters 6–10 as discussed below.

Part 1: Python Fundamentals Quickstart

We recommend that you read all the chapters in order:

- **Chapter 1, Introduction to Computers and Python**, introduces concepts that lay the groundwork for the Python programming in Chapters 2–10 and the big data, artificial-intelligence and cloud-based case studies in Chapters 11–16. The chapter also includes **test-drives** of the **IPython** interpreter and **Jupyter Notebooks**.

- **Chapter 2, Introduction to Python Programming**, presents Python programming fundamentals with code examples illustrating key language features.

- **Chapter 3, Control Statements**, presents Python's **control statements** and introduces **basic list processing**.

- **Chapter 4, Functions**, introduces custom functions, presents **simulation techniques** with **random-number generation** and introduces **tuple fundamentals**.

- **Chapter 5, Sequences: Lists and Tuples**, presents Python's built-in list and tuple collections in more detail and begins introducing **functional-style programming**.

art 2: Python Data Structures, Strings and Files

The following summarizes inter-chapter dependencies for Python hapters 6– 9 and assumes that you've read hapters 1– 5 .

- **hapter 6, Dictionaries and Sets**—The Intro to Data Science section in this chapter is not dependent on the chapter's contents.
- **hapter 7, Array-Oriented Programming with NumPy**—The Intro to Data Science section requires dictionaries (hapter 6) and arrays (hapter 7).
- **hapter 8, Strings: A Deeper Look**—The Intro to Data Science section requires raw strings and regular expressions (ections 8.11– .12), and pandas `Series` and `DataFrame` features from ection 7.14's Intro to Data Science.
- **hapter 9, Files and Exceptions**—For **JSON serialization**, it's useful to understand dictionary fundamentals (ection 6.2). Also, the Intro to Data Science section requires the built-in `open` function and the `with` statement (ection 9.3), and pandas `DataFrame` features from ection 7.14's Intro to Data Science.

Part 3: Python High-End Topics

The following summarizes inter-chapter dependencies for Python hapter 10 and assumes that you've read hapters 1– 5 .

- **hapter 10, Object-Oriented Programming**—The Intro to Data Science section requires pandas `DataFrame` features from Intro to Data Science ection 7.14. Trainers wanting to cover only **classes and objects** can present ections 10.1– 0.6. Trainers wanting to cover more advanced topics like **inheritance, polymorphism and duck typing**, can present ections 10.7– 0.9. ections 10.10– 0.15 provide additional advanced perspectives.

Part 4: AI, Cloud and Big Data Case Studies

The following summary of inter-chapter dependencies for hapters 11– 6 assumes that you've read hapters 1– 5 . Most of hapters 11– 6 also require dictionary fundamentals from ection 6.2.

- **hapter 11, Natural Language Processing (NLP)**, uses pandas `DataFrame` features from ection 7.14's Intro to Data Science.
- **hapter 12, Data Mining Twitter**, uses pandas `DataFrame` features from ection .14's Intro to Data Science, string method `join` (ection 8.9), JSON fundamentals (ection 9.5), `TextBlob` (ection 11.2) and Word clouds (ection 11.3). Several examples require defining a class via inheritance (hapter 10).
- **hapter 13, IBM Watson and Cognitive Computing**, uses built-in function `open` and the `with` statement (ection 9.3).
- **hapter 14, Machine Learning: Classification, Regression and Clustering**, uses NumPy array fundamentals and method `unique` (hapter 7), pandas `DataFrame` features from ection 7.14's Intro to Data Science and Matplotlib function `subplots` (ection 10.6).
- **hapter 15, Deep Learning**, requires NumPy array fundamentals (hapter 7), string method `join` (ection 8.9), general machine-learning concepts from hapter 14 and features from hapter 14's Case Study: Classification with k-Nearest Neighbors and the Digits Dataset.
- **hapter 16, ig Data: Hadoop, Spark, NoSQL and IoT**, uses string method `split` (ection 6.2.7), Matplotlib `FuncAnimation` from ection 6.4's Intro to Data Science, pandas `Series` and `DataFrame` features from ection 7.14's Intro to Data Science, string

ethod `join` (ection 8.9), the `json` module (ection 9.5), NLTK stop words (ection 1.2.13) and from hapter 12, Twitter authentication, Tweepy's `StreamListener` class for streaming tweets, and the `geopy` and `folium` libraries. A few examples require defining a class via inheritance (hapter 10), but you can simply mimic the class definitions we provide without reading hapter 10.

JUPYTER NOTEBOOKS

For your convenience, we provide the book's code examples in **Python source code (`.py`) files** for use with the command-line IPython interpreter *and* as **Jupyter Notebooks (`.ipynb`) files** that you can load into your web browser and execute.

Jupyter Notebooks is a free, open-source project that enables you to combine text, graphics, audio, video, and interactive coding functionality for entering, editing, executing, debugging, and modifying code quickly and conveniently in a web browser. According to the article, "What Is Jupyter?":

Jupyter has become a standard for scientific research and data analysis. It packages computation and argument together, letting you build "computational narratives"; and it simplifies the problem of distributing working software to teammates and associates.[9]

[9] ttps://www.oreilly.com/ideas/what-is-jupyter.

In our experience, it's a wonderful learning environment and **rapid prototyping tool**. For this reason, we use **Jupyter Notebooks** rather than a traditional IDE, such as **Eclipse**, **Visual Studio**, **PyCharm** or **Spyder**. Academics and professionals already use Jupyter extensively for sharing research results. Jupyter Notebooks support is provided through the traditional open-source community mechanisms [0] (see "Getting Jupyter Help" later in this Preface). See the Before You Begin section that follows this Preface for software installation details and see the test-drives in ection 1.5 for information on running the book's examples.

[0] ttps://jupyter.org/community.

Collaboration and Sharing Results

Working in teams and communicating research results are both important for developers in or moving into data-analytics positions in industry, government or academia:

- The notebooks you create are **easy to share** among team members simply by copying the files or via **GitHub**.
- Research results, including code and insights, can be shared as static web pages via tools like **nbviewer** (ttps://nbviewer.jupyter.org) and **GitHub**—both automatically render notebooks as web pages.

Reproducibility: A Strong Case for Jupyter Notebooks

In data science, and in the sciences in general, experiments and studies should be reproducible. This has been written about in the literature for many years, including

- Donald Knuth's 1992 computer science publication—*Literate Programming*.[1]

  [1]Knuth, D., "Literate Programming" (PDF), *The Computer Journal*, British Computer Society, 1992.

- The article "Language-Agnostic Reproducible Data Analysis Using Literate Programming," [2] which says, "Lir (literate, reproducible computing) is based on the idea of literate programming as proposed by Donald Knuth."

  [2] ttp://journals.plos.org/plosone/article?d=10.1371/journal.pone.0164023.

Essentially, reproducibility captures the complete environment used to produce results—hardware, software, communications, algorithms (especially code), data and the **data's**

rovenance (origin and lineage).

DOCKER

In hapter 16, we'll use **Docker**—a tool for packaging software into containers that bundle everything required to execute that software conveniently, reproducibly and portably across platforms. Some software packages we use in hapter 16 require complicated setup and configuration. For many of these, you can download free preexisting **Docker containers**. These enable you to avoid complex installation issues and execute software locally on your desktop or notebook computers, making Docker a great way to help you get started with new technologies quickly and conveniently.

Docker also helps with reproducibility. You can create custom Docker containers that are configured with the versions of every piece of software and every library you used in your study. This would enable other developers to recreate the environment you used, then reproduce your work, and will help you reproduce your own results. In hapter 16, you'll use Docker to download and execute a container that's preconfigured for you to code and run big data Spark applications using Jupyter Notebooks.

SPECIAL FEATURE: IBM WATSON ANALYTICS AND COGNITIVE COMPUTING

Early in our research for this book, we recognized the rapidly growing interest in **IBM's Watson**. We investigated competitive services and found Watson's "no credit card required" policy for its "free tiers" to be among the most friendly for our readers.

IBM Watson is a **cognitive-computing** platform being employed across a wide range of real-world scenarios. Cognitive-computing systems simulate the **pattern-recognition** and **decision-making** capabilities of the human brain to "learn" as they consume more data.[3, 4, 5] We include a significant hands-on Watson treatment. We use the free **Watson Developer Cloud: Python SDK**, which provides APIs that enable you to interact with Watson's services programmatically. Watson is fun to use and a great platform for letting your creative juices flow. You'll demo or use the following Watson APIs: **Conversation**, **Discovery**, **Language Translator**, **Natural Language Classifier**, **Natural Language Understanding**, **Personality Insights**, **Speech to Text**, **Text to Speech**, **Tone Analyzer** and **Visual Recognition**.

[3] ttp://whatis.techtarget.com/definition/cognitive-computing.

[4] ttps://en.wikipedia.org/wiki/Cognitive_computing.

[5] ttps://www.forbes.com/sites/bernardmarr/2016/03/23/what-everyone-hould-know-about-cognitive-computing.

Watson's Lite Tier Services and a Cool Watson Case Study

IBM encourages learning and experimentation by providing *free lite tiers* for many of its APIs.[6] In hapter 13, you'll try demos of many Watson services.[7] Then, you'll use the lite tiers of Watson's **Text to Speech**, **Speech to Text** and **Translate** services to implement a **"traveler's assistant" translation app**. You'll speak a question in English, then the app will transcribe your speech to English text, translate the text to Spanish and speak the Spanish text. Next, you'll speak a Spanish response (in case you don't speak Spanish, we provide an audio file you can use). Then, the app will quickly transcribe the speech to Spanish text, translate the text to English and speak the English response. Cool stuff!

[6] Always check the latest terms on IBM's website, as the terms and services may change.

[7] ttps://console.bluemix.net/catalog/.

TEACHING APPROACH

Python for Programmers contains a rich collection of examples drawn from many fields. You'll work through interesting, real-world examples using **real-world datasets**. The book concentrates on the principles of good **software engineering** and stresses **program**

clarity.

Using Fonts for Emphasis

We place the key terms and the index's page reference for each defining occurrence in **bold** text for easier reference. We refer to on-screen components in the **bold Helvetica** font (for example, the **File** menu) and use the `Lucida` font for Python code (for example, `x = 5`).

Syntax Coloring

For readability, we syntax color all the code. Our syntax-coloring conventions are as follows:

```
comments appear in green
keywords appear in dark blue
constants and literal values appear in light blue
errors appear in red
all other code appears in black
```

538 Code Examples

The book's **538 examples** contain approximately **4000 lines of code**. This is a relatively small amount for a book this size and is due to the fact that Python is such an expressive language. Also, our coding style is to use powerful class libraries to do most of the work wherever possible.

160 Tables/Illustrations/Visualizations

We include abundant tables, line drawings, and static, dynamic and interactive visualizations.

Programming Wisdom

We *integrate* into the discussions programming wisdom from the authors' combined nine decades of programming and teaching experience, including:

- **Good programming practices** and preferred Python idioms that help you produce clearer, more understandable and more maintainable programs.
- **Common programming errors** to reduce the likelihood that you'll make them.
- **Error-prevention tips** with suggestions for exposing bugs and removing them from your programs. Many of these tips describe techniques for preventing bugs from getting into your programs in the first place.
- **Performance tips** that highlight opportunities to make your programs run faster or minimize the amount of memory they occupy.
- **Software engineering observations** that highlight architectural and design issues for proper software construction, especially for larger systems.

SOFTWARE USED IN THE BOOK

The software we use is available for Windows, macOS and Linux and is free for download from the Internet. We wrote the book's examples using the free **Anaconda Python distribution**. It includes most of the Python, visualization and data science libraries you'll need, as well as the IPython interpreter, Jupyter Notebooks and Spyder, considered one of the best Python data science IDEs. We use only IPython and Jupyter Notebooks for program development in the book. The Before You Begin section following this Preface discusses installing Anaconda and a few other items you'll need for working with our examples.

PYTHON DOCUMENTATION

You'll find the following documentation especially helpful as you work through the book:

- The Python Language Reference:

  `ttps://docs.python.org/3/reference/index.html`

h

- The Python Standard Library:

  `ttps://docs.python.org/3/library/index.html`

- Python documentation list:

  `ttps://docs.python.org/3/`

GETTING YOUR QUESTIONS ANSWERED

Popular Python and general programming online forums include:

- `python-forum.io`
- `ttps://www.dreamincode.net/forums/forum/29-python/`
- `StackOverflow.com`

Also, many vendors provide forums for their tools and libraries. Many of the libraries you'll use in this book are managed and maintained at `github.com`. Some library maintainers provide support through the **Issues** tab on a given library's GitHub page. If you cannot find an answer to your questions online, please see our web page for the book at

ttp://www.deitel.com [8]

[8] Our website is undergoing a major upgrade. If you do not find something you need, please write to us directly at `eitel@deitel.com`.

GETTING JUPYTER HELP

Jupyter Notebooks support is provided through:

- Project Jupyter Google Group:

  ttps://groups.google.com/forum/#!forum/jupyter

- Jupyter real-time chat room:

  ttps://gitter.im/jupyter/jupyter

- GitHub

  ttps://github.com/jupyter/help

- StackOverflow:

  ttps://stackoverflow.com/questions/tagged/jupyter

- Jupyter for Education Google Group (for instructors teaching with Jupyter):

  ttps://groups.google.com/forum/#!forum/jupyter-education

SUPPLEMENTS

To get the most out of the presentation, you should execute each code example in parallel with reading the corresponding discussion in the book. On the book's web page at

ttp://www.deitel.com

we provide:

- **Downloadable Python source code** (`.py` files) and **Jupyter Notebooks** (`.ipynb` files) for the book's **code examples**.
- **Getting Started videos** showing how to use the code examples with IPython and Jupyter Notebooks. We also introduce these tools in ection 1.5.

- **Blog posts** and **book updates**.

For download instructions, see the Before You Begin section that follows this Preface.

KEEPING IN TOUCH WITH THE AUTHORS

For answers to questions or to report an error, send an e-mail to us at

eitel@deitel.com

or interact with us via **social media**:

- **Facebook**® (ttp://www.deitel.com/deitelfan)
- **Twitter**® (@deitel)
- **LinkedIn**® (ttp://linkedin.com/company/deitel-&-associates)
- **YouTube**® (ttp://youtube.com/DeitelTV)

ACKNOWLEDGMENTS

We'd like to thank Barbara Deitel for long hours devoted to Internet research on this project. We're fortunate to have worked with the dedicated team of publishing professionals at Pearson. We appreciate the efforts and 25-year mentorship of our friend and colleague Mark L. Taub, Vice President of the Pearson IT Professional Group. Mark and his team publish our professional books, LiveLessons video products and Learning Paths in the Safari service (ttps://learning.oreilly.com/). They also sponsor our Safari live online training seminars. Julie Nahil managed the book's production. We selected the cover art and Chuti Prasertsith designed the cover.

We wish to acknowledge the efforts of our reviewers. Patricia Byron-Kimball and Meghan Jacoby recruited the reviewers and managed the review process. Adhering to a tight schedule, the reviewers scrutinized our work, providing countless suggestions for improving the accuracy, completeness and timeliness of the presentation.

Reviewers

Book Reviewers

Daniel Chen, Data Scientist, Lander Analytics

Garrett Dancik, Associate Professor of Computer Science/Bioinformatics, Eastern Connecticut State University

Pranshu Gupta, Assistant Professor, Computer Science, DeSales University

David Koop, Assistant Professor, Data Science Program Co-Director, U-Mass Dartmouth

Ramon Mata-Toledo, Professor, Computer Science, James Madison University

Shyamal Mitra, Senior Lecturer, Computer Science, University of Texas at Austin

Alison Sanchez, Assistant Professor in

Daniel Chen, Data Scientist, Lander Analytics

Garrett Dancik, Associate Professor of Computer Science/Bioinformatics, Eastern Connecticut State University

Dr. Marsha Davis, Department Chair of Mathematical Sciences, Eastern Connecticut State University

Roland DePratti, Adjunct Professor of Computer Science, Eastern Connecticut State University

Shyamal Mitra, Senior Lecturer, Computer Science, University of Texas at Austin

Dr. Mark Pauley, Senior Research Fellow, Bioinformatics, School of Interdisciplinary

Economics, University of San Diego

José Antonio González Seco, IT Consultant

Jamie Whitacre, Independent Data Science Consultant

Elizabeth Wickes, Lecturer, School of Information Sciences, University of Illinois

Proposal Reviewers

Dr. Irene Bruno, Associate Professor in the Department of Information Sciences and Technology, George Mason University

Lance Bryant, Associate Professor, Department of Mathematics, Shippensburg University

Informatics, University of Nebraska at Omaha

Sean Raleigh, Associate Professor of Mathematics, Chair of Data Science, Westminster College-

Alison Sanchez, Assistant Professor in Economics, University of San Diego

Dr. Harvey Siy, Associate Professor of Computer Science, Information Science and Technology, University of Nebraska at Omaha

Jamie Whitacre, Independent Data Science Consultant

As you read the book, we'd appreciate your comments, criticisms, corrections and suggestions for improvement. Please send all correspondence to:

eitel@deitel.com

We'll respond promptly.

Welcome again to the exciting open-source world of Python programming. We hope you enjoy this look at leading-edge computer-applications development with Python, IPython, Jupyter Notebooks, data science, AI, big data and the cloud. We wish you great success!

Paul and Harvey Deitel

ABOUT THE AUTHORS

Paul J. Deitel, CEO and Chief Technical Officer of Deitel & Associates, Inc., is an MIT graduate with 38 years of experience in computing. Paul is one of the world's most experienced programming-languages trainers, having taught professional courses to software developers since 1992. He has delivered hundreds of programming courses to industry clients internationally, including Cisco, IBM, Siemens, Sun Microsystems (now Oracle), Dell, Fidelity, NASA at the Kennedy Space Center, the National Severe Storm Laboratory, White Sands Missile Range, Rogue Wave Software, Boeing, Nortel Networks, Puma, iRobot and many more. He and his co-author-, Dr. Harvey M. Deitel, are the world's best-selling programming-language textbook/professional book/video authors.

Dr. Harvey M. Deitel, Chairman and Chief Strategy Officer of Deitel & Associates, Inc., has 58 years of experience in computing. Dr. Deitel earned B.S. and M.S. degrees in Electrical Engineering from MIT and a Ph.D. in Mathematics from Boston University—he studied computing in each of these programs before they spun off Computer Science programs. He has extensive college teaching experience, including earning tenure and serving as the Chairman of the Computer Science Department at Boston College before founding Deitel & Associates, Inc., in 1991 with his son, Paul. The Deitels' publications have earned international recognition, with more than 100 translations published in Japanese, German, Russian, Spanish, French, Polish, Italian, Simplified Chinese, Traditional Chinese, Korean, Portuguese, Greek, Urdu and Turkish. Dr. Deitel has delivered hundreds of programming courses to academic, corporate, government and military clients.

ABOUT DEITEL® & ASSOCIATES, INC.

d

eitel & Associates, Inc., founded by Paul Deitel and Harvey Deitel, is an internationally recognized authoring and corporate training organization, specializing in computer programming languages, object technology, mobile app development and Internet and web software technology. The company's training clients include some of the world's largest companies, government agencies, branches of the military and academic institutions. The company offers instructor-led training courses delivered at client sites worldwide on major programming languages.

Through its 44-year publishing partnership- with Pearson/Prentice Hall, Deitel & Associates, Inc., publishes leading-edge programming textbooks and professional books in print and e-book formats, **Live-Lessons** video courses (available for purchase at `ttps://www.informit.com`), **Learning Paths** and live online training seminars in the Safari service (`ttps://learning.oreilly.com`) and **Revel™** interactive multimedia courses.

To contact Deitel & Associates, Inc. and the authors, or to request a proposal on-site, instructor-led training, write to:

`eitel@deitel.com`

To learn more about Deitel on-site corporate training, visit

`ttp://www.deitel.com/training`

Individuals wishing to purchase Deitel books can do so at

`ttps://www.amazon.com`

Bulk orders by corporations, the government, the military and academic institutions should be placed directly with Pearson. For more information, visit

`ttps://www.informit.com/store/sales.aspx`

Before You Begin

This section contains information you should review before using this book. We'll post updates at: `http://www.deitel.com`.

FONT AND NAMING CONVENTIONS

We show Python code and commands and file and folder names in a `sans-serif font`, and on-screen components, such as menu names, in a **bold sans-serif font**. We use *italics for emphasis* and **bold occasionally for strong emphasis**.

GETTING THE CODE EXAMPLES

You can download the `examples.zip` file containing the book's examples from our *Python for Programmers* web page at:

http://www.deitel.com

Click the **Download Examples** link to save the file to your local computer. Most web browsers place the file in your user account's `Downloads` folder. When the download completes, locate it on your system, and extract its `examples` folder into your user account's `Documents` folder:

- Windows: `C:\Users\`*YourAccount*`\Documents\examples`
- macOS or Linux: `~/Documents/examples`

Most operating systems have a built-in extraction tool. You also may use an archive tool such as 7-Zip (`www.7-zip.org`) or WinZip (`www.winzip.com`).

STRUCTURE OF THE `EXAMPLES` FOLDER

You'll execute three kinds of examples in this book:

- Individual code snippets in the IPython interactive environment.
- Complete applications, which are known as scripts.
- Jupyter Notebooks—a convenient interactive, web-browser-based environment in which you can write and execute code and intermix the code with text, images and video.

We demonstrate each in ection 1.5's test drives.

The `examples` folder contains one subfolder per chapter. These are named `ch##`, where `##` is the two-digit chapter number `01` to `16`—for example, `ch01`. Except for hapters 13, 5 and 6, each chapter's folder contains the following items:

- `snippets_ipynb`—A folder containing the chapter's Jupyter Notebook files.
- `snippets_py`—A folder containing Python source code files in which each code snippet we present is separated from the next by a blank line. You can copy and paste these snippets into IPython or into new Jupyter Notebooks that you create.
- Script files and their supporting files.

hapter 13 contains one application. hapters 15 and 6 explain where to find the files you need in the `ch15` and `ch16` folders, respectively.

INSTALLING ANACONDA

We use the easy-to-install Anaconda Python distribution with this book. It comes with almost everything you'll need to work with our examples, including:

- the IPython interpreter,
- most of the Python and data science libraries we use,
- a local Jupyter Notebooks server so you can load and execute our notebooks, and
- various other software packages, such as the Spyder Integrated Development Environment (IDE)—we use only IPython and Jupyter Notebooks in this book.

Download the Python 3.x Anaconda installer for Windows, macOS or Linux from:

ttps://www.anaconda.com/download/

When the download completes, run the installer and follow the on-screen instructions. To ensure that Anaconda runs correctly, do not move its files after you install it.

UPDATING ANACONDA

Next, ensure that Anaconda is up to date. Open a command-line window on your system as follows:

- On macOS, open a **Terminal** from the **Applications** folder's **Utilities** subfolder.
- On Windows, open the **Anaconda Prompt** from the start menu. When doing this to update Anaconda (as you'll do here) or to install new packages (discussed momentarily), execute the **Anaconda Prompt** as an *administrator* by right-clicking, then selecting **More > Run as administrator**. (If you cannot find the Anaconda Prompt in the start menu, simply search for it in the **Type here to search** field at the bottom of your screen.)
- On Linux, open your system's **Terminal** or shell (this varies by Linux distribution).

In your system's command-line window, execute the following commands to update Anaconda's installed packages to their latest versions:

1. `conda update conda`
2. `conda update --all`

PACKAGE MANAGERS

The `conda` command used above invokes the **`conda` package manager**—one of the two key Python package managers you'll use in this book. The other is **`pip`**. Packages contain the files required to install a given Python library or tool. Throughout the book, you'll use `conda` to install additional packages, unless those packages are not available through `conda`, in which case you'll use `pip`. Some people prefer to use `pip` exclusively as it currently supports more packages. If you ever have trouble installing a package with `conda`, try `pip` instead.

INSTALLING THE PROSPECTOR STATIC CODE ANALYSIS TOOL

h

ou may want to analyze you Python code using the Prospector analysis tool, which checks your code for common errors and helps you improve it. To install Prospector and the Python libraries it uses, run the following command in the command-line window:

```
pip install prospector
```

INSTALLING JUPYTER-MATPLOTLIB

We implement several animations using a visualization library called Matplotlib. To use them in Jupyter Notebooks, you must install a tool called `ipympl`. In the **Terminal**, **Anaconda Command Prompt** or shell you opened previously, execute the following commands [1] one at a time:

[1] ttps://github.com/matplotlib/jupyter-matplotlib.

```
conda install -c conda-forge ipympl
conda install nodejs
jupyter labextension install @jupyter-widgets/jupyterlab-manager
jupyter labextension install jupyter-matplotlib
```

INSTALLING THE OTHER PACKAGES

Anaconda comes with approximately 300 popular Python and data science packages for you, such as NumPy, Matplotlib, pandas, Regex, BeautifulSoup, requests, Bokeh, SciPy, SciKit-Learn, Seaborn, Spacy, sqlite, statsmodels and many more. The number of additional packages you'll need to install throughout the book will be small and we'll provide installation instructions as necessary. As you discover new packages, their documentation will explain how to install them.

GET A TWITTER DEVELOPER ACCOUNT

If you intend to use our "Data Mining Twitter" chapter and any Twitter-based examples in subsequent chapters, apply for a Twitter developer account. Twitter now requires registration for access to their APIs. To apply, fill out and submit the application at

ttps://developer.twitter.com/en/apply-for-access

Twitter reviews every application. At the time of this writing, personal developer accounts were being approved immediately and company-account applications were

aking from several days to several weeks. Approval is not guaranteed.

INTERNET CONNECTION REQUIRED IN SOME CHAPTERS

While using this book, you'll need an Internet connection to install various additional Python libraries. In some chapters, you'll register for accounts with cloud-based services, mostly to use their free tiers. Some services require credit cards to verify your identity. In a few cases, you'll use services that are not free. In these cases, you'll take advantage of monetary credits provided by the vendors so you can try their services without incurring charges. **Caution: Some cloud-based services incur costs once you set them up. When you complete our case studies using such services, be sure to promptly delete the resources you allocated.**

SLIGHT DIFFERENCES IN PROGRAM OUTPUTS

When you execute our examples, you might notice some differences between the results we show and your own results:

- Due to differences in how calculations are performed with floating-point numbers (like –123.45, 7.5 or 0.0236937) across operating systems, you might see minor variations in outputs—especially in digits far to the right of the decimal point.
- When we show outputs that appear in separate windows, we crop the windows to remove their borders.

GETTING YOUR QUESTIONS ANSWERED

Online forums enable you to interact with other Python programmers and get your Python questions answered. Popular Python and general programming forums include:

- `python-forum.io`
- `StackOverflow.com`
- `ttps://www.dreamincode.net/forums/forum/29-python/`

Also, many vendors provide forums for their tools and libraries. Most of the libraries you'll use in this book are managed and maintained at `github.com`. Some library maintainers provide support through the **Issues** tab on a given library's GitHub page.

h

f you cannot find an answer to your questions online, please see our web page for the book at

ttp://www.deitel.com [2]

[2] Our website is undergoing a major upgrade. If you do not find something you need, please write to us directly at `eitel@deitel.com`.

You're now ready to begin reading *Python for Programmers*. We hope you enjoy the book!

h

1. Introduction to Computers and Python

Objectives

In this chapter you'll:

- Learn about exciting recent developments in computing.
- Review object-oriented programming basics.
- Understand the strengths of Python.
- Be introduced to key Python and data-science libraries you'll use in this book.
- Test-drive the IPython interpreter's interactive mode for executing Python code.
- Execute a Python script that animates a bar chart.
- Create and test-drive a web-browser-based Jupyter Notebook for executing Python code.
- Learn how big "big data" is and how quickly it's getting even bigger.
- Read a big-data case study on a popular mobile navigation app.
- Be introduced to artificial intelligence—at the intersection of computer science and data science.

Outline

1.1 INTRODUCTION

Welcome to Python—one of the world's most widely used computer programming languages and, according to the *Popularity of Programming Languages (PYPL) Index*, the world's most popular. [1]

[1] `ttps://pypl.github.io/PYPL.html` (as of January 2019).

Here, we introduce terminology and concepts that lay the groundwork for the Python programming you'll learn in hapters 2– 0 and the big-data, artificial-intelligence and cloud-based case studies we present in hapters 11– 6.

We'll review *object-oriented programming* terminology and concepts. You'll learn why Python has become so popular. We'll introduce the Python Standard Library and various data-science libraries that help you avoid "reinventing the wheel." You'll use these libraries to create software objects that you'll interact with to perform significant tasks with modest numbers of instructions.

Next, you'll work through three test-drives showing how to execute Python code:

- In the first, you'll use IPython to execute Python instructions interactively and immediately see their results.
- In the second, you'll execute a substantial Python application that will display an animated bar chart summarizing rolls of a six-sided die as they occur. You'll see the "aw f Large Numbers" in action. In hapter 6, you'll build this application with the Matplotlib visualization library.
- In the last, we'll introduce Jupyter Notebooks using JupyterLab—an interactive, web-browser-based tool in which you can conveniently write and execute Python instructions. Jupyter Notebooks enable you to include text, images, audios, videos, animations and code.

In the past, most computer applications ran on standalone computers (that is, not networked together). Today's applications can be written with the aim of communicating among the world's billions of computers via the Internet. We'll introduce the Cloud and the Internet of Things (IoT), laying the groundwork for the contemporary applications you'll develop in hapters 11– 6.

You'll learn just how big "big data" is and how quickly it's getting even bigger. Next, we'll present a big-data case study on the Waze mobile navigation app, which uses many current technologies to provide dynamic driving directions that get you to your destination as quickly and as safely as possible. As we walk through those technologies, we'll mention where you'll use many of them in this book. The chapter closes with our first Intro to Data Science section in which we discuss a key intersection between computer science and data science—artificial intelligence.

1.2 A QUICK REVIEW OF OBJECT TECHNOLOGY BASICS

As demands for new and more powerful software are soaring, building software quickly, correctly and economically is important. *Objects*, or more precisely, the *classes* objects come from, are essentially *reusable* software components. There are date objects, time objects, audio objects, video objects, automobile objects, people objects, etc. Almost any *noun* can be reasonably represented as a software object in terms of *attributes* (e.g., name, color and size) and *behaviors* (e.g., calculating, moving and communicating). Software-development groups can use a modular, object-oriented design-and-implementation approach to be much more productive than with earlier popular techniques like "structured programming." Object-oriented programs are often easier to understand, correct and modify.

Automobile as an Object

To help you understand objects and their contents, let's begin with a simple analogy. Suppose you want to *drive a car and make it go faster by pressing its accelerator pedal*. What must happen before you can do this? Well, before you can drive a car, someone has to *design* it. A car typically begins as engineering drawings, similar to the *blueprints* that describe the

esign of a house. These drawings include the design for an accelerator pedal. The pedal *hides* from the driver the complex mechanisms that make the car go faster, just as the brake pedal "hides" the mechanisms that slow the car, and the steering wheel "hides" the mechanisms that turn the car. This enables people with little or no knowledge of how engines, braking and steering mechanisms work to drive a car easily.

Just as you cannot cook meals in the blueprint of a kitchen, you cannot drive a car's engineering drawings. Before you can drive a car, it must be *built* from the engineering drawings that describe it. A completed car has an *actual* accelerator pedal to make it go faster, but even that's not enough—the car won't accelerate on its own (hopefully!), so the driver must *press* the pedal to accelerate the car.

Methods and Classes

Let's use our car example to introduce some key object-oriented programming concepts. Performing a task in a program requires a **method**. The method houses the program statements that perform its tasks. The method hides these statements from its user, just as the accelerator pedal of a car hides from the driver the mechanisms of making the car go faster. In Python, a program unit called a **class** houses the set of methods that perform the class's tasks. For example, a class that represents a bank account might contain one method to *deposit* money to an account, another to *withdraw* money from an account and a third to *inquire* what the account's balance is. A class is similar in concept to a car's engineering drawings, which house the design of an accelerator pedal, steering wheel, and so on.

Instantiation

Just as someone has to *build a car* from its engineering drawings before you can drive a car, you must *build an object* of a class before a program can perform the tasks that the class's methods define. The process of doing this is called *instantiation*. An object is then referred to as an **instance** of its class.

Reuse

Just as a car's engineering drawings can be *reused* many times to build many cars, you can *reuse* a class many times to build many objects. Reuse of existing classes when building new classes and programs saves time and effort. Reuse also helps you build more reliable and effective systems because existing classes and components often have undergone extensive *testing*, *debugging* and *performance tuning*. Just as the notion of *interchangeable parts* was crucial to the Industrial Revolution, reusable classes are crucial to the software revolution that has been spurred by object technology.

In Python, you'll typically use a *building-block approach* to create your programs. To avoid reinventing the wheel, you'll use existing high-quality pieces wherever possible. This software reuse is a key benefit of object-oriented programming.

Messages and Method Calls

d

hen you drive a car, pressing its gas pedal sends a *message* to the car to perform a task—that is, to go faster. Similarly, you *send messages to an object*. Each message is implemented as a method call that tells a method of the object to perform its task. For example, a program might call a bank-account object's *deposit* method to increase the account's balance.

Attributes and Instance Variables

A car, besides having capabilities to accomplish tasks, also has *attributes*, such as its color, its number of doors, the amount of gas in its tank, its current speed and its record of total miles driven (i.e., its odometer reading). Like its capabilities, the car's attributes are represented as part of its design in its engineering diagrams (which, for example, include an odometer and a fuel gauge). As you drive an actual car, these attributes are carried along with the car. Every car maintains its *own* attributes. For example, each car knows how much gas is in its own gas tank, but *not* how much is in the tanks of *other* cars.

An object, similarly, has attributes that it carries along as it's used in a program. These attributes are specified as part of the object's class. For example, a bank-account object has a *balance attribute* that represents the amount of money in the account. Each bank-account object knows the balance in the account it represents, but *not* the balances of the *other* accounts in the bank. Attributes are specified by the class's **instance variables**. A class's (and its object's) attributes and methods are intimately related, so classes wrap together their attributes and methods.

Inheritance

A new class of objects can be created conveniently by **inheritance**—the new class (called the **subclass**) starts with the characteristics of an existing class (called the **superclass**), possibly customizing them and adding unique characteristics of its own. In our car analogy, an object of class "convertible" certainly *is an* object of the more *general* class "automobile," but more *specifically*, the roof can be raised or lowered.

Object-Oriented Analysis and Design (OOAD)

Soon you'll be writing programs in Python. How will you create the code for your programs? Perhaps, like many programmers, you'll simply turn on your computer and start typing. This approach may work for small programs (like the ones we present in the early chapters of the book), but what if you were asked to create a software system to control thousands of automated teller machines for a major bank? Or suppose you were asked to work on a team of 1,000 software developers building the next generation of the U.S. air traffic control system? For projects so large and complex, you should not simply sit down and start writing programs.

To create the best solutions, you should follow a detailed **analysis** process for determining your project's **requirements** (i.e., defining *what* the system is supposed to do), then develop a **design** that satisfies them (i.e., specifying *how* the system should do it). Ideally, you'd go through this process and carefully review the design (and have your design reviewed

y other software professionals) before writing any code. If this process involves analyzing and designing your system from an object-oriented point of view, it's called an **object-oriented analysis-and-design (OOAD) process**. Languages like Python are object-oriented. Programming in such a language, called **object-oriented programming (OOP)**, allows you to implement an object-oriented design as a working system.

1.3 PYTHON

Python is an object-oriented scripting language that was released publicly in 1991. It was developed by Guido van Rossum of the National Research Institute for Mathematics and Computer Science in Amsterdam.

Python has rapidly become one of the world's most popular programming languages. It's now particularly popular for educational and scientific computing, [2] and it recently surpassed the programming language R as the most popular data-science programming language. [3, 4, 5] Here are some reasons why Python is popular and everyone should consider learning it: [6, 7, 8]

[2] ttps://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017.

[3] ttps://www.kdnuggets.com/2017/08/python-overtakes-r-leader-nalytics-data-science.html.

[4] ttps://www.r-bloggers.com/data-science-job-report-2017-r-passes-as-but-python-leaves-them-both-behind/.

[5] ttps://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017.

[6] ttps://dbader.org/blog/why-learn-python.

[7] ttps://simpleprogrammer.com/2017/01/18/7-reasons-why-you-should-earn-python/.

[8] ttps://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017.

- It's open source, free and widely available with a massive open-source community.
- It's easier to learn than languages like C, C++, C# and Java, enabling novices and professional developers to get up to speed quickly.
- It's easier to read than many other popular programming languages.
- It's widely used in education. [9]

[9] Tollervey, N., *Python in Education: Teach, Learn, Program* (O'Reilly Media, Inc., 2015).

- It enhances developer productivity with extensive standard libraries and third-party open-source libraries, so programmers can write code faster and perform complex tasks with minimal code. We'll say more about this in ection 1.4.
- There are massive numbers of free open-source Python applications.
- It's popular in web development (e.g., Django, Flask).
- It supports popular programming paradigms—procedural, functional-style and object-oriented. [0] We'll begin introducing functional-style programming features in hapter 4 and use them in subsequent chapters.

  [0] ttps://en.wikipedia.org/wiki/Python_(programming_language).
- It simplifies concurrent programming—with asyncio and async/await, you're able to write single-threaded concurrent code [1], greatly simplifying the inherently complex processes of writing, debugging and maintaining that code. [2]

  [1] ttps://docs.python.org/3/library/asyncio.html.

  [2] ttps://www.oreilly.com/ideas/5-things-to-watch-in-python-in-017.
- There are lots of capabilities for enhancing Python performance.
- It's used to build anything from simple scripts to complex apps with massive numbers of users, such as Dropbox, YouTube, Reddit, Instagram and Quora. [3]

  [3] ttps://www.hartmannsoftware.com/Blog/Articles_from_Software_Fans/Mosamous-Software-Programs-Written-in-Python.
- It's popular in artificial intelligence, which is enjoying explosive growth, in part because of its special relationship with data science.
- It's widely used in the financial community. [4]

  [4]Kolanovic, M. and R. Krishnamachari, *Big Data and AI Strategies: Machine Learning and Alternative Data Approach to Investing* (J.P. Morgan, 2017).
- There's an extensive job market for Python programmers across many disciplines, especially in data-science--oriented positions, and Python jobs are among the highest paid of all programming jobs. [5, 6]

  [5] ttps://www.infoworld.com/article/3170838/developer/get-paid-10-rogramming-languages-to-learn-in-2017.html.

  [6] ttps://medium.com/@ChallengeRocket/top-10-of-programming-

anguages-with-the-highest-salaries-in-2017-4390f468256e.

- *R* is a popular open-source programming language for statistical applications and visualization. Python and R are the two most widely data-science languages.

Anaconda Python Distribution

We use the Anaconda Python distribution because it's easy to install on Windows, macOS and Linux and supports the latest versions of Python, the IPython interpreter (introduced in ection 1.5.1) and Jupyter Notebooks (introduced in ection 1.5.3). Anaconda also includes other software packages and libraries commonly used in Python programming and data science, allowing you to focus on Python and data science, rather than software installation issues. The IPython interpreter [7] has features that help you explore, discover and experiment with Python, the Python Standard Library and the extensive set of third-party libraries.

[7] ttps://ipython.org/.

Zen of Python

We adhere to Tim Peters' *The Zen of Python*, which summarizes Python creator Guido van Rossum's design principles for the language. This list can be viewed in IPython with the command `import this`. The Zen of Python is defined in Python Enhancement Proposal (PEP) 20. "A PEP is a design document providing information to the Python community, or describing a new feature for Python or its processes or environment." [8]

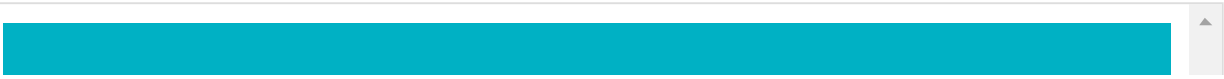
[8] ttps://www.python.org/dev/peps/pep-0001/.

1.4 IT' S THE LIBRARIES!

Throughout the book, we focus on using existing libraries to help you avoid "reinventing the wheel," thus leveraging your program-development efforts. Often, rather than developing lots of original code—a costly and time-consuming process—you can simply create an object of a pre-existing library class, which takes only a single Python statement. So, libraries will help you perform significant tasks with modest amounts of code. In this book, you'll use a broad range of Python standard libraries, data-science libraries and third-party libraries.

1.4.1 Python Standard Library

The **Python Standard Library** provides rich capabilities for text/binary data processing, mathematics, functional-style programming, file/directory access, data persistence, data compression/archiving, cryptography, operating-system services, concurrent programming, interprocess communication, networking protocols, JSON/XML/other Internet data formats, multimedia, internationalization, GUI, debugging, profiling and more. The following table lists some of the Python Standard Library modules that we use in examples.



ome of the Python Standard Library modules we use in the book

collections—Additional data structures beyond lists, tuples, dictionaries and sets.

csv—Processing comma-separated value files.

datetime, **time**—Date and time manipulations.

decimal—Fixed-point and floating-point arithmetic, including monetary calculations.

doctest—Simple unit testing via validation tests and expected results embedded in docstrings.

json—JavaScript Object Notation (JSON) processing for use with web services and NoSQL document databases.

math—Common math constants and operations.

os—Interacting with the operating system.

queue—First-in, first-out data structure.

random—Pseudorandom numbers.

re—Regular expressions for pattern matching.

sqlite3—SQLite relational database access.

statistics—Mathematical statistics functions like `mean`, `median`, `mode` and `variance`.

string—String processing.

sys—Command-line argument processing; standard input, standard output and standard error streams.

timeit—Performance analysis.

1.4.2 Data-Science Libraries

Python has an enormous and rapidly growing community of open-source developers in many fields. One of the biggest reasons for Python's popularity is the extraordinary range of open-source libraries developed by its open-source community. One of our goals is to create examples and implementation case studies that give you an engaging, challenging and entertaining introduction to Python programming, while also involving you in hands-on data science, key data-science libraries and more. You'll be amazed at the substantial tasks you can accomplish in just a few lines of code. The following table lists various popular data-science libraries. You'll use many of these as you work through our data-science examples. For visualization, we'll use Matplotlib, Seaborn and Folium, but there are many more. For a nice summary of Python visualization libraries see `ttp://pyviz.org/`.

h

opular Python libraries used in data science

Scientific Computing and Statistics

NumPy (Numerical Python)—Python does not have a built-in array data structure. It uses lists, which are convenient but relatively slow. NumPy provides the high-performance `ndarray` data structure to represent lists and matrices, and it also provides routines for processing such data structures.

SciPy (Scientific Python)—Built on NumPy, SciPy adds routines for scientific processing, such as integrals, differential equations, additional matrix processing and more. `scipy.org` controls SciPy and NumPy.

StatsModels—Provides support for estimations of statistical models, statistical tests and statistical data exploration.

Data Manipulation and Analysis

Pandas—An extremely popular library for data manipulations. Pandas makes abundant use of NumPy's `ndarray`. Its two key data structures are `Series` (one dimensional) and `DataFrame`s (two dimensional).

Visualization

Matplotlib—A highly customizable visualization and plotting library. Supported plots include regular, scatter, bar, contour, pie, quiver, grid, polar axis, 3D and text.

Seaborn—A higher-level visualization library built on Matplotlib. Seaborn adds a nicer look-and-feel, additional visualizations and enables you to create visualizations with less code.

Machine Learning, Deep Learning and Reinforcement Learning

scikit-learn—Top machine-learning library. Machine learning is a subset of AI. Deep learning is a subset of machine learning that focuses on neural networks.

keras—One of the easiest to use deep-learning libraries. Keras runs on top of

Keras —The easiest of the deep learning libraries—Keras runs on TensorFlow (Google), CNTK (Microsoft's cognitive toolkit for deep learning) or Theano (Université de Montréal).

TensorFlow—From Google, this is the most widely used deep learning library. TensorFlow works with GPUs (graphics processing units) or Google's custom TPUs (Tensor processing units) for performance. TensorFlow is important in AI and big data analytics—where processing demands are huge. You'll use the version of Keras that's built into TensorFlow.

OpenAI Gym—A library and environment for developing, testing and comparing reinforcement-learning algorithms.

Natural Language Processing (NLP)

NLTK (Natural Language Toolkit)—Used for natural language processing (NLP) tasks.

TextBlob—An object-oriented NLP text-processing library built on the NLTK and pattern NLP libraries. TextBlob simplifies many NLP tasks.

Gensim—Similar to NLTK. Commonly used to build an index for a collection of documents, then determine how similar another document is to each of those in the index.

1.5 TEST-DRIVES: USING IPYTHON AND JUPYTER NOTEBOOKS

In this section, you'll test-drive the IPython interpreter [9] in two modes:

[9] Before reading this section, follow the instructions in the Before You Begin section to install the Anaconda- Python distribution, which contains the IPython interpreter.

- In **interactive mode**, you'll enter small bits of Python code called **snippets** and immediately see their results.
- In **script mode**, you'll execute code loaded from a file that has the `.py` extension (short

for Python). Such files are called **scripts** or **programs**, and they're generally longer than the code snippets you'll use in interactive mode.

Then, you'll learn how to use the browser-based environment known as the Jupyter Notebook for writing and executing Python code.[0]

[0] Jupyter supports many programming languages by installing their "kernels." For more information see ttps://github.com/jupyter/jupyter/wiki/Jupyter-kernels.

1.5.1 Using IPython Interactive Mode as a Calculator

Let's use IPython interactive mode to evaluate simple arithmetic expressions.

Entering IPython in Interactive Mode

First, open a command-line window on your system:

- On macOS, open a **Terminal** from the **Applications** folder's **Utilities** subfolder.
- On Windows, open the **Anaconda Command Prompt** from the start menu.
- On Linux, open your system's **Terminal** or shell (this varies by Linux distribution).

In the command-line window, type `ipython`, then press *Enter* (or *Return*). You'll see text like the following, this varies by platform and by IPython version:

lick here to view code image

```
Python 3.7.0 | packaged by conda-forge | (default, Jan 20   2019, 17:24:52)
Type 'copyright', 'credits' or 'license' for more   information
IPython 6.5.0 -- An enhanced Interactive Python. Type '?'
   for help.

In [1]:
```

The text `"In [1]:"` is a *prompt*, indicating that IPython is waiting for your input. You can type `?` for help or begin entering snippets, as you'll do momentarily.

Evaluating Expressions

In interactive mode, you can evaluate expressions:

```
In [1]: 45 + 72
Out[1]: 117

In [2]:
```

After you type `45 + 72` and press *Enter*, IPython *reads* the snippet, *evaluates* it and *prints* its result in `Out[1]`.[1] Then IPython displays the `In [2]` prompt to show that it's waiting for you to enter your second snippet. For each new snippet, IPython adds 1 to the number in the square brackets. Each `In [1]` prompt in the book indicates that we've started a new interactive session. We generally do that for each new section of a chapter.

[1] In the next chapter, youll see that there are some cases in which `Out[]` is not displayed.

Let's evaluate a more complex expression:

lick here to view code image

```
In [2]: 5 * (12.7 - 4) / 2
Out[2]: 21.75
```

Python uses the asterisk (`*`) for multiplication and the forward slash (`/`) for division. As in mathematics, parentheses force the evaluation order, so the parenthesized expression `(12.7 - 4)` evaluates first, giving `8.7`. Next, `5 * 8.7` evaluates giving `43.5`. Then, `43.5 / 2` evaluates, giving the result `21.75`, which IPython displays in `Out[2]`. Whole numbers, like `5`, `4` and `2`, are called **integers**. Numbers with decimal points, like `12.7`, `43.5` and `21.75`, are called **floating-point numbers**.

Exiting Interactive Mode

To leave interactive mode, you can:

- Type the `exit` command at the current `In []` prompt and press *Enter* to exit immediately.
- Type the key sequence <*Ctrl*> + *d* (or <*control*> + *d*). This displays the prompt `"Do you really want to exit ([y]/n)?"`. The square brackets around `y` indicate that it's the default response—pressing *Enter* submits the default response and exits.
- Type <*Ctrl*> + *d* (or <*control*> + *d*) twice (macOS and Linux only).

1.5.2 Executing a Python Program Using the IPython Interpreter

In this section, you'll execute a script named `RollDieDynamic.py` that you'll write in hapter 6. The `.py` **extension** indicates that the file contains Python source code. The script `RollDieDynamic.py` simulates rolling a six-sided die. It presents a colorful animated visualization that dynamically graphs the frequencies of each die face.

Changing to This Chapter's Examples Folder

You'll find the script in the book's `ch01` source-code folder. In the Before You Begin section you extracted the `examples` folder to your user account's `Documents` folder. Each chapter

has a folder containing that chapter's source code. The folder is named `ch##`, where `##` is a two-digit chapter number from `01` to `17`. First, open your system's command-line window. Next, use the `cd` ("change directory") command to change to the `ch01` folder:

- On macOS/Linux, type `cd ~/Documents/examples/ch01`, then press *Enter*.
- On Windows, type `cd C:\Users\`*YourAccount*`\Documents\examples\ch01`, then press *Enter*.

Executing the Script

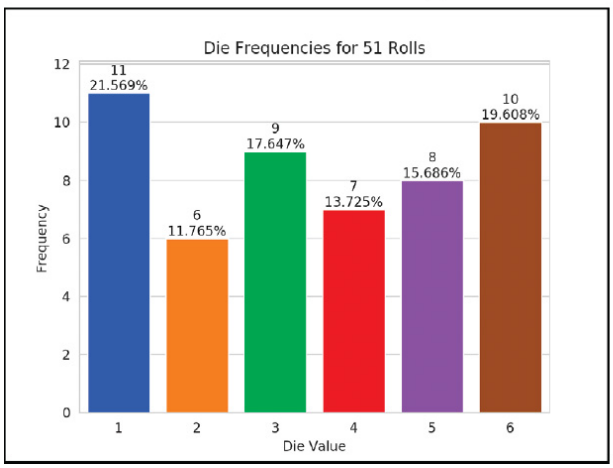
To execute the script, type the following command at the command line, then press *Enter*:

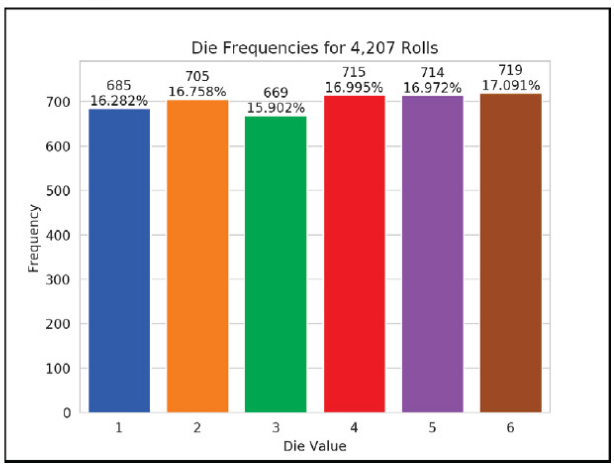
```
ipython RollDieDynamic.py 6000 1
```

The script displays a window, showing the visualization. The numbers `6000` and `1` tell this script the number of times to roll dice and how many dice to roll each time. In this case, we'll update the chart `6000` times for `1` die at a time.

For a six-sided die, the values 1 through 6 should each occur with "equal likelihood"—the probability of each is $1/6^{th}$ or about 16.667%. If we roll a die 6000 times, we'd expect about 1000 of each face. Like coin tossing, die rolling is *random*, so there could be some faces with fewer than 1000, some with 1000 and some with more than 1000. We took the screen captures below during the script's execution. This script uses randomly generated die values, so your results will differ. Experiment with the script by changing the value 1 to 100, 1000 and 10000. Notice that as the number of die rolls gets larger, the frequencies zero in on 16.667%. This is a phenomenon of the "aw of Large Numbers."

Roll the dice 6000 times and roll 1 die each time:
`ipython RollDieDynamic.py 6000 1`





Creating Scripts

Typically, you create your Python source code in an editor that enables you to type text. Using the editor, you type a program, make any necessary corrections and save it to your computer. **Integrated development environments (IDEs)** provide tools that support the entire

L

software-development process, such as editors, debuggers for locating logic errors that cause programs to execute incorrectly and more. Some popular Python IDEs include Spyder (which comes with Anaconda), PyCharm and Visual Studio Code.

Problems That May Occur at Execution Time

Programs often do not work on the first try. For example, an executing program might try to divide by zero (an illegal operation in Python). This would cause the program to display an error message. If this occurred in a script, you'd return to the editor, make the necessary corrections and re-execute the script to determine whether the corrections fixed the problem(s).

Errors such as division by zero occur as a program runs, so they're called **runtime errors** or **execution-time errors**. **Fatal runtime errors** cause programs to terminate immediately without having successfully performed their jobs. **Non-fatal runtime errors** allow programs to run to completion, often producing incorrect results.

1.5.3 Writing and Executing Code in a Jupyter Notebook

The Anaconda Python Distribution that you installed in the Before You Begin section comes with the **Jupyter Notebook**—an interactive, browser-based environment in which you can write and execute code and intermix the code with text, images and video. Jupyter Notebooks are broadly used in the data-science community in particular and the broader scientific community in general. They're the preferred means of doing Python-based data analytics studies and *reproducibly* communicating their results. The Jupyter Notebook environment supports a growing number of programming languages.

For your convenience, all of the book's source code also is provided in Jupyter Notebooks that you can simply load and execute. In this section, you'll use the **JupyterLab** interface, which enables you to manage your notebook files and other files that your notebooks use (like images and videos). As you'll see, JupyterLab also makes it convenient to write code, execute it, see the results, modify the code and execute it again.

You'll see that coding in a Jupyter Notebook is similar to working with IPython—in fact, Jupyter Notebooks use IPython by default. In this section, you'll create a notebook, add the code from ection 1.5.1 to it and execute that code.

Opening JupyterLab in Your Browser

To open JupyterLab, change to the `ch01` examples folder in your Terminal, shell or Anaconda Command Prompt (as in ection 1.5.2), type the following command, then press *Enter* (or *Return*):

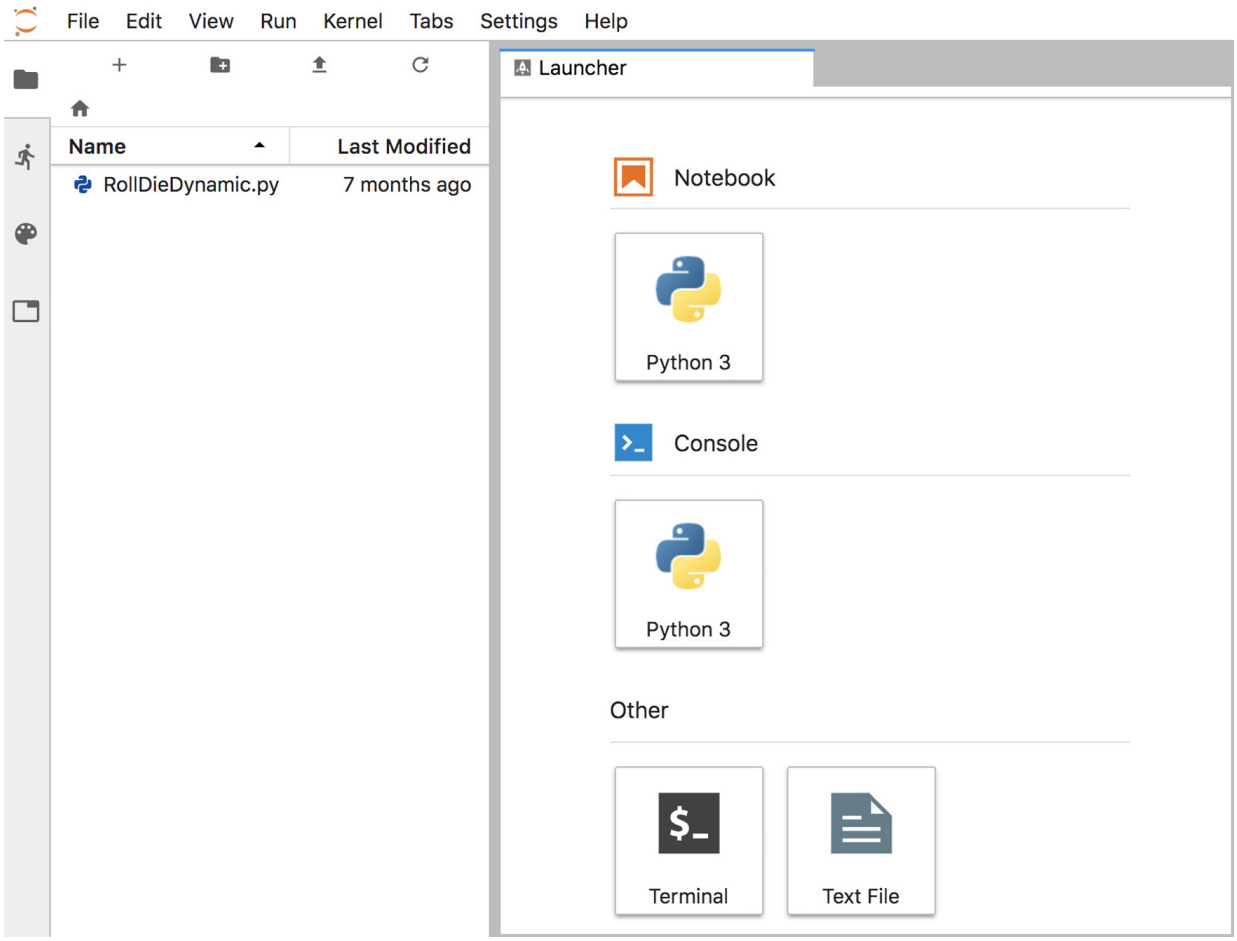
```
jupyter lab
```

This executes the Jupyter Notebook server on your computer and opens JupyterLab in your

S

default web browser, showing the `ch01` folder's contents in the **File Browser** tab

at the left side of the JupyterLab interface:



The Jupyter Notebook server enables you to load and run Jupyter Notebooks in your web browser. From the JupyterLab **Files** tab, you can double-click files to open them in the right side of the window where the **Launcher** tab is currently displayed. Each file you open appears as a separate tab in this part of the window. If you accidentally close your browser, you can reopen JupyterLab by entering the following address in your web browser

```
http://localhost:8888/lab
```

Creating a New Jupyter Notebook

In the **Launcher** tab under **Notebook**, click the **Python 3** button to create a new Jupyter Notebook named `Untitled.ipynb` in which you can enter and execute Python 3 code. The file extension **`.ipynb`** is short for IPython Notebook—the original name of the Jupyter Notebook.

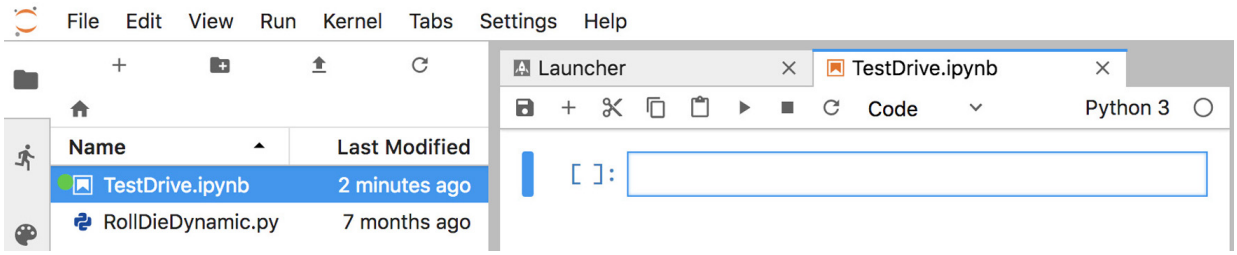
Renaming the Notebook

Rename `Untitled.ipynb` as `TestDrive.ipynb`:

1. Right-click the `Untitled.ipynb` tab and select **Rename Notebook**.

2. Change the name to `TestDrive.ipynb` and click **RENAME**.

The top of JupyterLab should now appear as follows:

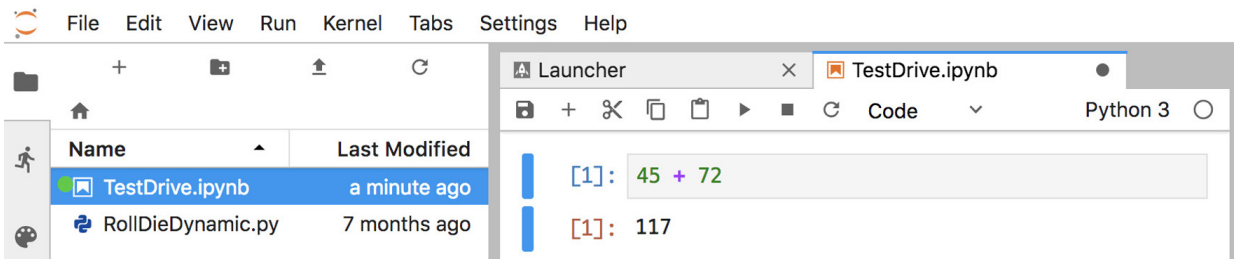


Evaluating an Expression

The unit of work in a notebook is a **cell** in which you can enter code snippets. By default, a new notebook contains one cell—the rectangle in the `TestDrive.ipynb` notebook—but you can add more. To the cell's left, the notation `[ ]:` is where the Jupyter Notebook will display the cell's snippet number *after* you execute the cell. Click in the cell, then type the expression

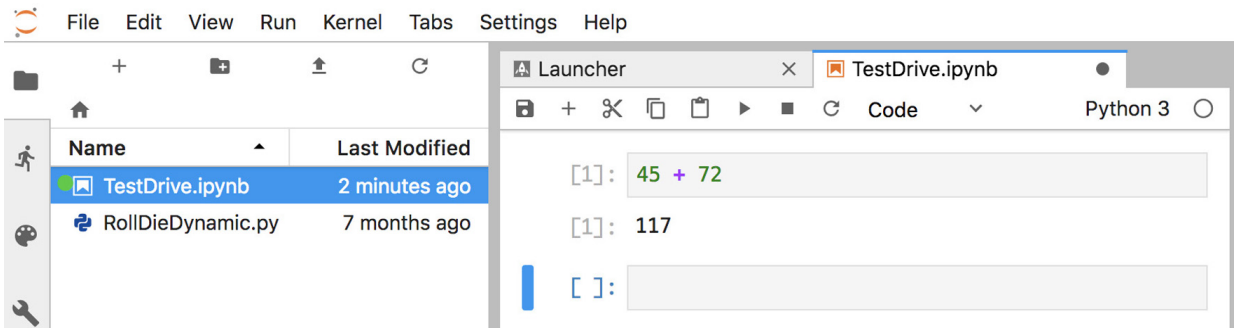
```
45 + 72
```

To execute the current cell's code, type *Ctrl* + *Enter* (or *control* + *Enter*). JupyterLab executes the code in IPython, then displays the results below the cell:



Adding and Executing Another Cell

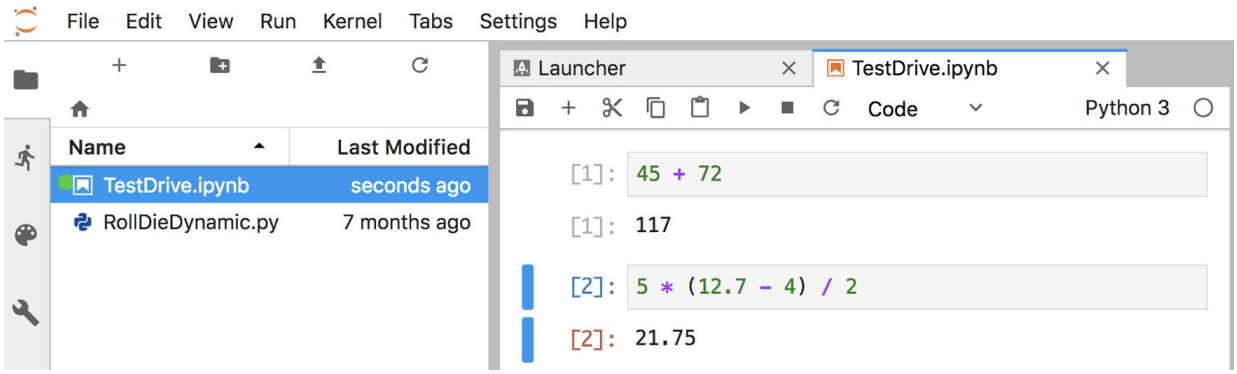
Let's evaluate a more complex expression. First, click the + button in the toolbar above the notebook's first cell—this adds a new cell below the current one:



Click in the new cell, then type the expression

```
5 * (12.7 - 4) / 2
```

and execute the cell by typing *Ctrl* + *Enter* (or *control* + *Enter*):



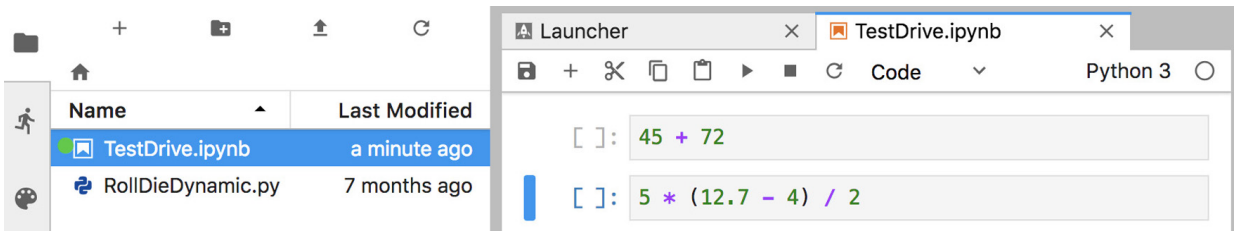
Saving the Notebook

If your notebook has unsaved changes, the **X** in the notebook's tab will change to . To save the notebook, select the **File** menu in JupyterLab (not at the top of your browser's window), then select **Save Notebook**.

Notebooks Provided with Each Chapter's Examples

For your convenience, each chapter's examples also are provided as ready-to-execute notebooks without their outputs. This enables you to work through them snippet-by-snippet and see the outputs appear as you execute each snippet.

So that we can show you how to load an existing notebook and execute its cells, let's reset the `TestDrive.ipynb` notebook to remove its output and snippet numbers. This will return it to a state like the notebooks we provide for the subsequent chapters' examples. From the **Kernel** menu select **Restart Kernel and Clear All Outputs...**, then click the **RESTART** button. The preceding command also is helpful whenever you wish to re-execute a notebook's snippets. The notebook should now appear as follows:



From the **File** menu, select **Save Notebook**, then click the `TestDrive.ipynb` tab's **X** button to close the notebook.

Opening and Executing an Existing Notebook

When you launch JupyterLab from a given chapter's examples folder, you'll be able to open notebooks from that folder or any of its subfolders. Once you locate a specific notebook, double-click it to open it. Open the `TestDrive.ipynb` notebook again now. Once a notebook is open, you can execute each cell individually, as you did earlier in this section, or you can execute the entire notebook at once. To do so, from the **Run** menu select **Run All**

Cells. The notebook will execute the cells in order, displaying each cell's output below that cell.

Closing JupyterLab

When you're done with JupyterLab, you can close its browser tab, then in the Terminal, shell or Anaconda Command Prompt from which you launched JupyterLab, type *Ctrl* + *c* (or *control* + *c*) twice.

JupyterLab Tips

While working in JupyterLab, you might find these tips helpful:

- If you need to enter and execute many snippets, you can execute the current cell *and* add a new one below it by typing *Shift* + *Enter*, rather than *Ctrl* + *Enter* (or *control* + *Enter*).
- As you get into the later chapters, some of the snippets you'll enter in Jupyter Notebooks will contain many lines of code. To display line numbers within each cell, select **Show line numbers** from JupyterLab's **View** menu.

More Information on Working with JupyterLab

JupyterLab has many more features that you'll find helpful. We recommend that you read the Jupyter team's introduction to JupyterLab at:

```
ttps://jupyterlab.readthedocs.io/en/stable/index.html
```

For a quick overview, click **Overview** under **GETTING STARTED**. Also, under **USER GUIDE** read the introductions to **The JupyterLab Interface**, **Working with Files**, **Text Editor** and **Notebooks** for many additional features.

1.6 THE CLOUD AND THE INTERNET OF THINGS

1.6.1 The Cloud

More and more computing today is done "in the cloud"—that is, distributed across the Internet worldwide. Many apps you use daily are dependent on **cloud-based services** that use massive clusters of computing resources (computers, processors, memory, disk drives, etc.) and databases that communicate over the Internet with each other and the apps you use. A service that provides access to itself over the Internet is known as a **web service**. As you'll see, using cloud-based services in Python often is as simple as creating a software object and interacting with it. That object then uses web services that connect to the cloud on your behalf.

Throughout the hapters 11– 6 examples, you'll work with many cloud-based services:

- In hapters 12 and 6, you'll use Twitter's web services (via the Python library Tweepy) to get information about specific Twitter users, search for tweets from the last seven days and receive streams of tweets as they occur—that is, in real time.

- In hapters 11 and 2, you'll use the Python library TextBlob to translate text between languages. Behind the scenes, TextBlob uses the Google Translate web service to perform those translations.

- In hapter 13, you'll use the IBM Watson's Text to Speech, Speech to Text and Translate services. You'll implement a traveler's assistant translation app that enables you to speak a question in English, transcribes the speech to text, translates the text to Spanish and speaks the Spanish text. The app then allows you to speak a Spanish response (in case you don't speak Spanish, we provide an audio file you can use), transcribes the speech to text, translates the text to English and speaks the English response. Via IBM Watson demos, you'll also experiment with many other Watson cloud-based services in hapter 13.

- In hapter 16, you'll work with Microsoft Azure's HDInsight service and other Azure web services as you implement big-data applications using Apache Hadoop and Spark. Azure is Microsoft's set of cloud-based services.

- In hapter 16, you'll use the Dweet.io web service to simulate an Internet-connected thermostat that publishes temperature readings online. You'll also use a web-based service to create a "dashboard" that visualizes the temperature readings over time and warns you if the temperature gets too low or too high.

- In hapter 16, you'll use a web-based dashboard to visualize a simulated stream of live sensor data from the PubNub web service. You'll also create a Python app that visualizes a PubNub simulated stream of live stock-price changes.

In most cases, you'll create Python objects that interact with web services on your behalf, hiding the details of how to access these services over the Internet.

Mashups

The applications-development methodology of *mashups* enables you to rapidly develop powerful software applications by combining (often free) complementary web services and other forms of information feeds—as you'll do in our IBM Watson traveler's assistant translation app. One of the first mashups combined the real-estate listings provided by `ttp://www.craigslist.org` with the mapping capabilities of Google Maps to offer maps that showed the locations of homes for sale or rent in a given area.

ProgrammableWeb (`ttp://www.programmableweb.com/`) provides a directory of over 20,750 web services and almost 8,000 mashups. They also provide how-to guides and sample code for working with web services and creating your own mashups. According to their website, some of the most widely used web services are Facebook, Google Maps, Twitter and

ouTube.

1.6.2 Internet of Things

The Internet is no longer just a network of *computers*—it's an **Internet of Things (IoT)**. A *thing* is any object with an IP address and the ability to send, and in some cases receive, data automatically over the Internet. Such *things* include:

- a car with a transponder for paying tolls,
- monitors for parking-space availability in a garage,
- a heart monitor implanted in a human,
- water quality monitors,
- a smart meter that reports energy usage,
- radiation detectors,
- item trackers in a warehouse,
- mobile apps that can track your movement and location,
- smart thermostats that adjust room temperatures based on weather forecasts and activity in the home, and
- intelligent home appliances.

According to `statista.com`, there are already over 23 billion IoT devices in use today, and there could be over 75 billion IoT devices in 2025.[2]

[2] `ttps://www.statista.com/statistics/471264/iot-number-of-connected-evices-worldwide/.`

1.7 HOW BIG IS BIG DATA?

For computer scientists and data scientists, data is now as important as writing programs. According to IBM, approximately 2.5 quintillion bytes (2.5 *exabytes*) of data are created daily, [3] and 90% of the world's data was created in the last two years. [4] According to IDC, the global data supply will reach 175 *zettabytes* (equal to 175 trillion gigabytes or 175 billion terabytes) annually by 2025. [5] Consider the following examples of various popular data measures.

[3] `ttps://www.ibm.com/blogs/watson/2016/06/welcome-to-the-world-of-a-/.`

[4] ttps://public.dhe.ibm.com/common/ssi/ecm/wr/en/wrl12345usen/watson-customer-engagement--watson-marketing-wr-other-papers-and-reports-rl12345usen-20170719.pdf.

[5] ttps://www.networkworld.com/article/3325397/storage/idc-expect-75-zettabytes-of-data-worldwide-by-2025.html.

egabytes (MB)

One megabyte is about one million (actually 2^{20}) bytes. Many of the files we use on a daily basis require one or more MBs of storage. Some examples include:

- MP3 audio files—High-quality MP3s range from 1 to 2.4 MB per minute. [6]

  [6] ttps://www.audiomountain.com/tech/audio-file-size.html.

- Photos—JPEG format photos taken on a digital camera can require about 8 to 10 MB per photo.

- Video—Smartphone cameras can record video at various resolutions. Each minute of video can require many megabytes of storage. For example, on one of our iPhones, the **Camera** settings app reports that 1080p video at 30 frames-per-second (FPS) requires 130 MB/minute and 4K video at 30 FPS requires 350 MB/minute.

Gigabytes (GB)

One gigabyte is about 1000 megabytes (actually 2^{30} bytes). A dual-layer DVD can store up to 8.5 GB [7], which translates to:

[7] ttps://en.wikipedia.org/wiki/DVD.

- as much as 141 hours of MP3 audio,
- approximately 1000 photos from a 16-megapixel camera,
- approximately 7.7 minutes of 1080p video at 30 FPS, or
- approximately 2.85 minutes of 4K video at 30 FPS.

The current highest-capacity Ultra HD Blu-ray discs can store up to 100 GB of video. [8] Streaming a 4K movie can use between 7 and 10 GB per hour (highly compressed).

[8] ttps://en.wikipedia.org/wiki/Ultra_HD_Blu-ray.

Terabytes (TB)

One terabyte is about 1000 gigabytes (actually 2^{40} bytes). Recent disk drives for desktop

omputers come in sizes up to 15 TB, [9] which is equivalent to:

[9] ttps://www.zdnet.com/article/worlds-biggest-hard-drive-meet-estern-digitals-15tb-monster/.

- approximately 28 years of MP3 audio,
- approximately 1.68 million photos from a 16-megapixel camera,
- approximately 226 hours of 1080p video at 30 FPS and
- approximately 84 hours of 4K video at 30 FPS.

Nimbus Data now has the largest solid-state drive (SSD) at 100 TB, which can store 6.67 times the 15-TB examples of audio, photos and video listed above. [0]

[0] ttps://www.cinema5d.com/nimbus-data-100tb-ssd-worlds-largest-ssd/.

Petabytes, Exabytes and Zettabytes

There are nearly four billion people online creating about 2.5 quintillion bytes of data each day [1]—that's 2500 petabytes (each petabyte is about 1000 terabytes) or 2.5 exabytes (each exabyte is about 1000 petabytes). According to a March 2016 *AnalyticsWeek* article, within five years there will be over 50 billion devices connected to the Internet (most of them through the Internet of Things, which we discuss in ections 1.6.2 and 6.8) and by 2020 we'll be producing 1.7 megabytes of new data every second *for every person on the planet*. [2] At today's numbers (approximately 7.7 billion people [3]), that's about

[1] ttps://public.dhe.ibm.com/common/ssi/ecm/wr/en/wrl12345usen/watson-customer-engagement--watson-marketing-wr-other-papers-and-reports-rl12345usen-20170719.pdf.

[2] ttps://analyticsweek.com/content/big-data-facts/.

[3] ttps://en.wikipedia.org/wiki/World_population.

- 13 petabytes of new data per second,
- 780 petabytes per minute,
- 46,800 petabytes (46.8 exabytes) per hour and
- 1,123 exabytes per day—that's 1.123 zettabytes (ZB) per day (each zettabyte is about 1000 exabytes).

That's the equivalent of over 5.5 million hours (over 600 years) of 4K video every day or

pproximately 116 billion photos every day!

Additional Big-Data Stats

For an entertaining real-time sense of big data, check out ttps://www.internetlivestats.com, with various statistics, including the numbers so far today of

- Google searches.
- Tweets.
- Videos viewed on YouTube.
- Photos uploaded on Instagram.

You can click each statistic to drill down for more information. For instance, they say over 250 *billion* tweets were sent in 2018.

Some other interesting big-data facts:

- Every hour, YouTube users upload 24,000 hours of video, and almost 1 billion hours of video are watched on YouTube every day. [4]

  [4] ttps://www.brandwatch.com/blog/youtube-stats/.

- Every second, there are 51,773 GBs (or 51.773 TBs) of Internet traffic, 7894 tweets sent, 64,332 Google searches and 72,029 YouTube videos viewed. [5]

  [5] ttp://www.internetlivestats.com/one-second.

- On Facebook each day there are 800 million "**likes**," [6] 60 million emojis are sent, [7] and there are over two billion searches of the more than 2.5 trillion Facebook posts since the site's inception. [8]

  [6] ttps://newsroom.fb.com/news/2017/06/two-billion-people-coming-ogether-on-facebook.

  [7] ttps://mashable.com/2017/07/17/facebook-world-emoji-day/.

  [8] ttps://techcrunch.com/2016/07/27/facebook-will-make-you-talk/.

- In June 2017, Will Marshall, CEO of Planet, said the company has 142 satellites that image the whole planet's land mass once per day. They add one million images and seven TBs of new data each day. Together with their partners, they're using machine learning on that data to improve crop yields, see how many ships are in a given port and track

eforestation. With respect to Amazon deforestation, he said: “Used to be we’d wake up after a few years and there’s a big hole in the Amazon. Now we can literally count every tree on the planet every day.” [9]

[9] ttps://www.bloomberg.com/news/videos/2017-06-30/learning-from-lanet-s-shoe-boxed-sized-satellites-video, June 30, 2017.

Domo, Inc. has a nice infographic called “Data Never Sleeps 6.0” showing how much data is generated *every minute*, including: [0]

[0] ttps://www.domo.com/learn/data-never-sleeps-6.

- 473,400 tweets sent.
- 2,083,333 Snapchat photos shared.
- 97,222 hours of Netflix video viewed.
- 12,986,111 million text messages sent.
- 49,380 Instagram posts.
- 176,220 Skype calls.
- 750,000 Spotify songs streamed.
- 3,877,140 Google searches.
- 4,333,560 YouTube videos watched.

Computing Power Over the Years

Data is getting more massive and so is the computing power for processing it. The performance of today’s processors is often measured in terms of **FLOPS (floating-point operations per second)**. In the early to mid-1990s, the fastest supercomputer speeds were measured in gigaflops (10^9 FLOPS). By the late 1990s, Intel produced the first teraflop (10^{12} FLOPS) supercomputers. In the early-to-mid 2000s, speeds reached hundreds of teraflops, then in 2008, IBM released the first petaflop (10^{15} FLOPS) supercomputer. Currently, the fastest supercomputer—the IBM Summit, located at the Department of Energy’s (DOE) Oak Ridge National Laboratory (ORNL)—is capable of 122.3 peta-flops. [1]

[1] ttps://en.wikipedia.org/wiki/FLOPS.

Distributed computing can link thousands of personal computers via the Internet to produce even more FLOPS. In late 2016, the Folding@home network—a distributed network in which people volunteer their personal computers’ resources for use in disease research and drug

esign [2]—was capable of over 100 petaflops. [3] Companies like IBM are now working toward supercomputers capable of exaflops (10^{18} FLOPS). [4]

[2] ttps://en.wikipedia.org/wiki/Folding@home.

[3] ttps://en.wikipedia.org/wiki/FLOPS.

[4] ttps://www.ibm.com/blogs/research/2017/06/supercomputing-weather-odel-exascale/.

The **quantum computers** now under development theoretically could operate at 18,000,000,000,000,000,000 times the speed of today's "conventional computers"! [5] This number is so extraordinary that in one second, a quantum computer theoretically could do staggeringly more calculations than the total that have been done by all computers since the world's first computer appeared. This almost unimaginable computing power could wreak havoc with blockchain-based cryptocurrencies like Bitcoin. Engineers are already rethinking blockchain to prepare for such massive increases in computing power. [6]

[5] ttps://medium.com/@n.biedrzycki/only-god-can-count-that-fast-the-orld-of-quantum-computing-406a0a91fcf4.

[6] ttps://singularityhub.com/2017/11/05/is-quantum-computing-an-xistential-threat-to-blockchain-technology/.

The history of supercomputing power is that it eventually works its way down from research labs, where extraordinary amounts of money have been spent to achieve those performance numbers, into "reasonably priced" commercial computer systems and even desktop computers, laptops, tablets and smartphones.

Computing power's cost continues to decline, especially with cloud computing. People used to ask the question, "How much computing power do I need on my system to deal with my *peak* processing needs?" Today, that thinking has shifted to "Can I quickly carve out on the cloud what I need *temporarily* for my most demanding computing chores?" You pay for only what you use to accomplish a given task.

Processing the World's Data Requires Lots of Electricity

Data from the world's Internet-connected devices is exploding, and processing that data requires tremendous amounts of energy. According to a recent article, energy use for processing data in 2015 was growing at 20% per year and consuming approximately three to five percent of the world's power. The article says that total data-processing power consumption could reach 20% by 2025. [7]

[7] ttps://www.theguardian.com/environment/2017/dec/11/tsunami-of-ata-could-consume--fifth-global-electricity-by-2025.

nother enormous electricity consumer is the blockchain-based cryptocurrency Bitcoin. Processing just one Bitcoin transaction uses approximately the same amount of energy as powering the average American home for a week! The energy use comes from the process Bitcoin "miners" use to prove that transaction data is valid. [8]

[8] ttps://motherboard.vice.com/en_us/article/ywbbpm/bitcoin-mining-lectricity-consumption--ethereum-energy-climate-change.

According to some estimates, a year of Bitcoin transactions consumes more energy than many countries. [9] Together, Bitcoin and Ethereum (another popular blockchain-based platform and cryptocurrency) consume more energy per year than Israel and almost as much as Greece. [0]

[9] ttps://digiconomist.net/bitcoin-energy-consumption.

[0] ttps://digiconomist.net/ethereum-energy-consumption.

Morgan Stanley predicted in 2018 that "the electricity consumption required to create cryptocurrencies this year could actually outpace the firm's projected global electric vehicle demand—in 2025." [1] This situation is unsustainable, especially given the huge interest in blockchain-based applications, even beyond the cryptocurrency explosion. The blockchain community is working on fixes. [2, 3]

[1] ttps://www.morganstanley.com/ideas/cryptocurrencies-global-tilities.

[2] ttps://www.technologyreview.com/s/609480/bitcoin-uses-massive-mounts-of-energybut-theres-a-plan-to-fix-it/.

[3] ttp://mashable.com/2017/12/01/bitcoin-energy/.

Big-Data Opportunities

The big-data explosion is likely to continue exponentially for years to come. With 50 billion computing devices on the horizon, we can only imagine how many more there will be over the next few decades. It's crucial for businesses, governments, the military and even individuals to get a handle on all this data.

It's interesting that some of the best writings about big data, data science, artificial intelligence and more are coming out of distinguished business organizations, such as J.P. Morgan, McKinsey and more. Big data's appeal to big business is undeniable given the rapidly accelerating accomplishments. Many companies are making significant investments and getting valuable results through technologies in this book, such as big data, machine learning, deep learning and natural-language processing. This is forcing competitors to invest as well, rapidly increasing the need for computing professionals with data-science and computer science experience. This growth is likely to continue for many years.

.7.1 Big Data Analytics

Data analytics is a mature and well-developed academic and professional discipline. The term "data analysis" was coined in 1962, [4] though people have been analyzing data using statistics for thousands of years going back to the ancient Egyptians. [5] Big data analytics is a more recent phenomenon—the term "big data" was coined around 2000. [6]

[4] ttps://www.forbes.com/sites/gilpress/2013/05/28/a-very-short-istory-of-data-science/.

[5] ttps://www.flydata.com/blog/a-brief-history-of-data-analysis/.

[6] ttps://bits.blogs.nytimes.com/2013/02/01/the-origins-of-big-data-n-etymological--detective-story/.

Consider four of the V's of big data [7, 8]:

[7] ttps://www.ibmbigdatahub.com/infographic/four-vs-big-data.

[8]There are lots of articles and papers that add many other V-words to this list.

1. Volume—the amount of data the world is producing is growing exponentially.
2. Velocity—the speed at which that data is being produced, the speed at which it moves through organizations and the speed at which data changes are growing quickly. [9, 10, 11]

   [9] ttps://www.zdnet.com/article/volume-velocity-and-variety-nderstanding-the-three-vs-of-big-data/.

   [10] ttps://whatis.techtarget.com/definition/3Vs.

   [11] ttps://www.forbes.com/sites/brentdykes/2017/06/28/big-data-orget-volume-and-variety--focus-on-velocity.
3. Variety—data used to be alphanumeric (that is, consisting of alphabetic characters, digits, punctuation and some special characters)—today it also includes images, audios, videos and data from an exploding number of Internet of Things sensors in our homes, businesses, vehicles, cities and more.
4. Veracity—the validity of the data—is it complete and accurate? Can we trust that data when making crucial decisions? Is it real?

Most data is now being created digitally in a *variety* of types, in extraordinary *volumes* and moving at astonishing *velocities*. Moore's Law and related observations have enabled us to store data economically and to process and move it faster—and all at rates growing exponentially over time. Digital data storage has become so vast in capacity, cheap and small

that we can now conveniently and economically retain *all* the digital data we're creating. [2] That's big data.

[2] ttp://www.lesk.com/mlesk/ksg97/ksg.html. [The following article pointed us to this Michael Lesk article: ttps://www.forbes.com/sites/gilpress/2013/05/28/a-very-short-istory-of-data-science/.]

The following Richard W. Hamming quote—although from 1962—sets the tone for the rest of this book:

"The purpose of computing is insight, not numbers." [3]

[3]Hamming, R. W., *Numerical Methods for Scientists and Engineers* (New York, NY., McGraw Hill, 1962). [The following article pointed us to Hammings book and his quote that we cited: ttps://www.forbes.com/sites/gilpress/2013/05/28/a-very-hort-history-of-data-science/.]

Data science is producing new, deeper, subtler and more valuable insights at a remarkable pace. It's truly making a difference. Big data analytics is an integral part of the answer. We address big data infrastructure in hapter 16 with hands-on case studies on NoSQL databases, Hadoop MapReduce programming, Spark, real-time Internet of Things (IoT) stream programming and more.

To get a sense of big data's scope in industry, government and academia, check out the high-resolution graphic. [4] You can click to zoom for easier readability:

[4]Turck, M., and J. Hao, Great Power, Great Responsibility: The 2018 Big Data & AI Landscape, ttp://mattturck.com/bigdata2018/.

```
ttp://mattturck.com/wp-content/uploads/2018/07/Matt_Turck_FirstMark_Big_Data_L
```

.7.2 Data Science and Big Data Are Making a Difference: Use Cases

The data-science field is growing rapidly because it's producing significant results that are making a difference. We enumerate data-science and big data use cases in the following table. We expect that the use cases and our examples throughout the book will inspire you to pursue new use cases in your career. Big-data analytics has resulted in improved profits, better customer relations, and even sports teams winning more games and championships while spending less on players. [5, 6, 7]

[5]Sawchik, T., *Big Data Baseball: Math, Miracles, and the End of a 20-Year Losing Streak* (New York, Flat Iron Books, 2015).

[6]Ayres, I., *Super Crunchers* (Bantam Books, 2007), pp. 710.

[7]Lewis, M., *Moneyball: The Art of Winning an Unfair Game* (W. W. Norton & Company, 2004).

ata-science use cases		
anomaly detection		
assisting people with disabilities		
auto-insurance risk prediction		
automated closed captioning		
automated image captions		predicting weather-sensitive product sales
automated investing	facial recognition	predictive analytics
autonomous ships	fitness tracking	preventative medicine
brain mapping	fraud detection	preventing disease outbreaks
caller identification	game playing	reading sign language
cancer diagnosis/treatment	genomics and healthcare	real-estate valuation
carbon emissions reduction	Geographic Information Systems (GIS)	recommendation systems
classifying handwriting	GPS Systems	reducing overbooking
computer vision	health outcome improvement	ride sharing
credit scoring	hospital readmission reduction	risk minimization
crime: predicting locations	human genome sequencing	robo financial advisors
	identity-theft prevention	security enhancements

rime: predicting recidivism

crime: predictive policing

crime: prevention

CRISPR gene editing

crop-yield improvement

customer churn

customer experience

customer retention

customer satisfaction

customer service

customer service agents

customized diets

cybersecurity

data mining

data visualization

detecting new viruses

diagnosing breast cancer

diagnosing heart disease

diagnostic medicine

immunotherapy

insurance pricing

intelligent assistants

Internet of Things (IoT) and medical device monitoring

Internet of Things and weather forecasting

inventory control

language translation

location-based services

loyalty programs

malware detection

mapping

marketing

marketing analytics

music generation

natural-language translation

new pharmaceuticals

opioid abuse prevention

personal assistants

personalized medicine

personalized shopping

phishing elimination

pollution reduction

precision medicine

predicting cancer survival

self-driving cars

sentiment analysis

sharing economy

similarity detection

smart cities

smart homes

smart meters

smart thermostats

smart traffic control

social analytics

social graph analysis

spam detection

spatial data analysis

sports recruiting and coaching

stock market forecasting

student performance assessment

summarizing text

telemedicine

terrorist attack prevention

theft prevention

travel recommendations

trend spotting

visual product search

c

disaster-victim identification	predicting disease outbreaks	voice recognition
drones	predicting health outcomes	voice search
dynamic driving routes	predicting student enrollments	weather forecasting
dynamic pricing		
electronic health records		
emotion detection		
energy-consumption reduction		

1.8 CASE STUDY—A BIG-DATA MOBILE APPLICATION

Google's Waze GPS navigation app, with its 90 million monthly active users,[8] is one of the most successful big-data apps. Early GPS navigation devices and apps relied on static maps and GPS coordinates to determine the best route to your destination. They could not adjust dynamically to changing traffic situations.

[8] ttps://www.waze.com/brands/drivers/.

Waze processes massive amounts of **crowdsourced data**—that is, the data that's continuously supplied by their users and their users' devices worldwide. They analyze this data as it arrives to determine the best route to get you to your destination in the least amount of time. To accomplish this, Waze relies on your smartphone's Internet connection. The app automatically sends location updates to their servers (assuming you allow it to). They use that data to dynamically re-route you based on current traffic conditions and to tune their maps. Users report other information, such as roadblocks, construction, obstacles, vehicles in breakdown lanes, police locations, gas prices and more. Waze then alerts other drivers in those locations.

Waze uses many technologies to provide its services. We're not privy to how Waze is implemented, but we infer below a list of technologies they probably use. You'll use many of these in hapters 11– 6. For example,

- Most apps created today use at least some open-source software. You'll take advantage of many open-source libraries and tools throughout this book.

- Waze communicates information over the Internet between their servers and their users' mobile devices. Today, such data often is transmitted in JSON (JavaScript Object Notation) format, which we'll introduce in hapter 9 and use in subsequent chapters. The JSON data is typically hidden from you by the libraries you use.

- Waze uses speech synthesis to speak driving directions and alerts to you, and speech recognition to understand your spoken commands. We use IBM Watson's speech-synthesis and speech-recognition capabilities in hapter 13.

- Once Waze converts a spoken natural-language command to text, it must determine the correct action to perform, which requires natural language processing (NLP). We present NLP in hapter 11 and use it in several subsequent chapters.

- Waze displays dynamically updated visualizations such as alerts and maps. Waze also enables you to interact with the maps by moving them or zooming in and out. We create dynamic visualizations with Matplotlib and Seaborn throughout the book, and we display interactive maps with Folium in hapters 12 and 6.

- Waze uses your phone as a streaming Internet of Things (IoT) device. Each phone is a GPS sensor that continuously streams data over the Internet to Waze. In hapter 16, we introduce IoT and work with simulated IoT streaming sensors.

- Waze receives IoT streams from millions of phones at once. It must process, store and analyze that data immediately to update your device's maps, to display and speak relevant alerts and possibly to update your driving directions. This requires massively parallel processing capabilities implemented with clusters of computers in the cloud. In hapter 6, we'll introduce various big-data infrastructure technologies for receiving streaming data, storing that big data in appropriate databases and processing the data with software and hardware that provide massively parallel processing capabilities.

- Waze uses artificial-intelligence capabilities to perform the data-analysis tasks that enable it to predict the best routes based on the information it receives. In hapters 14 and 5 we use machine learning and deep learning, respectively, to analyze massive amounts of data and make predictions based on that data.

- Waze probably stores its routing information in a graph database. Such databases can efficiently calculate shortest routes. We introduce graph databases, such as Neo4J, in hapter 16.

- Many cars are now equipped with devices that enable them to "see" cars and obstacles around them. These are used, for example, to help implement automated braking systems and are a key part of self-driving car technology. Rather than relying on users to report

bstacles and stopped cars on the side of the road, navigation apps could take advantage of cameras and other sensors by using deep-learning computer-vision techniques to analyze images "on the fly" and automatically report those items. We introduce deep learning for computer vision in hapter 15.

1.9 INTRO TO DATA SCIENCE: ARTIFICIAL INTELLIGENCE—AT THE INTERSECTION OF CS AND DATA SCIENCE

When a baby first opens its eyes, does it "see" its parent's faces? Does it understand any notion of what a face is—or even what a simple shape is? Babies must "learn" the world around them. That's what artificial intelligence (AI) is doing today. It's looking at massive amounts of data and learning from it. AI is being used to play games, implement a wide range of computer-vision applications, enable self-driving cars, enable robots to learn to perform new tasks, diagnose medical conditions, translate speech to other languages in near real time, create chatbots that can respond to arbitrary questions using massive databases of knowledge, and much more. Who'd have guessed just a few years ago that artificially intelligent self-driving cars would be allowed on our roads—or even become common? Yet, this is now a highly competitive area. The ultimate goal of all this learning is **artificial general intelligence**—an AI that can perform intelligence tasks as well as humans. This is a scary thought to many people.

Artificial-Intelligence Milestones

Several artificial-intelligence milestones, in particular, captured people's attention and imagination, made the general public start thinking that AI is real and made businesses think about commercializing AI:

- In a 1997 match between **IBM's DeepBlue** computer system and chess Grandmaster Gary Kasparov, DeepBlue became the first computer to beat a reigning world chess champion under tournament conditions. [9] IBM loaded DeepBlue with hundreds of thousands of grandmaster chess games. [0] DeepBlue was capable of using *brute force* to evaluate up to 200 million moves per second! [1] This is big data at work. IBM received the Carnegie Mellon University Fredkin Prize, which in 1980 offered $100,000 to the creators of the first computer to beat a world chess champion. [2]

  [9] ttps://en.wikipedia.org/wiki/Deep_Blue_versus_Garry_Kasparov.

  [0] ttps://en.wikipedia.org/wiki/Deep_Blue_(chess_computer).

  [1] ttps://en.wikipedia.org/wiki/Deep_Blue_(chess_computer).

  [2] ttps://articles.latimes.com/1997/jul/30/news/mn-17696.

- In 2011, **IBM's Watson** beat the two best human Jeopardy! players in a $1 million match. Watson simultaneously used hundreds of language-analysis techniques to locate

orrect answers in 200 million pages of content (including all of Wikipedia) requiring four terabytes of storage. [3, 4] Watson was trained with **machine learning** and **reinforcement-learning techniques**. [5] hapter 13 discusses IBM Watson and hapter 14 discusses machine-learning.

[3] ttps://www.techrepublic.com/article/ibm-watson-the-inside-story-of-how-the-jeopardy--winning-supercomputer-was-born-and-hat-it-wants-to-do-next/.

[4] ttps://en.wikipedia.org/wiki/Watson_(computer).

[5] ttps://www.aaai.org/Magazine/Watson/watson.php, *AI Magazine*, Fall 2010.

- Go—a board game created in China thousands of years ago [6]—is widely considered to be one of the most complex games ever invented with 10^{170} possible board configurations. [7] To give you a sense of how large a number that is, it's believed that there are (only) between 10^{78} and 10^{87} atoms in the known universe! [8, 9] In 2015, **AlphaGo**—created by Google's DeepMind group—used *deep learning with two neural networks to beat the European Go champion Fan Hui*. Go is considered to be a far more complex game than chess. hapter 15 discusses neural networks and deep learning.

[6] ttp://www.usgo.org/brief-history-go.

[7] ttps://www.pbs.org/newshour/science/google-artificial-intelligence-beats-champion--at-worlds-most-complicated-board-ame.

[8] ttps://www.universetoday.com/36302/atoms-in-the-universe/.

[9] ttps://en.wikipedia.org/wiki/Observable_universe#Matter_content.

- More recently, Google generalized its AlphaGo AI to create **AlphaZero**—a game-playing AI that *teaches itself to play other games*. In December 2017, AlphaZero learned the rules of and taught itself to play chess in less than four hours using reinforcement learning. It then beat the world champion chess program, Stockfish 8, in a 100-game match—winning or drawing every game. After *training itself* in Go for just eight hours, AlphaZero was able to play Go vs. its AlphaGo predecessor, winning 60 of 100 games. [0]

[0] ttps://www.theguardian.com/technology/2017/dec/07/alphazero-google-deepmind-ai-beats-champion-program-teaching-itself-to-play-our-hours.

A Personal Anecdote

When one of the authors, Harvey Deitel, was an undergraduate student at MIT in the mid-

960s, he took a graduate-level artificial-intelligence course with Marvin Minsky (to whom this book is dedicated), one of the founders of artificial intelligence (AI). Harvey:

Professor Minsky required a major term project. He told us to think about what intelligence is and to make a computer do something intelligent. Our grade in the course would be almost solely dependent on the project. No pressure!

*I researched the standardized IQ tests that schools administer to help evaluate their students' intelligence capabilities. Being a mathematician at heart, I decided to tackle the popular IQ-test problem of predicting the next number in a sequence of numbers of arbitrary length and complexity. I used interactive Lisp running on an early Digital Equipment Corporation PDP-1 and was able to get my sequence predictor running on some pretty complex stuff, handling challenges well beyond what I recalled seeing on IQ tests. Lisp's ability to manipulate arbitrarily long lists recursively was exactly what I needed to meet the project's requirements. Python offers recursion and generalized list processing (**hapter 5**).*

I tried the sequence predictor on many of my MIT classmates. They would make up number sequences and type them into my predictor. The PDP-1 would "think" for a while—often a long while—and almost always came up with the right answer.

Then I hit a snag. One of my classmates typed in the sequence 14, 23, 34 and 42. My predictor went to work on it, and the PDP-1 chugged away for a long time, failing to predict the next number. I couldn't get it either. My classmate told me to think about it overnight, and he'd reveal the answer the next day, claiming that it was a simple sequence. My efforts were to no avail.

The following day he told me the next number was 57, but I didn't understand why. So he told me to think about it overnight again, and the following day he said the next number was 125. That didn't help a bit—I was stumped. He said that the sequence was the numbers of the two-way crosstown streets of Manhattan. I cried, "foul," but he said it met my criterion of predicting the next number in a numerical sequence. My world view was mathematics—his was broader.

Over the years, I've tried that sequence on friends, relatives and professional colleagues. A few who spent time in Manhattan got it right. My sequence predictor needed a lot more than just mathematical knowledge to handle problems like this, requiring (a possibly vast) world knowledge.

Watson and Big Data Open New Possibilities

When Paul and I started working on this Python book, we were immediately drawn to IBM's Watson using big data and artificial-intelligence techniques like natural language processing (NLP) and machine learning to beat two of the world's best human Jeopardy! players. We realized that Watson could probably handle problems like the sequence predictor because it was loaded with the world's street maps and a whole lot more. That

C

het our appetite for digging in deep on big data and today's artificial-intelligence technologies, and helped shape hapters 11– 6 of this book.

It's notable that all of the data-science implementation case studies in hapters 11– 6 either are rooted in artificial intelligence technologies or discuss the big data hardware and software infrastructure that enables computer scientists and data scientists to implement leading-edge AI-based solutions effectively.

AI: A Field with Problems But No Solutions

For many decades, AI has been viewed as a field with problems but *no* solutions. That's because once a particular problem is solved people say, "Well, that's not intelligence, it's just a computer program that tells the computer exactly what to do." However, with machine learning (hapter 14) and deep learning (hapter 15) we're not pre-programming- solutions to *specific* problems. Instead, we're letting our computers solve problems by learning from data—and, typically, lots of it.

Many of the most interesting and challenging problems are being pursued with deep learning. Google alone has thousands of deep-learning projects underway and that number is growing quickly. [1, 2] As you work through this book, we'll introduce you to many edge-of-the-practice artificial intelligence, big data and cloud technologies.

[1] ttp://theweek.com/speedreads/654463/google-more-than-1000-rtificial-intelligence-projects-works.

[2] ttps://www.zdnet.com/article/google-says-exponential-growth-of-ai-s-changing-nature-of-compute/.

1.10 WRAP-UP

In this chapter, we introduced terminology and concepts that lay the groundwork for the Python programming you'll learn in hapters 2– 0 and the big-data, artificial-intelligence and cloud-based case studies we present in hapters 11– 6.

We reviewed object-oriented programming concepts and discussed why Python has become so popular. We introduced the Python Standard Library and various data-science libraries that help you avoid "reinventing the wheel." In subsequent chapters, you'll use these libraries to create software objects that you'll interact with to perform significant tasks with modest numbers of instructions.

You worked through three test-drives showing how to execute Python code with the IPython interpreter and Jupyter Notebooks. We introduced the Cloud and the Internet of Things (IoT), laying the groundwork for the contemporary applications you'll develop in hapters 1– 6.

We discussed just how big "big data" is and how quickly it's getting even bigger, and

resented a big-data case study on the Waze mobile navigation app, which uses many current technologies to provide dynamic driving directions that get you to your destination as quickly and as safely as possible. We mentioned where in this book you'll use many of those technologies. The chapter closed with our first Intro to Data Science section in which we discussed a key intersection between computer science and data science—artificial intelligence.

p

2. Introduction to Python Programming

Objectives

In this chapter, you'll:

- Continue using IPython interactive mode to enter code snippets and see their results immediately.
- Write simple Python statements and scripts.
- Create variables to store data for later use.
- Become familiar with built-in data types.
- Use arithmetic operators and comparison operators, and understand their precedence.
- Use single-, double- and triple-quoted strings.
- Use built-in function `print` to display text.
- Use built-in function `input` to prompt the user to enter data at the keyboard and get that data for use in the program.
- Convert text to integer values with built-in function `int`.
- Use comparison operators and the `if` statement to decide whether to execute a statement or group of statements.
- Learn about objects and Python's dynamic typing.
- Use built in function `type` to get an object's type

Outline

2.1 INTRODUCTION

In this chapter, we introduce Python programming and present examples illustrating key language features. We assume you've read the IPython Test-Drive in hapter 1, which introduced the IPython interpreter and used it to evaluate simple arithmetic expressions.

2.2 VARIABLES AND ASSIGNMENT STATEMENTS

You've used IPython's interactive mode as a calculator with expressions such as

```
In [1]: 45 + 72
Out[1]: 117
```

Let's create a variable named `x` that stores the integer `7`:

```
In [2]: x = 7
```

Snippet `[2]` is a **statement**. Each statement specifies a task to perform. The preceding statement creates `x` and uses the **assignment symbol (=)** to give `x` a value. Most

statements stop at the end of the line, though it's possible for statements to span more than one line. The following statement creates the variable `y` and assigns to it the value `3`:

```
In [3]: y = 3
```

You can now use the values of `x` and `y` in expressions:

```
In [4]: x + y
Out[4]: 10
```

Calculations in Assignment Statements

The following statement adds the values of variables `x` and `y` and assigns the result to the variable `total`, which we then display:

```
In [5]: total = x + y

In [6]: total
Out[6]: 10
```

The = symbol is not an operator. The right side of the = symbol always executes first, then the result is assigned to the variable on the symbol's left side.

Python Style

The *Style Guide for Python Code* [1] helps you write code that conforms to Python's coding conventions. The style guide recommends inserting one space on each side of the assignment symbol = and binary operators like + to make programs more readable.

[1] ttps://www.python.org/dev/peps/pep-0008/.

Variable Names

A variable name, such as `x`, is an **identifier**. Each identifier may consist of letters, digits and underscores (_) but may not begin with a digit. Python is *case sensitive*, so `number` and `Number` are *different* identifiers because one begins with a lowercase letter and the other begins with an uppercase letter.

Types

h

ach value in Python has a **type** that indicates the kind of data the value represents. You can view a value's type with Python's built-in **`type` function**, as in:

```
In [7]: type(x)
Out[7]: int

In [8]: type(10.5)
Out[8]: float
```

The variable `x` contains the integer value `7` (from snippet `[2]`), so Python displays `int` (short for integer). The value `10.5` is a floating-point number, so Python displays `float`.

2.3 ARITHMETIC

The following table summarizes the **arithmetic operators**, which include some symbols not used in algebra.

Python operation	Arithmetic operator	Algebraic expression	Python expression
Addition	`+`	$f + 7$	`f + 7`
Subtraction	`-`	$p - c$	`p - c`
Multiplication	`*`	$b \cdot m$	`b * m`
Exponentiation	`**`	x	`x ** y`
True division	`/`	x/y or $\frac{x}{y}$ or $x \div y$	`x / y`

E

Floor division	`//`	$\lfloor x/y \rfloor$ or $\left\lfloor \frac{x}{y} \right\rfloor$ or $\lfloor x \div y \rfloor$	`x // y`
Remainder (modulo)	`%`	$r \bmod s$	`r % s`

Multiplication (*)

Python uses the **asterisk (*) multiplication operator**:

```
In [1]: 7 * 4
Out[1]: 28
```

Exponentiation (**)

The **exponentiation (**) operator** raises one value to the power of another:

```
In [2]: 2 ** 10
Out[2]: 1024
```

To calculate the square root, you can use the exponent `1/2` (that is, `0.5`):

```
In [3]: 9 ** (1 / 2)
Out[3]: 3.0
```

True Division (/) vs. Floor Division (//)

True division (/) divides a numerator by a denominator and yields a floating-point number with a decimal point, as in:

```
In [4]: 7 / 4
Out[4]: 1.75
```

Floor division (//) divides a numerator by a denominator, yielding the highest *integer* that's not greater than the result. Python **truncates** (discards) the fractional part:

```
In [5]: 7 // 4
Out[5]: 1

In [6]: 3 // 5
Out[6]: 0

In [7]: 14 // 7
Out[7]: 2
```

In true division, `-13` divided by `4` gives `-3.25`:

```
In [8]: -13 / 4
Out[8]: -3.25
```

Floor division gives the closest integer that's *not greater than* `-3.25`—which is `-4`:

```
In [9]: -13 // 4
Out[9]: -4
```

Exceptions and Tracebacks

Dividing by zero with `/` or `//` is not allowed and results in an **exception**—a sign that a problem occurred:

lick here to view code image

```
In [10]: 123 / 0
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last
 ipython-input-10-cd759d3fcf39> in <module>()
----> 1 123 / 0

ZeroDivisionError: division   by zero
```

Python reports an exception with a **traceback**. This traceback indicates that an exception of type `ZeroDivisionError` occurred—most exception names end with `Error`. In interactive mode, the snippet number that caused the exception is specified

by the 10 in the line

```
<ipython-input-10-cd759d3fcf39> in <module>()
```

The line that begins with ----> shows the code that caused the exception. Sometimes snippets have more than one line of code—the 1 to the right of ----> indicates that line 1 within the snippet caused the exception. The last line shows the exception that occurred, followed by a colon (:) and an error message with more information about the exception:

```
ZeroDivisionError: division by zero
```

The "Files and Exceptions" chapter discusses exceptions in detail.

An exception also occurs if you try to use a variable that you have not yet created. The following snippet tries to add 7 to the undefined variable z, resulting in a NameError:

lick here to view code image

```
In [11]: z + 7
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last
 ipython-input-11-f2cdbf4fe75d> in <module>()
----> 1 z + 7

NameError: name 'z' is not   defined
```

Remainder Operator

Python's **remainder operator (%)** yields the remainder after the left operand is divided by the right operand:

```
In [12]: 17 % 5
Out[12]: 2
```

In this case, 17 divided by 5 yields a quotient of 3 and a remainder of 2. This operator is most commonly used with integers, but also can be used with other numeric types:

```
In [13]: 7.5 % 3.5
Out[13]: 0.5
```

€

Straight-Line Form

Algebraic notations such as

generally are not acceptable to compilers or interpreters. For this reason, algebraic expressions must be typed in **straight-line form** using Python's operators. The expression above must be written as `a / b` (or `a // b` for floor division) so that all operators and operands appear in a horizontal straight line.

Grouping Expressions with Parentheses

Parentheses group Python expressions, as they do in algebraic expressions. For example, the following code multiplies `10` times the quantity `5 + 3`:

```
In [14]: 10 * (5 + 3)
Out[14]: 80
```

Without these parentheses, the result is *different*:

```
In [15]: 10 * 5 + 3
Out[15]: 53
```

The parentheses are **redundant** (unnecessary) if removing them yields the *same* result.

Operator Precedence Rules

Python applies the operators in arithmetic expressions according to the following **rules of operator precedence**. These are generally the same as those in algebra:

1. Expressions in parentheses evaluate first, so parentheses may force the order of evaluation to occur in any sequence you desire. Parentheses have the highest level of precedence. In expressions with **nested parentheses**, such as `(a / (b - c))`, the expression in the *innermost* parentheses (that is, `b - c`) evaluates first.

2. Exponentiation operations evaluate next. If an expression contains several exponentiation operations, Python applies them from right to left.

3. Multiplication, division and modulus operations evaluate next. If an expression

contains several multiplication, true-division, floor-division and modulus operations, Python applies them from left to right. Multiplication, division and modulus are "on the same level of precedence."

4. Addition and subtraction operations evaluate last. If an expression contains several addition and subtraction operations, Python applies them from left to right. Addition and subtraction also have the same level of precedence.

For the complete list of operators and their precedence (in lowest-to-highest order), see

ttps://docs.python.org/3/reference/expressions.html#operator-precedence

Operator Grouping

When we say that Python applies certain operators from left to right, we are referring to the operators' **grouping**. For example, in the expression

```
a + b + c
```

the addition operators (+) group from left to right as if we parenthesized the expression as `(a + b) + c`. All Python operators of the same precedence group left-to-right except for the exponentiation operator (`**`), which groups right-to-left.

Redundant Parentheses

You can use redundant parentheses to group subexpressions to make the expression clearer. For example, the second-degree polynomial

```
y = a * x ** 2 + b * x + c
```

can be parenthesized, for clarity, as

lick here to view code image

```
y = (a * (x ** 2)) + (b * x) + c
```

Breaking a complex expression into a sequence of statements with shorter, simpler expressions also can promote clarity.

Operand Types

Each arithmetic operator may be used with integers and floating-point numbers. If both operands are integers, the result is an integer—except for the true-division (`/`) operator, which always yields a floating-point number. If both operands are floating-point numbers, the result is a floating-point number. Expressions containing an integer and a floating-point number are **mixed-type expressions**—these always produce floating-point results.

2.4 FUNCTION PRINT AND AN INTRO TO SINGLE- AND DOUBLE-QUOTED STRINGS

The built-in **`print` function** displays its argument(s) as a line of text:

lick here to view code image

```
In [1]: print('Welcome to Python!')
Welcome to Python!
```

In this case, the argument `'Welcome to Python!'` is a string—a sequence of characters enclosed in single quotes (`'`). Unlike when you evaluate expressions in interactive mode, the text that `print` displays here is not preceded by `Out[1]`. Also, `print` does not display a string's quotes, though we'll soon show how to display quotes in strings.

You also may enclose a string in double quotes (`"`), as in:

lick here to view code image

```
In [2]: print("Welcome to   Python!")
Welcome to Python!
```

Python programmers generally prefer single quotes. When `print` completes its task, it positions the screen cursor at the beginning of the next line.

Printing a Comma-Separated List of Items

The `print` function can receive a comma-separated list of arguments, as in:

lick here to view code image

```
In [3]: print('Welcome', 'to', 'Python!')
Welcome to Python!
```

C

t displays each argument separated from the next by a space, producing the same output as in the two preceding snippets. Here we showed a comma-separated list of strings, but the values can be of any type. We'll show in the next chapter how to prevent automatic spacing between values or use a different separator than space.

Printing Many Lines of Text with One Statement

When a backslash (`\`) appears in a string, it's known as the **escape character**. The backslash and the character immediately following it form an **escape sequence**. For example, `\n` represents the **newline character** escape sequence, which tells `print` to move the output cursor to the next line. The following snippet uses three newline characters to create several lines of output:i

lick here to view code image

```
In [4]: print('Welcome\nto\n\nPython!')
Welcome
to

Python!
```

Other Escape Sequences

The following table shows some common escape sequences.

Escape sequence	Description
`\n`	Insert a newline character in a string. When the string is displayed, for each newline, move the screen cursor to the beginning of the next line.
`\t`	Insert a horizontal tab. When the string is displayed, for each tab, move the screen cursor to the next tab stop.
`\\`	Insert a backslash character in a string.

C

\"	Insert a double quote character in a string.
\'	Insert a single quote character in a string.

Ignoring a Line Break in a Long String

You may also split a long string (or a long statement) over several lines by using the **\ continuation character** as the last character on a line to ignore the line break:

lick here to view code image

```
In [5]: print('this is a longer string, so we \
   ...: split it over two lines')
this is a longer string, so we split it over two lines
```

The interpreter reassembles the string's parts into a single string with no line break. Though the backslash character in the preceding snippet is inside a string, it's not the escape character because another character does not follow it.

Printing the Value of an Expression

Calculations can be performed in `print` statements:

lick here to view code image

```
In [6]: print('Sum is', 7 + 3)
Sum is 10
```

2.5 TRIPLE-QUOTED STRINGS

Earlier, we introduced strings delimited by a pair of single quotes (') or a pair of double quotes ("). **Triple-quoted strings** begin and end with three double quotes (""") or three single quotes ('''). The *Style Guide for Python Code* recommends three double quotes ("""). Use these to create:

C

- multiline strings,
- strings containing single or double quotes and
- **docstrings**, which are the recommended way to document the purposes of certain program components.

Including Quotes in Strings

In a string delimited by single quotes, you may include double-quote characters:

lick here to view code image

```
In [1]: print('Display "hi”   in quotes')
Display "hi” in quotes
```

but not single quotes:

lick here to view code image

```
In [2]: print('Display 'hi' in   quotes')
  File "<ipython-input-2-19bf596ccf72>",   line 1
    print('Display 'hi' in quotes')
                      ^
SyntaxError: invalid syntax
```

unless you use the `\'` escape sequence:

lick here to view code image

```
In [3]: print('Display \'hi\' in quotes')
Display 'hi' in quotes
```

Snippet `[2]` displayed a syntax error due to a single quote inside a single-quoted string. IPython displays information about the line of code that caused the syntax error and points to the error with a `^` symbol. It also displays the message `SyntaxError: invalid syntax`.

A string delimited by double quotes may include single quote characters:

lick here to view code image

C

```
In [4]: print("Display the name   O'Brien")
Display the name O'Brien
```

but not double quotes, unless you use the `\"` escape sequence:

lick here to view code image

```
In [5]: print("Display   \"hi\" in quotes")
Display "hi" in quotes
```

To avoid using `\'` and `\"` inside strings, you can enclose such strings in triple quotes:

lick here to view code image

```
In [6]: print("""Display   "hi" and 'bye' in quotes""")
Display "hi" and 'bye' in quotes
```

Multiline Strings

The following snippet assigns a multiline triple-quoted string to `triple_quoted_string`:

lick here to view code image

```
In [7]: triple_quoted_string = """This   is a triple-quoted
   ...: string that spans two   lines"""
```

IPython knows that the string is incomplete because we did not type the closing `"""` before we pressed *Enter*. So, IPython displays a **continuation prompt** `...:` at which you can input the multiline string's next line. This continues until you enter the ending `"""` and press *Enter*. The following displays `triple_quoted_string`:

lick here to view code image

```
In [8]: print(triple_quoted_string)
This is a triple-quoted
string that spans two lines
```

Python stores multiline strings with embedded newline characters. When we evaluate `triple_quoted_string` rather than printing it, IPython displays it in single quotes

C

with a `\n` character where you pressed *Enter* in snippet `[7]`. The quotes IPython displays indicate that `triple_quoted_string` is a string—they're not part of the string's contents:

lick here to view code image

```
In [9]: triple_quoted_string
Out[9]: 'This is a triple-quoted\nstring that spans two   lines'
```

2.6 GETTING INPUT FROM THE USER

The built-in **`input` function** requests and obtains user input:

lick here to view code image

```
In [1]: name = input("What's   your name? ")
What's your name? Paul

In [2]: name
Out[2]: 'Paul'

In [3]: print(name)
Paul
```

The snippet executes as follows:

- First, `input` displays its string argument—a prompt—to tell the user what to type and waits for the user to respond. We typed `Paul` and pressed *Enter*. We use **bold** text to distinguish the user's input from the prompt text that `input` displays.
- Function `input` then returns those characters as a string that the program can use. Here we assigned that string to the variable `name`.

Snippet `[2]` shows `name`'s value. Evaluating `name` displays its value in single quotes as `'Paul'` because it's a string. Printing `name` (in snippet `[3]`) displays the string without the quotes. If you enter quotes, they're part of the string, as in:

lick here to view code image

```
In [4]: name = input("What's   your name? ")
What's your name? 'Paul'
```

C

```
In [5]: name
Out[5]: "'Paul'"

In [6]: print(name)
'Paul'
```

Function `input` Always Returns a String

Consider the following snippets that attempt to read two numbers and add them:

lick here to view code image

```
In [7]: value1 = input('Enter first   number: ')
Enter first number: 7

In [8]: value2 = input('Enter second   number: ')
Enter second number: 3

In [9]: value1 + value2
Out[9]: '73'
```

Rather than adding the integers `7` and `3` to produce `10`, Python "adds" the *string* values `'7'` and `'3'`, producing the *string* `'73'`. This is known as **string concatenation**. It creates a new string containing the left operand's value followed by the right operand's value.

Getting an Integer from the User

If you need an integer, convert the string to an integer using the built-in **`int` function**:

lick here to view code image

```
In [10]: value = input('Enter an   integer: ')
Enter an integer: 7

In [11]: value = int(value)

In [12]: value
Out[12]: 7
```

We could have combined the code in snippets `[10]` and `[11]`:

lick here to view code image

C

```
In [13]: another_value = int(input('Enter   another integer: '))
Enter another integer: 13

In [14]: another_value
Out[14]: 13
```

Variables `value` and `another_value` now contain integers. Adding them produces an integer result (rather than concatenating them):

```
In [15]: value + another_value
Out[15]: 20
```

If the string passed to `int` cannot be converted to an integer, a `ValueError` occurs:

lick here to view code image

```
In [16]: bad_value = int(input('Enter   another integer: '))
Enter another integer: hello
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last
 ipython-input-16-cd36e6cf8911> in <module>()
----> 1 bad_value = int(input('Enter   another integer: '))

ValueError: invalid literal   for int() with base 10: 'hello'
```

Function `int` also can convert a floating-point value to an integer:

```
In [17]: int(10.5)
Out[17]: 10
```

To convert strings to floating-point numbers, use the built-in **`float` function**.

2.7 DECISION MAKING: THE `IF` STATEMENT AND COMPARISON OPERATORS

A **condition** is a Boolean expression with the value **`True`** or **`False`**. The following determines whether `7` is greater than `4` and whether `7` is less than `4`:

```
In [1]: 7 > 4
Out[1]: True
```

€

```
In [2]: 7 < 4
Out[2]: False
```

`True` and `False` are Python keywords. Using a keyword as an identifier causes a `Syntax-Error`. `True` and `False` are each capitalized.

You'll often create conditions using the **comparison operators** in the following table:

Algebraic operator	Python operator	Sample condition	Meaning
>	>	`x > y`	`x` is greater than `y`
<	<	`x < y`	`x` is less than `y`
≥	>=	`x >= y`	`x` is greater than or equal to `y`
≤	<=	`x <= y`	`x` is less than or equal to `y`
=	==	`x == y`	`x` is equal to `y`
≠	!=	`x != y`	`x` is not equal to `y`

Operators >, <, >= and <= all have the same precedence. Operators == and != both have the same precedence, which is lower than that of >, <, >= and <=. A syntax error occurs when any of the operators ==, !=, >= and <= contains spaces between its pair of symbols:

lick here to view code image

```
In [3]: 7 > = 4
  File "<ipython-input-3-5c6e2897f3b3>",   line 1
    7 > = 4
        ^
SyntaxError: invalid syntax
```

Another syntax error occurs if you reverse the symbols in the operators `!=`, `>=` and `<=` (by writing them as `=!`, `=>` and `=<`).

Making Decisions with the `if` Statement: Introducing Scripts

We now present a simple version of the **`if` statement**, which uses a condition to decide whether to execute a statement (or a group of statements). Here we'll read two integers from the user and compare them using six consecutive `if` statements, one for each comparison operator. If the condition in a given `if` statement is `True`, the corresponding `print` statement executes; otherwise, it's skipped.

IPython interactive mode is helpful for executing brief code snippets and seeing immediate results. When you have many statements to execute as a group, you typically write them as a **script** stored in a file with the `.py` (short for Python) extension—such as `fig02_01.py` for this example's script. Scripts are also called programs. For instructions on locating and executing the scripts in this book, see hapter 1's IPython Test-Drive.

Each time you execute this script, three of the six conditions are `True`. To show this, we execute the script three times—once with the first integer *less than* the second, once with the *same* value for both integers and once with the first integer *greater than* the second. The three sample executions appear after the script

Each time we present a script like the one below, we introduce it before the figure, then explain the script's code after the figure. We show line numbers for your convenience—these are not part of Python. IDEs enable you to choose whether to display line numbers. To run this example, change to this chapter's `ch02` examples folder, then enter:

```
ipython fig02_01.py
```

or, if you're in IPython already, you can use the command:

C

```
run fig02_01.py
```

lick here to view code image

```
# fig02_01.py
"""Comparing integers using if statements and   comparison operators."""

print('Enter two integers, and I will tell you',
      'the relationships they satisfy.')

# read first integer
number1 =   int(input('Enter first integer: '))

# read second integer
number2 =   int(input('Enter second integer: '))

if number1 == number2:
    print(number1,   'is equal to', number2)

if number1 != number2:
    print(number1,   'is not equal to', number2)

if number1 < number2:
    print(number1,   'is less than', number2)

if number1 > number2:
    print(number1,   'is greater than', number2)

if number1 <= number2:
    print(number1,   'is less than or equal to', number2)

if number1 >= number2:
    print(number1,   'is greater than or equal to', number2)
```

lick here to view code image

```
Enter two integers and I will tell you the relationships they   satisfy.
Enter first integer: 37
Enter second integer: 42
37 is not equal to 42
37 is less than 42
37 is less than or equal to 42
```

lick here to view code image

C

```
Enter two integers and I will tell you the relationships they   satisfy.
Enter first integer: 7
Enter second integer: 7
7 is equal to 7
7 is less than or equal to 7
7 is greater than or equal to 7
```

lick here to view code image

```
Enter two integers and I will tell you the relationships they   satisfy.
Enter first integer: 54
Enter second integer: 17
54 is not equal to 17
54 is greater than 17
54 is greater than or equal to 17
```

Comments

Line 1 begins with the hash character (**#**), which indicates that the rest of the line is a **comment**:

```
# fig02_01.py
```

For easy reference, we begin each script with a comment indicating the script's file name. A comment also can begin to the right of the code on a given line and continue until the end of that line.

Docstrings

The *Style Guide for Python Code* states that each script should start with a docstring that explains the script's purpose, such as the one in line 2:

```
"""Comparing integers using if statements and comparison operators."""
```

For more complex scripts, the docstring often spans many lines. In later chapters, you'll use docstrings to describe script components you define, such as new functions and new types called classes. We'll also discuss how to access docstrings with the IPython help mechanism.

C

Blank Lines

Line 3 is a blank line. You use blank lines and space characters to make code easier to read. Together, blank lines, space characters and tab characters are known as **white space**. Python ignores most white space—you'll see that some indentation is required.

Splitting a Lengthy Statement Across Lines

Lines 4–5

lick here to view code image

```
print('Enter two integers, and I will tell you',
      'the relationships they satisfy.')
```

display instructions to the user. These are too long to fit on one line, so we broke them into two strings. Recall that you can display several values by passing to `print` a comma-separated list—`print` separates each value from the next with a space.

Typically, you write statements on one line. You may spread a lengthy statement over several lines with the `\` continuation character. Python also allows you to split long code lines in parentheses without using continuation characters (as in lines 4–5). This is the preferred way to break long code lines according to the *Style Guide for Python Code*. Always choose breaking points that make sense, such as after a comma in the preceding call to `print` or before an operator in a lengthy expression.

Reading Integer Values from the User

Next, lines 8 and 11 use the built-in `input` and `int` functions to prompt for and read two integer values from the user.

`if` Statements

The `if` statement in lines 13–14

lick here to view code image

```
if number1 == number2:
    print(number1, 'is equal to', number2)
```

uses the == comparison operator to determine whether the values of variables `number1` and `number2` are equal. If so, the condition is `True`, and line 14 displays a

C

ine of text indicating that the values are equal. If any of the remaining `if` statements' conditions are `True` (lines 16, 19, 22, 25 and 28), the corresponding `print` displays a line of text.

Each `if` statement consists of the keyword `if`, the condition to test, and a colon (`:`) followed by an indented body called a **suite**. Each suite must contain one or more statements. Forgetting the colon (`:`) after the condition is a common syntax error.

Suite Indentation

Python requires you to indent the statements in suites. The *Style Guide for Python Code* recommends four-space indents—we use that convention throughout this book. You'll see in the next chapter that incorrect indentation can cause errors.

Confusing == and =

Using the assignment symbol (=) instead of the equality operator (==) in an `if` statement's condition is a common syntax error. To help avoid this, read == as "is equal to" and = as "is assigned." You'll see in the next chapter that using == in place of = in an assignment statement can lead to subtle problems.

Chaining Comparisons

You can chain comparisons to check whether a value is in a range. The following comparison determines whether `x` is in the range `1` through `5`, inclusive:

```
In [1]: x = 3

In [2]: 1 <= x <= 5
Out[2]: True

In [3]: x = 10

In [4]: 1 <= x <= 5
Out[4]: False
```

Precedence of the Operators We've Presented So Far

The precedence of the operators introduced in this chapter is shown below:

Operators	Grouping	Type

`()`	left to right	parentheses
`**`	right to left	exponentiation
`* / // %`	left to right	multiplication, true division, floor division, remainder
`+ -`	left to right	addition, subtraction
`> <= < >=`	left to right	less than, less than or equal, greater than, greater than or equal
`== !=`	left to right	equal, not equal

The table lists the operators top-to-bottom in decreasing order of precedence. When writing expressions containing multiple operators, confirm that they evaluate in the order you expect by referring to the operator precedence chart at

ttps://docs.python.org/3/reference/expressions.html#operator-precedence

2.8 OBJECTS AND DYNAMIC TYPING

Values such as `7` (an integer), `4.1` (a floating-point number) and `'dog'` are all objects. Every object has a type and a value:

```
In [1]: type(7)
Out[1]: int

In [2]: type(4.1)
```

h

```
Out[2]: float

In [3]: type('dog')
Out[3]: str
```

An object's value is the data stored in the object. The snippets above show objects of built-in types **`int`** (for integers), **`float`** (for floating-point numbers) and **`str`** (for strings).

Variables Refer to Objects

Assigning an object to a variable **binds** (associates) that variable's name to the object. As you've seen, you can then use the variable in your code to access the object's value:

```
In [4]: x = 7

In [5]: x + 10
Out[5]: 17

In [6]: x
Out[6]: 7
```

After snippet `[4]`'s assignment, the variable `x` **refers** to the integer object containing `7`. As shown in snippet `[6]`, snippet `[5]` does not change `x`'s value. You can change `x` as follows:

```
In [7]: x = x + 10

In [8]: x
Out[8]: 17
```

Dynamic Typing

Python uses **dynamic typing**—it determines the type of the object a variable refers to while executing your code. We can show this by rebinding the variable `x` to different objects and checking their types:

```
In [9]: type(x)
Out[9]: int

In [10]: x = 4.1

In [11]: type(x)
Out[11]: float
```

```
In [12]: x = 'dog'

In [13]: type(x)
Out[13]: str
```

Garbage Collection

Python creates objects in memory and removes them from memory as necessary. After snippet `[10]`, the variable `x` now refers to a `float` object. The integer object from snippet `[7]` is no longer bound to a variable. As we'll discuss in a later chapter, Python automatically removes such objects from memory. This process—called **garbage collection**—helps ensure that memory is available for new objects you create.

2.9 INTRO TO DATA SCIENCE: BASIC DESCRIPTIVE STATISTICS

In data science, you'll often use statistics to describe and summarize your data. Here, we begin by introducing several such **descriptive statistics**, including:

- **minimum**—the smallest value in a collection of values.
- **maximum**—the largest value in a collection of values.
- **range**—the range of values from the minimum to the maximum.
- **count**—the number of values in a collection.
- **sum**—the total of the values in a collection.

We'll look at determining the *count* and *sum* in the next chapter. **Measures of dispersion** (also called **measures of variability**), such as *range*, help determine how spread out values are. Other measures of dispersion that we'll present in later chapters include *variance* and *standard deviation*.

Determining the Minimum of Three Values

First, let's show how to determine the minimum of three values manually. The following script prompts for and inputs three values, uses `if` statements to determine the minimum value, then displays it.

lick here to view code image

C

```
1 # fig02_02.py
2 """Find the minimum of three   values."""
3
4 number1 =   int(input('Enter first integer: '))
5 number2 =   int(input('Enter second integer: '))
6 number3 =   int(input('Enter third integer: '))
7
8 minimum =   number1
9
10 if number2 < minimum:
11     minimum =   number2
12
13 if number3 < minimum:
14     minimum =   number3
15
16 print('Minimum value is', minimum)
```

lick here to view code image

```
Enter first integer: 12
Enter second integer: 27
Enter third integer: 36
Minimum value is 12
```

lick here to view code image

```
Enter first integer: 27
Enter second integer: 12
Enter third integer: 36
Minimum value is 12
```

lick here to view code image

```
Enter first integer: 36
Enter second integer: 27
Enter third integer: 12
Minimum value is 12
```

After inputting the three values, we process one value at a time:

- First, we assume that `number1` contains the smallest value, so line 8 assigns it to the variable `minimum`. Of course, it's possible that `number2` or `number3` contains

C

the actual smallest value, so we still must compare each of these with `minimum`.

- The first `if` statement (lines 10–11) then tests `number2 < minimum` and if this condition is `True` assigns `number2` to `minimum`.
- The second `if` statement (lines 13–14) then tests `number3 < minimum`, and if this condition is `True` assigns `number3` to `minimum`.

Now, `minimum` contains the smallest value, so we display it. We executed the script three times to show that it always finds the smallest value regardless of whether the user enters it first, second or third.

Determining the Minimum and Maximum with Built-In Functions `min` and `max`

Python has many built-in functions for performing common tasks. Built-in functions **`min`** and **`max`** calculate the minimum and maximum, respectively, of a collection of values:

```
In [1]: min(36, 27, 12)
Out[1]: 12

In [2]: max(36, 27, 12)
Out[2]: 36
```

The functions `min` and `max` can receive any number of arguments.

Determining the Range of a Collection of Values

The *range* of values is simply the minimum through the maximum value. In this case, the range is 12 through 36. Much data science is devoted to getting to know your data. Descriptive statistics is a crucial part of that, but you also have to understand how to interpret the statistics. For example, if you have 100 numbers with a range of 12 through 36, those numbers could be distributed evenly over that range. At the opposite extreme, you could have clumping with 99 values of 12 and one 36, or one 12 and 99 values of 36.

Functional-Style Programming: Reduction

Throughout this book, we introduce various *functional-style programming* capabilities. These enable you to write code that can be more concise, clearer and easier to **debug**—that is, find and correct errors. The `min` and `max` functions are examples of

functional-style programming concept called **reduction**. They reduce a collection of values to a *single* value. Other reductions you'll see include the sum, average, variance and standard deviation of a collection of values. You'll also see how to define custom reductions.

Upcoming Intro to Data Science Sections

In the next two chapters, we'll continue our discussion of basic descriptive statistics with *measures of central tendency*, including *mean*, *median* and *mode*, and *measures of dispersion*, including *variance* and *standard deviation*.

2.10 WRAP-UP

This chapter continued our discussion of arithmetic. You used variables to store values for later use. We introduced Python's arithmetic operators and showed that you must write all expressions in straight-line form. You used the built-in function `print` to display data. We created single-, double- and triple-quoted strings. You used triple-quoted strings to create multiline strings and to embed single or double quotes in strings.

You used the `input` function to prompt for and get input from the user at the keyboard. We used the functions `int` and `float` to convert strings to numeric values. We presented Python's comparison operators. Then, you used them in a script that read two integers from the user and compared their values using a series of `if` statements.

We discussed Python's dynamic typing and used the built-in function `type` to display an object's type. Finally, we introduced the basic descriptive statistics minimum and maximum and used them to calculate the range of a collection of values. In the next chapter, we present Python's control statements.

a

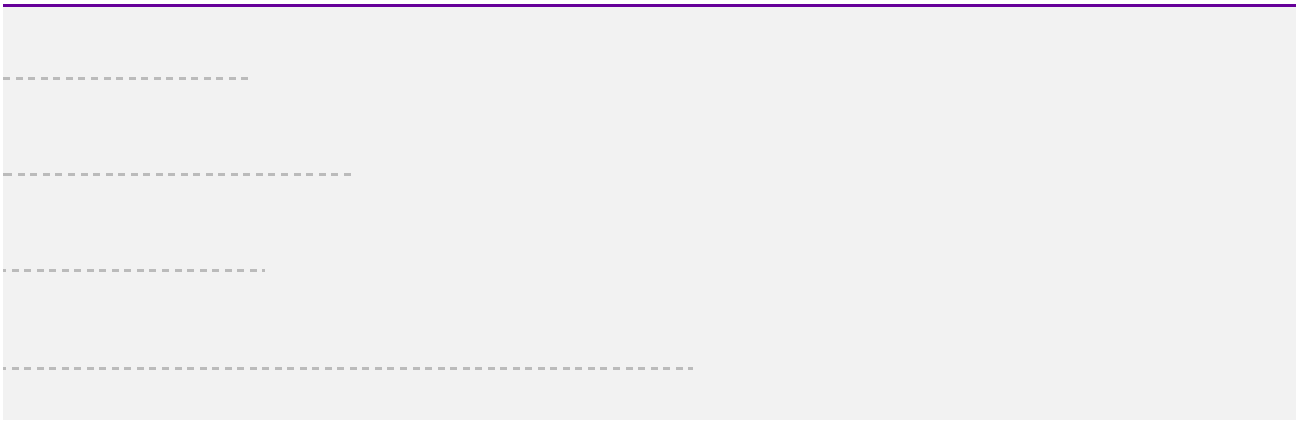
3. Control Statements

Objectives

In this chapter, you'll:

- Make decisions with `if`, `if else` and `if elif else`.
- Execute statements repeatedly with `while` and `for`.
- Shorten assignment expressions with augmented assignments.
- Use the `for` statement and the built-in `range` function to repeat actions for a sequence of values.
- Perform sentinel-controlled iteration with `while`.
- Create compound conditions with the Boolean operators `and`, `or` and `not`.
- Stop looping with `break`.
- Force the next iteration of a loop with `continue`.
- Use functional-style programming features to write scripts that are more concise, clearer, easier to debug and easier to parallelize.

Outline



3.1 INTRODUCTION

In this chapter, we present Python's control statements—`if`, `if else`, `if elif else`, `while`, `for`, `break` and `continue`. You'll use the `for` statement to perform sequence-controlled- iteration—you'll see that the number of items in a sequence of item determines the `for` statement's number of iterations. You'll use the built-in function `range` to generate sequences of integers.

We'll show sentinel-controlled iteration with the `while` statement. You'll use the Python Standard Library's `Decimal` type for precise monetary calculations. We'll format data in f-strings (that is, format strings) using various format specifiers. We'll also show the Boolean operators `and`, `or` and `not` for creating compound conditions. In the Intro to Data Science section, we'll consider measures of central tendency—mean, median and mode—using the Python Standard Library's `statistics` module.

.2 CONTROL STATEMENTS

Python provides three selection statements that execute code based on a condition that evaluates to either `True` or `False`:

- The `if` statement performs an action if a condition is `True` or skips the action if the condition is `False`.
- The **`if...else` statement** performs an action if a condition is `True` or performs a *different* action if the condition is `False`.
- The **`if...elif...else` statement** performs one of many different actions, depending on the truth or falsity of *several* conditions.

Anywhere a single action can be placed, a group of actions can be placed.

Python provides two **iteration statements**—`while` and `for`:

- The **`while` statement** repeats an action (or a group of actions) as long as a condition remains `True`.
- The **`for` statement** repeats an action (or a group of actions) for every item in a sequence of items.

Keywords

The words `if`, `elif`, `else`, `while`, `for`, `True` and `False` are Python keywords. Using a keyword as an identifier such as a variable name is a syntax error. The following table lists Python's keywords.

Python keywords				
`and`	`as`	`assert`	`async`	`await`
`break`	`class`	`continue`	`def`	`del`

elif	else	except	False	finally
for	from	global	if	import
in	is	lambda	None	nonlocal
not	or	pass	raise	return
True	try	while	with	yield

3.3 `if` STATEMENT

Let's execute a Python `if` statement:

lick here to view code image

```
In [1]: grade = 85
In [2]: if grade >= 60:
   ...:     print('Passed')
   ...:
Passed
```

The condition `grade >= 60` is `True`, so the indented `print` statement in the `if`'s suite displays `'Passed'`.

Suite Indentation

Indenting a suite is required; otherwise, an `IndentationError` syntax error occurs:

lick here to view code image

```
In [3]: if grade >= 60:
   ...: print('Passed')  # statement is not indented properly
  File "<ipython-input-3-f42783904220>",   line 2
    print('Passed')  # statement is   not indented properly
```

C

```
        ^
IndentationError: expected   an indented block
```

An `IndentationError` also occurs if you have more than one statement in a suite and those statements do not have the *same* indentation:

lick here to view code image

```
In [4]: if grade >= 60:
   ...:     print('Passed')  # indented 4 spaces
   ...:   print('Good job!')  # incorrectly indented only two spaces
  File <ipython-input-4-8c0d75c127bf>, line 3
    print('Good job!')  #   incorrectly indented only two spaces
    ^
IndentationError: unindent does not match any outer indentation level
```

Sometimes error messages may not be clear. The fact that Python calls attention to the line is usually enough for you to figure out what's wrong. Apply indentation conventions uniformly throughout your code—programs that are not uniformly indented are hard to read.

Every Expression Can Be Interpreted as Either `True` or `False`

You can base decisions on *any* expression. A nonzero value is `True`. Zero is `False`:

lick here to view code image

```
In [5]: if 1:
   ...:     print('Nonzero values are   true, so this will print')
   ...:
Nonzero values are true, so this will print

In [6]: if 0:
   ...:     print('Zero is false, so this will not print')

In [7]:
```

Strings containing characters are `True` and empty strings (`''`, `""` or `"""""""`) are `False`.

Confusing == and =

Using the equality operator == instead of = in an assignment statement can lead to subtle problems. For example, in this session, snippet `[1]` defined `grade` with the

C

assignment:

```
grade = 85
```

If instead we accidentally wrote:

```
grade == 85
```

then `grade` would be undefined and we'd get a `NameError`. If `grade` had been defined before the preceding statement, then `grade == 85` would simply evaluate to `True` or `False`, and not perform an assignment. This is a logic error.

3.4 `if else` AND `if elif else` STATEMENTS

The `if else` statement executes different suites, based on whether a condition is `True` or `False`:

lick here to view code image

```
In [1]: grade = 85

In [2]: if grade >= 60:
   ...:     print('Passed')
   ...: else:
   ...:     print('Failed')
   ...:
Passed
```

The condition above is `True`, so the `if` suite displays `'Passed'`. Note that when you press *Enter* after typing `print('Passed')`, IPython indents the next line four spaces. You must delete those four spaces so that the `else:` suite correctly aligns under the `i` in `if`.

The following code assigns `57` to the variable `grade`, then shows the `if else` statement again to demonstrate that only the `else` suite executes when the condition is `False`:

lick here to view code image

```
In [3]: grade = 57
```

C

```
In [4]: if grade >= 60:
   ...:     print('Passed')
   ...: else:
   ...:     print('Failed')
   ...:
Failed
```

Use the up and down arrow keys to navigate backwards and forwards through the current interactive session's snippets. Pressing *Enter* re-executes the snippet that's displayed. Let's set `grade` to `99`, press the up arrow key twice to recall the code from snippet `[4]`, then press *Enter* to re-execute that code as snippet `[6]`. Every recalled snippet that you execute gets a new ID:

lick here to view code image

```
In [5]: grade = 99

In [6]: if grade >= 60:
   ...:     print('Passed')
   ...: else:
   ...:     print('Failed')
   ...:
Passed
```

Conditional Expressions

Sometimes the suites in an `if else` statement assign different values to a variable, based on a condition, as in:

lick here to view code image

```
In [7]: grade = 87

In [8]: if grade >= 60:
   ...:     result = 'Passed'
   ...: else:
   ...:     result = 'Failed'
   ...:
```

We can then `print` or evaluate that variable:

```
In [9]: result
Out[9]: 'Passed'
```

C

You can write statements like snippet [8] using a concise **conditional expression**:

lick here to view code image

```
In [10]: result = ('Passed' if grade >= 60 else 'Failed')

In [11]: result
Out[11]: 'Passed'
```

The parentheses are not required, but they make it clear that the statement assigns the conditional expression's value to `result`. First, Python evaluates the condition `grade >= 60`:

- If it's `True`, snippet [10] assigns to `result` the value of the expression to the *left* of `if`, namely `'Passed'`. The `else` part does not execute.
- If it's `False`, snippet [10] assigns to `result` the value of the expression to the *right* of `else`, namely `'Failed'`.

In interactive mode, you also can evaluate the conditional expression directly, as in:

lick here to view code image

```
In [12]: 'Passed' if grade >= 60 else 'Failed'
Out[12]: 'Passed'
```

Multiple Statements in a Suite

The following code shows two statements in the `else` suite of an `if...else` statement:

lick here to view code image

```
In [13]: grade = 49

In [14]: if grade >= 60:
    ...:     print('Passed')
    ...: else:
    ...:     print('Failed')
    ...:     print('You must take   this course again')
    ...:
Failed
You must take this course again
```

C

In this case, `grade` is less than `60`, so *both* statements in the `else`'s suite execute.

If you do not indent the second `print`, then it's not in the `else`'s suite. So, that statement *always* executes, possibly creating strange incorrect output:

lick here to view code image

```
In [15]: grade = 100

In [16]: if grade >= 60:
    ...:     print('Passed')
    ...: else:
    ...:     print('Failed')
    ...: print('You must take this course again')
    ...:
Passed
You must take this course again
```

`if...elif...else` Statement

You can test for many cases using the **`if...elif...else` statement**. The following code displays "A" for grades greater than or equal to 90, "B" for grades in the range 80–89, "C" for grades 70–79, "D" for grades 60–69 and "F" for all other grades. Only the action for the *first* `True` condition executes. Snippet `[18]` displays `C`, because `grade` is `77`:

```
In [17]: grade = 77

In [18]: if grade >= 90:
    ...:     print('A')
    ...: elif grade >= 80:
    ...:     print('B')
    ...: elif grade >= 70:
    ...:     print('C')
    ...: elif grade >= 60:
    ...:     print('D')
    ...: else:
    ...:     print('F')
    ...:
C
```

The first condition—`grade >= 90`—is `False`, so `print('A')` is skipped. The second condition—`grade >= 80`—also is `False`, so `print('B')` is skipped. The third condition—`grade >= 70`—is `True`, so `print('C')` executes. Then all the remaining

code in the `if...elif...else` statement is skipped. An `if...elif...else` is faster than separate `if` statements, because condition testing stops as soon as a condition is `True`.

`else` Is Optional

The `else` in the `if...elif...else` statement is optional. Including it enables you to handle values that do not satisfy *any* of the conditions. When an `if...elif` statement without an `else` tests a value that does not make any of its conditions `True`, the program does not execute any of the statement's suites—the next statement in sequence after the `if...elif...` statement executes. If you specify the `else`, you must place it *after* the last `elif`; otherwise, a `SyntaxError` occurs.

Logic Errors

The incorrectly indented code segment in snippet `[16]` is an example of a *nonfatal logic error*. The code executes, but it produces incorrect results. For a *fatal logic error in a script*, an exception occurs (such as a `ZeroDivisionError` from an attempt to divide by 0), so Python displays a traceback, then terminates the script. A *fatal error in interactive mode* terminates only the current snippet—then IPython waits for your next input.

3.5 `while` STATEMENT

The **`while` statement** allows you to *repeat* one or more actions while a condition remains `True`. Let's use a `while` statement to find the first power of 3 larger than 50:

lick here to view code image

```
In [1]: product = 3

In [2]: while product <= 50:
   ...:     product = product * 3
   ...:

In [3]: product
Out[3]: 81
```

Snippet `[3]` evaluates `product` to see its value, `81`, which is the first power of `3` larger than `50`.

Something in the `while` statement's suite must change `product`'s value, so the

C

condition eventually becomes `False`. Otherwise, an infinite loop occurs. In applications executed from a Terminal, Anaconda Command Prompt or shell, type *Ctrl* + *c* or *control* + *c* to terminate an infinite loop. IDEs typically have a toolbar button or menu option for stopping a program's execution.

3.6 `for` STATEMENT

The **`for` statement** allows you to *repeat* an action or several actions for each item in a **sequence** of items. For example, a string is a sequence of individual characters. Let's display `'Programming'` with its characters separated by two spaces:

lick here to view code image

```
In [1]: for character in 'Programming':
   ...:     print(character, end='  ')
   ...:
P  r  o  g  r  a  m  m  i  n  g
```

The `for` statement executes as follows:

- Upon entering the statement, it assigns the `'P'` in `'Programming'` to the **target** variable between keywords `for` and `in`—in this case, `character`.
- Next, the statement in the suite executes, displaying `character`'s value followed by two spaces—we'll say more about this momentarily.
- After executing the suite, Python assigns to `character` the next item in the sequence (that is, the `'r'` in `'Programming'`), then executes the suite again.
- This continues while there are more items in the sequence to process. In this case, the statement terminates after displaying the letter `'g'`, followed by two spaces.

Using the target variable in the suite, as we did here to display its value, is common but not required.

Function `print`'s `end` Keyword Argument

The built-in function `print` displays its argument(s), then moves the cursor to the next line. You can change this behavior with the argument **`end`**, as in

C

```
print(character, end='  ')
```

which displays `character`'s value followed by two spaces. So, all the characters display horizontally on the *same* line. Python calls `end` a **keyword argument**, but `end` itself is not a Python keyword. Keyword arguments are sometimes called **named arguments**. The `end` keyword argument is optional. If you do not include it, `print` uses a newline (`'\n'`) by default. The *Style Guide for Python Code* recommends placing no spaces around a keyword argument's =.

Function `print`'s `sep` Keyword Argument

You can use the keyword argument **`sep`** (short for separator) to specify the string that appears *between* the items that `print` displays. When you do not specify this argument, `print` uses a space character by default. Let's display three numbers, each separated from the next by a comma and a space, rather than just a space:

lick here to view code image

```
In [2]: print(10, 20, 30, sep=', ')
10, 20, 30
```

To remove the default spaces, use `sep=''` (that is, an empty string).

3.6.1 Iterables, Lists and Iterators

The sequence to the right of the `for` statement's `in` keyword must be an **iterable**—that is, an object from which the `for` statement can take one item at a time until no more items remain. Python has other iterable sequence types besides strings. One of the most common is a **list**, which is a comma-separated collection of items enclosed in square brackets (`[` and `]`). The following code totals five integers in a list:

lick here to view code image

```
In [3]: total = 0

In [4]: for number in [2, -3, 0, 17, 9]:
   ...:     total = total + number
   ...:

In [5]: total
Out[5]: 25
```

C

Each sequence has an **iterator**. The `for` statement uses the iterator "behind the scenes" to get each consecutive item until there are no more to process. The iterator is like a bookmark—it always knows where it is in the sequence, so it can return the next item when it's called upon to do so. We cover lists in detail in the "Sequences: Lists and Tuples" chapter. There, you'll see that the order of the items in a list matters and that a list's items are **mutable** (that is, modifiable).

3.6.2 Built-In `range` Function

Let's use a `for` statement and the built-in **`range` function** to iterate precisely `10` times, displaying the values from `0` through `9`:

lick here to view code image

```
In [6]: for counter in range(10):
   ...:     print(counter, end=' ')
   ...:
0 1 2 3 4 5 6 7 8 9
```

The function call `range(10)` creates an iterable object that represents a sequence of consecutive integers starting from `0` and continuing up to, but *not* including, the argument value (`10`)—in this case, `0`, `1`, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`. The `for` statement exits when it finishes processing the last integer that `range` produces. Iterators and iterable objects are two of Python's *functional-style programming* features. We'll introduce more of these throughout the book.

Off-By-One Errors

A common type of off-by-one error occurs when you assume that `range`'s argument value is included in the generated sequence. For example, if you provide `9` as `range`'s argument when trying to produce the sequence `0` through `9`, `range` generates only `0` through `8`.

3.7 AUGMENTED ASSIGNMENTS

Augmented assignments abbreviate assignment expressions in which the same variable name appears on the left and right of the assignment's =, as `total` does in:

lick here to view code image

```
for number in [1, 2, 3, 4, 5]:
```

C

```
        total = total + number
```

Snippet [2] reimplements this using an **addition augmented assignment (+=) statement**:

lick here to view code image

```
In [1]: total = 0

In [2]: for number in [1, 2, 3, 4, 5]:
   ...:     total += number  # add number to total
   ...:

In [3]: total
Out[3]: 15
```

The += expression in snippet [2] first adds number's value to the current total, then stores the new value in total. The table below shows sample augmented assignments:

Augmented assignment	Sample expression	Explanation	Assigns
Assume: c = 3, d = 5, e = 4, f = 2, g = 9, h = 12			
+=	c += 7	c = c + 7	10 to c
-=	d -= 4	d = d - 4	1 to d
*=	e *= 5	e = e * 5	20 to e
**=	f **= 3	f = f ** 3	8 to f
/=	g /= 2	g = g / 2	4.5 to g

C

//=	g //= 2	g = g // 2	4 to g
%=	h %= 9	h = h % 9	3 to h

3.8 SEQUENCE-CONTROLLED ITERATION; FORMATTED STRINGS

This section and the next solve two class-averaging problems. Consider the following requirements statement:

A class of ten students took a quiz. Their grades (integers in the range 0 – 100) are 98, 76, 71, 87, 83, 90, 57, 79, 82, 94. Determine the class average on the quiz.

The following script for solving this problem keeps a running total of the grades, calculates the average and displays the result. We placed the 10 grades in a list, but you could input the grades from a user at the keyboard (as we'll do in the next example) or read them from a file (as you'll see how to do in the "Files and Exceptions" chapter). We show how to read data from SQL and NoSQL databases in hapter 16.

lick here to view code image

```
1 # class_average.py
2 """Class average program with sequence-controlled iteration."""
3
4 # initialization phase
5 total = 0  # sum of grades
6 grade_counter =  0
7 grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94]  # list of 10 grades
8
9 # processing phase
10 for grade in grades:
11     total +=  grade  # add current grade to the running total
12     grade_counter  += 1  # indicate that  one more grade was process
13
14 # termination phase
15 average = total / grade_counter
16 print(f'Class average is {average}')
```

C

lick here to view code image

```
Class average is 81.7
```

Lines 5–6 create the variables `total` and `grade_counter` and initialize each to `0`. Line 7

lick here to view code image

```
grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94]  # list of 10 grades
```

creates the variable `grades` and initializes it with a list of 10 integer grades.

The `for` statement processes each `grade` in the list `grades`. Line 11 adds the current `grade` to the `total`. Then, line 12 adds `1` to the variable `grade_counter` to keep track of the number of grades processed so far. Iteration terminates when all 10 grades in the list have been processed. The *Style Guide for Python Code* recommends placing a blank line above and below each control statement (as in lines 8 and 13). When the `for` statement terminates, line 15 calculates the average and line 16 displays it. Later in this chapter, we use functional-style programming to calculate the average of a list's items more concisely.

Introduction to Formatted Strings

Line 16 uses the following simple **f-string** (short for **formatted string**) to format this script's result by inserting the value of `average` into a string:

lick here to view code image

```
f'Class average is {average}'
```

The letter `f` before the string's opening quote indicates it's an f-string. You specify where to insert values by using *placeholders* delimited by curly braces (`{` and `}`). The placeholder

```
{average}
```

converts the variable `average`'s value to a string representation, then replaces

C

`{average}` with that **replacement text**. Replacement-text expressions may contain values, variables or other expressions, such as calculations or function calls. In line 16, we could have used `total / grade_counter` in place of `average`, eliminating the need for line 15.

3.9 SENTINEL-CONTROLLED ITERATION

Let's generalize the class-average problem. Consider the following requirements statement:

Develop a class-averaging program that processes an arbitrary number of grades each time the program executes.

The requirements statement does not state what the grades are or how many there are, so we're going to have the user enter the grades. The program processes an arbitrary number of grades. The user enters grades one at a time until all the grades have been entered, then enters a *sentinel value* (also called a *signal value*, a *dummy value* or a *flag value*) to indicate that there are no more grades.

Implementing Sentinel-Controlled Iteration

The following script solves the class average problem with sentinel-controlled iteration. Notice that we test for the possibility of division by zero. If undetected, this would cause a fatal logic error. In the "Files and Exceptions" chapter, we write programs that recognize such exceptions and take appropriate actions.

lick here to view code image

```
1 # class_average_sentinel.py
2 """Class average program with sentinel-controlled iteration."""
3
4 # initialization phase
5 total = 0  # sum of grades
6 grade_counter = 0  # number of grades   entered
7
8 # processing phase
9 grade = int(input('Enter grade, -1 to end: '))  # get one grade
10
11 while grade != -1:
12     total += grade
13     grade_counter += 1
14     grade = int(input('Enter grade, -1 to end: '))
15
16 # termination phase
17 if grade_counter != 0:
```

C

```
    average = total / grade_counter
    print(f'Class average is {average:.2f}')
else:
    print('No grades were entered')
```

lick here to view code image

```
Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 72
Enter grade, -1 to end: -1
Class average is 85.67
```

Program Logic for Sentinel-Controlled Iteration

In sentinel-controlled iteration, the program reads the first value (line 9) before reaching the `while` statement. The value input in line 9 determines whether the program's flow of control should enter the `while`'s suite (lines 12–14). If the condition in line 11 is `False`, the user entered the sentinel value (`-1`), so the suite does not execute because the user did not enter any grades. If the condition is `True`, the suite executes, adding the `grade` value to the `total` and incrementing the `grade_counter`.

Next, line 14 inputs another grade from the user and the condition (line 11) is tested again, using the most recent `grade` entered by the user. The value of `grade` is always input immediately before the program tests the `while` condition, so we can determine whether the value just input is the sentinel before processing that value as a grade.

When the sentinel value is input, the loop terminates, and the program does not add –1 to `total`. In a sentinel-controlled loop that performs user input, any prompts (lines 9 and 14) should remind the user of the sentinel value.

Formatting the Class Average with Two Decimal Places

This example formatted the class average with two digits to the right of the decimal point. In an f-string, you can optionally follow a replacement-text expression with a colon (`:`) and a **format specifier** that describes how to format the replacement text. The format specifier `.2f` (line 19) formats the `average` as a floating-point number (`f`) with two digits to the right of the decimal point (`.2`). In this example, the sum of the grades was 257, which, when divided by 3, yields 85.666666666.... Formatting the `average` with `.2f` *rounds* it to the hundredths position, producing the replacement

C

text `85.67`. An average with only one digit to the right of the decimal point would be formatted with a **trailing zero** (e.g., `85.50`). The chapter "Strings: A Deeper Look" discusses many more string-formatting features.

3.10 BUILT-IN FUNCTION `RANGE`: A DEEPER LOOK

Function `range` also has two- and three-argument versions. As you've seen, `range`'s one-argument version produces a sequence of consecutive integers from `0` up to, but not including, the argument's value. Function `range`'s two-argument version produces a sequence of consecutive integers from its first argument's value up to, but not including, the second argument's value, as in:

lick here to view code image

```
In [1]: for number in range(5, 10):
   ...:     print(number, end=' ')
   ...:
5 6 7 8 9
```

Function `range`'s three-argument version produces a sequence of integers from its first argument's value up to, but not including, the second argument's value, *incrementing* by the third argument's value, which is known as the **step**:

lick here to view code image

```
In [2]: for number in range(0, 10, 2):
   ...:     print(number, end=' ')
   ...:
0 2 4 6 8
```

If the third argument is negative, the sequence progresses from the first argument's value *down* to, but not including the second argument's value, *decrementing* by the third argument's value, as in:

lick here to view code image

```
In [3]: for number in range(10, 0, -2):
   ...:     print(number, end=' ')
   ...:
10 8 6 4 2
```

C

3.11 USING TYPE `DECIMAL` FOR MONETARY AMOUNTS

In this section, we introduce `Decimal` capabilities for precise monetary calculations. If you're in banking or other fields that require "to-the-penny" accuracy, you should investigate `Decimal`'s capabilities in depth.

For most scientific and other mathematical applications that use numbers with decimal points, Python's built-in floating-point numbers work well. For example, when we speak of a "normal" body temperature of 98.6, we do not need to be precise to a large number of digits. When we view the temperature on a thermometer and read it as 98.6, the actual value may be 98.5999473210643. The point here is that calling this number 98.6 is adequate for most body-temperature applications.

Floating-point values are stored in binary format (we introduced binary in the first chapter and discuss it in depth in the online "Number Systems" appendix). Some floating-point values are represented only approximately when they're converted to binary. For example, consider the variable `amount` with the dollars-and-cents value `112.31`. If you display `amount`, it appears to have the exact value you assigned to it:

```
In [1]: amount = 112.31

In [2]: print(amount)
112.31
```

However, if you print `amount` with `20` digits of precision to the right of the decimal point, you can see that the actual floating-point value in memory is not exactly `112.31` —it's only an approximation:

lick here to view code image

```
In [3]: print(f'{amount:.20f}')
112.31000000000000227374
```

Many applications require *precise* representation of numbers with decimal points. Institutions like banks that deal with millions or even billions of transactions per day have to tie out their transactions "to the penny." Floating-point numbers can represent some but not all monetary amounts with to-the-penny precision.

The **Python Standard Library** [1] provides many predefined capabilities you can use in your Python code to avoid "reinventing the wheel." For monetary calculations and other applications that require precise representation and manipulation of numbers

C

with decimal points, the Python Standard Library provides type **`Decimal`**, which uses a special coding scheme to solve the problem of to-the-penny precision. That scheme requires additional memory to hold the numbers and additional processing time to perform calculations but provides the precision required for monetary calculations. Banks also have to deal with other issues such as using a *fair rounding algorithm* when they're calculating daily interest on accounts. Type `Decimal` offers such capabilities. [2]

[1] ttps://docs.python.org/3.7/library/index.html.

[2] For more `decimal` module features, visit ttps://docs.python.org/3.7/library/decimal.html.

Importing Type `Decimal` from the `decimal` Module

We've used several *built-in types*—**`int`** (for integers, like `10`), **`float`** (for floating-point numbers, like `7.5`) and **`str`** (for strings like `'Python'`). The `Decimal` type is not built into Python. Rather, it's part of the Python Standard Library, which is divided into groups of related capabilities called **modules**. The **`decimal`** module defines type `Decimal` and its capabilities.

To use type `Decimal`, you must first **`import`** the entire `decimal` module, as in

```
import decimal
```

and refer to the `Decimal` type as `decimal.Decimal`, or you must indicate a specific capability to import using **`from import`**, as we do here:

lick here to view code image

```
In [4]: from decimal import Decimal
```

This `imports` only the type `Decimal` from the `decimal` module so that you can use it in your code. We'll discuss other `import` forms beginning in the next chapter.

Creating `Decimal`s

You typically create a `Decimal` from a string:

lick here to view code image

```
In [5]: principal = Decimal('1000.00')

In [6]: principal
Out[6]: Decimal('1000.00')

In [7]: rate = Decimal('0.05')

In [8]: rate
Out[8]: Decimal('0.05')
```

We'll soon use these variables `principal` and `rate` in a compound-interest calculation.

`Decimal` Arithmetic

`Decimal`s support the standard arithmetic operators `+`, `-`, `*`, `/`, `//`, `**` and `%`, as well as the corresponding augmented assignments:

```
In [9]: x = Decimal('10.5')

In [10]: y = Decimal('2')

In [11]: x + y
Out[11]: Decimal('12.5')

In [12]: x // y
Out[12]: Decimal('5')

In [13]: x += y

In [14]: x
Out[14]: Decimal('12.5')
```

You may perform arithmetic between `Decimal`s and integers, but *not* between `Decimal`s and floating-point numbers.

Compound-Interest Problem Requirements Statement

Let's compute compound interest using the `Decimal` type for *precise* monetary calculations. Consider the following requirements statement:

A person invests $1000 in a savings account yielding 5% interest. Assuming that the person leaves all interest on deposit in the account, calculate and display the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:

$a = p(1 + r)^n$

where

p is the original amount invested (i.e., the principal),

r is the annual interest rate,

n is the number of years and

a is the amount on deposit at the end of the nth year.

Calculating Compound Interest

To solve this problem, let's use variables `principal` and `rate` that we defined in snippets `[5]` and `[7]`, and a `for` statement that performs the interest calculation for each of the 10 years the money remains on deposit. For each `year`, the loop displays a formatted string containing the year number and the `amount` on deposit at the end of that year:

lick here to view code image

```
In [15]: for year in range(1, 11):
    ...:     amount = principal * (1 + rate) ** year
    ...:     print(f'{year:>2}{amount:>10.2f}')
    ...:
 1   1050.00
 2   1102.50
 3   1157.62
 4   1215.51
 5   1276.28
 6   1340.10
 7   1407.10
 8   1477.46
 9   1551.33
10   1628.89
```

The algebraic expression $(1 + r)^n$ from the requirements statement is written as

```
(1 + rate) ** year
```

where variable `rate` represents r and variable `year` represents n.

Formatting the Year and Amount on Deposit

C

The statement

lick here to view code image

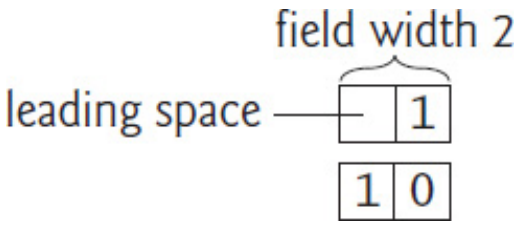
```
print(f'{year:>2}{amount:>10.2f}')
```

uses an f-string with two placeholders to format the loop's output.

The placeholder

```
{year:>2}
```

uses the format specifier `>2` to indicate that `year`'s value should be **right aligned (>)** in a field of width `2`—the **field width** specifies the number of character positions to use when displaying the value. For the single-digit `year` values `1`–`9`, the format specifier `>2` displays a space character followed by the value, thus right aligning the `year`s in the first column. The following diagram shows the numbers 1 and 10 each formatted in a field width of 2:



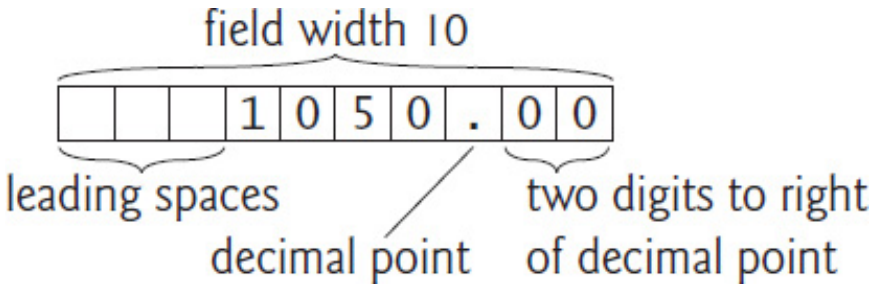
You can **left align** values with <.

The format specifier `10.2f` in the placeholder

```
{amount:>10.2f}
```

formats `amount` as a floating-point number (`f`) right aligned (`>`) in a field width of `10` with a decimal point and two digits to the right of the decimal point (`.2`). Formatting the `amount`s this way *aligns their decimal points vertically*, as is typical with monetary amounts. In the 10 character positions, the three rightmost characters are the number's decimal point followed by the two digits to its right. The remaining seven character positions are the leading spaces and the digits to the decimal point's left. In this example, all the dollar amounts have four digits to the left of the decimal point, so each number is formatted with three *leading spaces*. The following diagram shows the formatting for the value `1050.00`:

C



3.12 BREAK AND CONTINUE STATEMENTS

The `break` and `continue` statements alter a loop's flow of control. Executing a **`break`** statement in a `while` or `for` immediately exits that statement. In the following code, `range` produces the integer sequence 0–99, but the loop terminates when `number` is `10`:

lick here to view code image

```
In [1]: for number in range(100):
   ...:     if number == 10:
   ...:         break
   ...:     print(number, end=' ')
   ...:
0 1 2 3 4 5 6 7 8 9
```

In a script, execution would continue with the next statement after the `for` loop. The `while` and `for` statements each have an optional `else` clause that executes only if the loop terminates normally—that is, not as a result of a `break`.

Executing a **`continue`** statement in a `while` or `for` loop skips the remainder of the loop's suite. In a `while`, the condition is then tested to determine whether the loop should continue executing. In a `for`, the loop processes the next item in the sequence (if any):

lick here to view code image

```
In [2]: for number in range(10):
   ...:     if number == 5:
   ...:         continue
   ...:     print(number, end=' ')
   ...:
0 1 2 3 4 6 7 8 9
```

3.13 BOOLEAN OPERATORS AND, OR AND NOT

C

The conditional operators >, <, >=, <=, == and != can be used to form simple conditions such as `grade >= 60`. To form more complex conditions that combine simple conditions, use the `and`, `or` and `not` Boolean operators.

Boolean Operator `and`

To ensure that two conditions are *both* `True` before executing a control statement's suite, use the **Boolean `and` operator** to combine the conditions. The following code defines two variables, then tests a condition that's `True` if and only if *both* simple conditions are `True`—if either (or both) of the simple conditions is `False`, the entire `and` expression is `False`:

lick here to view code image

```
In [1]: gender = 'Female'

In [2]: age = 70

In [3]: if gender == 'Female' and age >= 65:
   ...:     print('Senior female')
   ...:
Senior female
```

The `if` statement has two simple conditions:

- `gender == 'Female'` determines whether a person is a female and
- `age >= 65` determines whether that person is a senior citizen.

The simple condition to the left of the `and` operator evaluates first because == has higher precedence than `and`. If necessary, the simple condition to the right of `and` evaluates next, because >= has higher precedence than `and`. (We'll discuss shortly why the right side of an `and` operator evaluates *only* if the left side is `True`.) The entire `if` statement condition is `True` if and only if *both* of the simple conditions are `True`. The combined condition can be made clearer by adding redundant parentheses

lick here to view code image

```
(gender == 'Female') and (age >= 65)
```

The table below summarizes the `and` operator by showing all four possible

C

combinations of `False` and `True` values for *expression1* and *expression2*—such tables are called *truth tables*:

expression1	*expression2*	*expression1* `and` *expression2*
False	False	False
False	True	False
True	False	False
True	True	True

Boolean Operator `or`

Use the **Boolean `or` operator** to test whether one *or* both of two conditions are `True`. The following code tests a condition that's `True` if either *or* both simple conditions are `True`—the entire condition is `False` only if *both* simple conditions are `False`:

lick here to view code image

```
In [4]:   semester_average = 83

In [5]:   final_exam = 95

In [6]: if semester_average >= 90 or final_exam >= 90:
   ...:       print('Student   gets an A')
   ...:
Student gets an A
```

Snippet `[6]` also contains two simple conditions:

- `semester_average >= 90` determines whether a student's average was an `A` (90 or above) during the semester, and

C

- `final_exam >= 90` determines whether a student's final-exam grade was an `A`.

The truth table below summarizes the Boolean `or` operator. Operator `and` has higher precedence than `or`.

expression1	*expression2*	*expression1* `or` *expression2*
`False`	`False`	`False`
`False`	`True`	`True`
`True`	`False`	`True`
`True`	`True`	`True`

Improving Performance with Short-Circuit Evaluation

Python stops evaluating an `and` expression as soon as it knows whether the entire condition is `False`. Similarly, Python stops evaluating an `or` expression as soon as it knows whether the entire condition is `True`. This is called *short-circuit evaluation*. So the condition

lick here to view code image

```
gender == 'Female' and age >= 65
```

stops evaluating immediately if `gender` is not equal to `'Female'` because the entire expression must be `False`. If `gender` is equal to `'Female'`, execution continues, because the entire expression will be `True` if the `age` is greater than or equal to `65`.

Similarly, the condition

C

lick here to view code image

```
semester_average >= 90 or final_exam >= 90
```

stops evaluating immediately if `semester_average` is greater than or equal to `90` because the entire expression must be `True`. If `semester_average` is less than `90`, execution continues, because the expression could still be `True` if the `final_exam` is greater than or equal to `90`.

In expressions that use `and`, make the condition that's more likely to be `False` the leftmost condition. In `or` operator expressions, make the condition that's more likely to be `True` the leftmost condition. These techniques can reduce a program's execution time.

Boolean Operator `not`

The **Boolean operator `not`** "reverses" the meaning of a condition—`True` becomes `False` and `False` becomes `True`. This is a **unary operator**—it has only *one* operand. You place the `not` operator before a condition to choose a path of execution if the original condition (without the `not` operator) is `False`, such as in the following code:

lick here to view code image

```
In [7]: grade = 87

In [8]: if not grade == -1:
   ...:     print('The next grade is', grade)
   ...:
The next grade is   87
```

Often, you can avoid using `not` by expressing the condition in a more "natural" or convenient manner. For example, the preceding `if` statement can also be written as follows:

lick here to view code image

```
In [9]: if grade != -1:
   ...:     print('The next grade is', grade)
   ...:
The next grade is   87
```

C

he truth table below summarizes the `not` operator.

expression	not expression
False	True
True	False

The following table shows the precedence and grouping of the operators introduced so far, from top to bottom, in decreasing order of precedence.

Operators	Grouping
`()`	left to right
`**`	right to left
`* / // %`	left to right
`+ -`	left to right
`< <= > >= == !=`	left to right
`not`	left to right

T

`and`	left to right
`or`	left to right

3.14 INTRO TO DATA SCIENCE: MEASURES OF CENTRAL TENDENCY—MEAN, MEDIAN AND MODE

Here we continue our discussion of using statistics to analyze data with several additional descriptive statistics, including:

- **mean**—the *average value* in a set of values.
- **median**—the *middle value* when all the values are arranged in *sorted order*.
- **mode**—the *most frequently occurring value*.

These are **measures of central tendency**—each is a way of producing a single value that represents a "central" value in a set of values, i.e., a value which is in some sense typical of the others.

Let's calculate the mean, median and mode on a list of integers. The following session creates a list called `grades`, then uses the built-in **`sum`** and **`len`** functions to calculate the mean "by hand"—`sum` calculates the total of the grades (397) and `len` returns the number of grades (5):

lick here to view code image

```
In [1]: grades = [85, 93, 45, 89, 85]

In [2]: sum(grades) / len(grades)
Out[2]: 79.4
```

The previous chapter mentioned the *descriptive statistics* count and sum—implemented in Python as the built-in functions `len` and `sum`. Like functions `min` and `max` (introduced in the preceding chapter), `sum` and `len` are both examples of functional-style programming **reductions**—they reduce a collection of values to a single value—the sum of those values and the number of values, respectively. In ection

S

.8's class-average example, we could have deleted lines 10–15 of the script and replaced `average` in line 16 with snippet `[2]`'s calculation.

The Python Standard Library's **`statistics` module** provides functions for calculating the mean, median and mode—these, too, are reductions. To use these capabilities, first `import` the `statistics` module:

```
In [3]: import statistics
```

Then, you can access the module's functions with "`statistics.`" followed by the name of the function to call. The following calculates the `grades` list's mean, median and mode, using the `statistics` module's **`mean`**, **`median`** and **`mode`** functions:

lick here to view code image

```
In [4]: statistics.mean(grades)
Out[4]: 79.4

In [5]: statistics.median(grades)
Out[5]: 85

In [6]: statistics.mode(grades)
Out[6]: 85
```

Each function's argument must be an *iterable*—in this case, the list `grades`. To confirm that the median and mode are correct, you can use the built-in **`sorted` function** to get a copy of `grades` with its values arranged in increasing order:

```
In [7]: sorted(grades)
Out[7]: [45, 85, 85, 89, 93]
```

The `grades` list has an odd number of values (5), so `median` returns the *middle value* (`85`). If the list's number of values is even, `median` returns the *average* of the *two* middle values. Studying the sorted values, you can see that `85` is the mode because it occurs most frequently (twice). The `mode` function causes a `StatisticsError` for lists like

```
[85, 93, 45, 89, 85, 93]
```

in which there are two or more "most frequent" values. Such a set of values is said to be

imodal. Here, both `85` and `93` occur twice.

3.15 WRAP-UP

In this chapter, we discussed Python's control statements, including `if`, `if...else`, `if...elif...else`, `while`, `for`, `break` and `continue`. You saw that the `for` statement performs sequence-controlled iteration—it processes each item in an iterable, such as a range of integers, a string or a list. You used the built-in function `range` to generate sequences of integers from `0` up to, but not including, its argument, and to determine how many times a `for` statement iterates.

You used sentinel-controlled iteration with the `while` statement to create a loop that continues executing until a sentinel value is encountered. You used built-in function `range`'s two-argument version to generate sequences of integers from the first argument's value up to, but not including, the second argument's value. You also used the three-argument version in which the third argument indicated the step between integers in a range.

We introduced the `Decimal` type for precise monetary calculations and used it to calculate compound interest. You used f-strings and various format specifiers to create formatted output. We introduced the `break` and `continue` statements for altering the flow of control in loops. We discussed the Boolean operators `and`, `or` and `not` for creating conditions that combine simple conditions.

Finally, we continued our discussion of descriptive statistics by introducing measures of central tendency—mean, median and mode—and calculating them with functions from the Python Standard Library's `statistics` module.

In the next chapter, you'll create custom functions and use existing functions from Python's `math` and `random` modules. We show several predefined functional-programming reductions and you'll see additional functional-programming capabilities.

b

. Functions

Objectives

In this chapter, you'll

- Create custom functions.
- Import and use Python Standard Library modules, such as `random` and `math`, to reuse code and avoid "reinventing the wheel."
- Pass data between functions.
- Generate a range of random numbers.
- See simulation techniques using random-number generation.
- Seed the random number generator to ensure reproducibility.
- Pack values into a tuple and unpack values from a tuple.
- Return multiple values from a function via a tuple.
- Understand how an identifier's scope determines where in your program you can use it.
- Create functions with default parameter values.
- Call functions with keyword arguments.
- Create functions that can receive any number of arguments.
- Use methods of an object.
- Write and use a recursive function.

Outline

4.1 INTRODUCTION

In this chapter, we continue our discussion of Python fundamentals with custom functions and related topics. We'll use the Python Standard Library's `random` module and random-number generation to simulate rolling a six-sided die. We'll combine custom functions and random-number generation in a script that implements the dice game craps. In that example, we'll also introduce Python's tuple sequence type and use tuples to return more than one value from a function. We'll discuss seeding the random number generator to ensure reproducibility.

You'll import the Python Standard Library's `math` module, then use it to learn about IPython tab completion, which speeds your coding and discovery processes. You'll create functions with default parameter values, call functions with keyword arguments and define functions with arbitrary argument lists. We'll demonstrate calling methods of objects. We'll also discuss how an identifier's scope determines where in your program you can use it.

We'll take a deeper look at importing modules. You'll see that arguments are passed-by-reference to functions. We'll also demonstrate a recursive function and begin presenting Python's functional-style programming capabilities.

In the Intro to Data Science section, we'll continue our discussion of descriptive statistics by introducing measures of dispersion—variance and standard deviation—and calculating them with functions from the Python Standard Library's `statistics` module.

4.2 DEFINING FUNCTIONS

You've called many built-in functions (`int`, `float`, `print`, `input`, `type`, `sum`, `len`, `min` and `max`) and a few functions from the statistics module (`mean`, `median` and `mode`). Each performed a single, well-defined task. You'll often define and call *custom* functions. The following session defines a `square` function that calculates the square of its argument. Then it calls the function twice—once to square the `int` value `7` (producing the `int` value `49`) and once to square the `float` value `2.5` (producing the `float` value `6.25`):

lick here to view code image

```
In [1]: def square(number):
   ...:     """Calculate   the square of number."""
   ...:     return number ** 2
   ...:
```

C

```
In [2]: square(7)
Out[2]: 49

In [3]: square(2.5)
Out[3]: 6.25
```

The statements defining the function in the first snippet are written only once, but may be called "to do their job" from many points throughout a program and as often as you like. Calling `square` with a non-numeric argument like `'hello'` causes a `TypeError` because the exponentiation operator (`**`) works only with numeric values.

Defining a Custom Function

A **function definition** (like `square` in snippet `[1]`) begins with the **`def` keyword**, followed by the function name (`square`), a set of parentheses and a colon (`:`). Like variable identifiers, by convention function names should begin with a lowercase letter and in multiword names underscores should separate each word.

The required parentheses contain the function's **parameter list**—a comma-separated list of **parameters** representing the data that the function needs to perform its task. Function `square` has only one parameter named `number`—the value to be squared. If the parentheses are empty, the function does not use parameters to perform its task.

The indented lines after the colon (`:`) are the function's **block**, which consists of an optional docstring followed by the statements that perform the function's task. We'll soon point out the difference between a function's block and a control statement's suite.

Specifying a Custom Function's Docstring

The *Style Guide for Python Code* says that the first line in a function's block should be a docstring that briefly explains the function's purpose:

```
    """Calculate the square of number."""
```

To provide more detail, you can use a multiline docstring—the style guide recommends starting with a brief explanation, followed by a blank line and the additional details.

Returning a Result to a Function's Caller

When a function finishes executing, it returns control to its caller—that is, the line of code that called the function. In `square`'s block, the **`return`** statement:

```
        return number ** 2
```

first squares `number`, then terminates the function and gives the result back to the caller. In this example, the first caller is in snippet `[2]`, so IPython displays the result in `Out[2]`. The second caller is in snippet `[3]`, so IPython displays the result in `Out[3]`.

Function calls also can be embedded in expressions. The following code calls `square` first, then `print` displays the result:

lick here to view code image

```
In [4]: print('The square of 7 is',   square(7))
The square of 7 is 49
```

There are two other ways to return control from a function to its caller:

- Executing a `return` statement without an expression terminates the function and *implicitly* returns the value **`None`** to the caller. The Python documentation states that `None` represents the absence of a value. `None` evaluates to `False` in conditions.
- When there's no `return` statement in a function, it *implicitly* returns the value `None` after executing the last statement in the function's block.

Local Variables

Though we did not define variables in `square`'s block, it is possible to do so. A function's parameters and variables defined in its block are all **local variables**—they can be used only inside the function and exist only while the function is executing. Trying to access a local variable outside its function's block causes a `NameError`, indicating that the variable is not defined.

Accessing a Function's Docstring via IPython's Help Mechanism

IPython can help you learn about the modules and functions you intend to use in your code, as well as IPython itself. For example, to view a function's docstring to learn how to use the function, type the function's name followed by a **question mark (`?`)**:

lick here to view code image

C

```
In [5]: square?
Signature: square(number)
Docstring: Calculate the square of number.
File:        ~/Documents/examples/ch04/<ipython-input-1-7268c8ff93a9>
Type:      function
```

For our `square` function, the information displayed includes:

- The function's name and parameter list—known as its **signature**.
- The function's docstring.
- The name of the file containing the function's definition. For a function in an interactive session, this line shows information for the snippet that defined the function—the `1` in `"<ipython-input-1-7268c8ff93a9>"` means snippet `[1]`.
- The type of the item for which you accessed IPython's help mechanism—in this case, a function.

If the function's source code is accessible from IPython—such as a function defined in the current session or imported into the session from a `.py` file—you can use **`??`** to display the function's full source-code definition:

lick here to view code image

```
In [6]: square??
Signature: square(number)
Source:
def square(number):
    """Calculate the   square of number."""
    return number ** 2
File:        ~/Documents/examples/ch04/<ipython-input-1-7268c8ff93a9>
Type:      function
```

If the source code is not accessible from IPython, `??` simply shows the docstring.

If the docstring fits in the window, IPython displays the next `In []` prompt. If a docstring is too long to fit, IPython indicates that there's more by displaying a colon (`:`) at the bottom of the window—press the *Space* key to display the next screen. You can navigate backwards and forwards through the docstring with the up and down arrow keys, respectively. IPython displays `(END)` at the end of the docstring. Press *q* (for "quit") at any `:` or the `(END)` prompt to return to the next `In []` prompt. To get a

C

sense of IPython's features, type `?` at any `In []` prompt, press *Enter*, then read the help documentation overview.

4.3 FUNCTIONS WITH MULTIPLE PARAMETERS

Let's define a `maximum` function that determines and returns the largest of three values—the following session calls the function three times with integers, floating-point numbers and strings, respectively.

lick here to view code image

```
In [1]: def maximum(value1,   value2, value3):
   ...:     """Return   the maximum of three values."""
   ...:     max_value = value1
   ...:     if value2 >   max_value:
   ...:         max_value = value2
   ...:     if value3 >   max_value:
   ...:         max_value = value3
   ...:     return max_value
   ...:

In [2]: maximum(12, 27, 36)
Out[2]: 36

In [3]: maximum(12.3, 45.6, 9.7)
Out[3]: 45.6

In [4]: maximum('yellow', 'red', 'orange')
Out[4]: 'yellow'
```

We did not place blank lines above and below the `if` statements, because pressing return on a blank line in interactive mode completes the function's definition.

You also may call `maximum` with mixed types, such as `int`s and `float`s:

```
In [5]: maximum(13.5, -3, 7)
Out[5]: 13.5
```

The call `maximum(13.5, 'hello', 7)` results in `TypeError` because strings and numbers cannot be compared to one another with the greater-than (>) operator.

Function `maximum`'s Definition

Function `maximum` specifies three parameters in a comma-separated list. Snippet `[2]`'s

C

arguments `12`, `27` and `36` are assigned to the parameters `value1`, `value2` and `value3`, respectively.

To determine the largest value, we process one value at a time:

- Initially, we assume that `value1` contains the largest value, so we assign it to the local variable `max_value`. Of course, it's possible that `value2` or `value3` contains the actual largest value, so we still must compare each of these with `max_value`.
- The first `if` statement then tests `value2 > max_value`, and if this condition is `True` assigns `value2` to `max_value`.
- The second `if` statement then tests `value3 > max_value`, and if this condition is `True` assigns `value3` to `max_value`.

Now, `max_value` contains the largest value, so we return it. When control returns to the caller, the parameters `value1`, `value2` and `value3` and the variable `max_value` in the function's block—which are all *local variables*—no longer exist.

Python's Built-In `max` and `min` Functions

For many common tasks, the capabilities you need already exist in Python. For example, built-in **`max`** and **`min`** functions know how to determine the largest and smallest of their two or more arguments, respectively:

lick here to view code image

```
In [6]: max('yellow', 'red', 'orange', 'blue', 'green')
Out[6]: 'yellow'

In [7]: min(15, 9, 27, 14)
Out[7]: 9
```

Each of these functions also can receive an iterable argument, such as a list or a string. Using built-in functions or functions from the Python Standard Library's modules rather than writing your own can reduce development time and increase program reliability, portability and performance. For a list of Python's built-in functions and modules, see

```
ttps://docs.python.org/3/library/index.html
```

€

4.4 RANDOM-NUMBER GENERATION

We now take a brief diversion into a popular type of programming application—simulation and game playing. You can introduce the **element of chance** via the Python Standard Library's **`random` module**.

Rolling a Six-Sided Die

Let's produce 10 random integers in the range 1–6 to simulate rolling a six-sided die:

lick here to view code image

```
In [1]: import random

In [2]: for roll in range(10):
   ...:     print(random.randrange(1,   7), end=' ')
   ...:
4 2 5 5 4 6 4 6 1 5
```

First, we import `random` so we can use the module's capabilities. The **`randrange`** function generates an integer from the first argument value up to, but *not* including, the second argument value. Let's use the up arrow key to recall the `for` statement, then press *Enter* to re-execute it. Notice that *different* values are displayed:

lick here to view code image

```
In [3]: for roll in range(10):
   ...:     print(random.randrange(1,   7), end=' ')
   ...:
4 5 4 5 1 4 1 4 6 5
```

Sometimes, you may want to guarantee **reproducibility** of a random sequence—for debugging, for example. At the end of this section, we'll use the `random` module's `seed` function to do this.

Rolling a Six-Sided Die 6,000,000 Times

If `randrange` truly produces integers at random, every number in its range has an equal **probability** (or *chance* or *likelihood*) of being returned each time we call it. To show that the die faces 1–6 occur with equal likelihood, the following script simulates 6,000,000 die rolls. When you run the script, each die face should occur approximately 1,000,000 times, as in the sample output.

C

lick here to view code image

```
# fig04_01.py
"""Roll a six-sided die 6,000,000   times."""
import random

# face frequency counters
frequency1 = 0
frequency2 = 0
frequency3 = 0
frequency4 = 0
frequency5 = 0
frequency6 = 0

# 6,000,000 die rolls
for roll in range(6_000_000):  # note underscore   separators
    face =   random.randrange(1, 7)

    # increment appropriate face counter
    if face == 1:
        frequency1   += 1
    elif face == 2:
        frequency2   += 1
    elif face == 3:
        frequency3   += 1
    elif face == 4:
        frequency4   += 1
    elif face == 5:
        frequency5   += 1
    elif face == 6:
        frequency6   += 1

print(f'Face{"Frequency":>13}')
print(f'{1:>4}{frequency1:>13}')
print(f'{2:>4}{frequency2:>13}')
print(f'{3:>4}{frequency3:>13}')
print(f'{4:>4}{frequency4:>13}')
print(f'{5:>4}{frequency5:>13}')
print(f'{6:>4}{frequency6:>13}')
```

lick here to view code image

```
Face    Frequency
   1       998686
   2      1001481
   3       999900
   4      1000453
   5       999953
   6       999527
```

C

The script uses *nested* control statements (an `if elif` statement nested in the `for` statement) to determine the number of times each die face appears. The `for` statement iterates 6,000,000 times. We used Python's underscore (_) digit separator to make the value `6000000` more readable. The expression `range(6,000,000)` would be incorrect. Commas separate arguments in function calls, so Python would treat `range(6,000,000)` as a call to `range` with the *three* arguments `6`, `0` and `0`.

For each die roll, the script adds `1` to the appropriate counter variable. Run the program, and observe the results. This program might take a few seconds to complete execution. As you'll see, each execution produces *different* results. Note that we did not provide an `else` clause in the `if elif` statement.

Seeding the Random-Number Generator for Reproducibility

Function `randrange` actually generates **pseudorandom numbers**, based on an internal calculation that begins with a numeric value known as a **seed**. Repeatedly calling `randrange` produces a sequence of numbers that *appear* to be random, because each time you start a new interactive session or execute a script that uses the `random` module's functions, Python internally uses a *different* seed value. [1] When you're debugging logic errors in programs that use randomly generated data, it can be helpful to use the *same* sequence of random numbers until you've eliminated the logic errors, before testing the program with other values. To do this, you can use the random module's **`seed`** function to **seed the random-number generator** yourself —this forces `randrange` to begin calculating its pseudorandom number sequence from the seed you specify. In the following session, snippets `[5]` and `[8]` produce the same results, because snippets `[4]` and `[7]` use the same seed (`32`):

[1] According to the documentation, Python bases the seed value on the system clock or an operating-system-dependent randomness source. For applications requiring secure random numbers, such as cryptography, the documentation recommends using the `secrets` module, rather than the `random` module.

lick here to view code image

```
In [4]: random.seed(32)

In [5]: for roll in range(10):
   ...:     print(random.randrange(1,   7), end=' ')
   ...:
1 2 2 3 6 2 4 1 6 1
In [6]: for roll in range(10):
   ...:     print(random.randrange(1,   7), end=' ')
```

C

```
   ...:
1 3 5 3 1 5 6 4 3 5
In [7]: random.seed(32)

In [8]: for roll in range(10):
   ...:     print(random.randrange(1,   7), end=' ')
   ...:
1 2 2 3 6 2 4 1 6 1
```

Snippet `[6]` generates *different* values because it simply continues the pseudorandom number sequence that began in snippet `[5]`.

4.5 CASE STUDY: A GAME OF CHANCE

In this section, we simulate the popular dice game known as “craps.” Here is the requirements statement:

You roll two six-sided dice, each with faces containing one, two, three, four, five and six spots, respectively. When the dice come to rest, the sum of the spots on the two upward faces is calculated. If the sum is 7 or 11 on the first roll, you win. If the sum is 2, 3 or 12 on the first roll (called “craps”), you lose (i.e., the “house” wins). If the sum is 4, 5, 6, 8, 9 or 10 on the first roll, that sum becomes your “point.” To win, you must continue rolling the dice until you “make your point” (i.e., roll that same point value). You lose by rolling a 7 before making your point.

The following script simulates the game and shows several sample executions, illustrating winning on the first roll, losing on the first roll, winning on a subsequent roll and losing on a subsequent roll.

lick here to view code image

```
1 # fig04_02.py
2 """Simulating the dice game   Craps."""
3 import random
4
5 def roll_dice():
6     """Roll two dice and return their face   values as a tuple."""
7     die1 =   random.randrange(1, 7)
8     die2 =   random.randrange(1, 7)
9     return (die1, die2)  # pack die   face values into a tuple
10
11 def display_dice(dice):
12     """Display one roll of the two   dice."""
13     die1, die2   = dice  # unpack the tuple into variables die1 and
14     print(f'Player rolled {die1} +   {die2} = {sum(dice)}')
```

C

```
15
16 die_values =   roll_dice()  # first roll
17 display_dice(die_values)
18
19 # determine game status and point, based on first roll
20 sum_of_dice =   sum(die_values)
21
22 if sum_of_dice in (7, 11):  # win
23     game_status   = 'WON'
24 elif sum_of_dice in (2, 3, 12):  # lose
25     game_status   = 'LOST'
26 else:  # remember point
27     game_status   = 'CONTINUE'
28     my_point =   sum_of_dice
29     print('Point is', my_point)
30
31 # continue rolling until player wins or loses
32 while game_status == 'CONTINUE':
33     die_values   = roll_dice()
34     display_dice(die_values)
35     sum_of_dice   = sum(die_values)
36
37     if sum_of_dice == my_point:  #   win by making point
38         game_status   = 'WON'
39     elif sum_of_dice == 7:  # lose   by rolling 7
40         game_status   = 'LOST'
41
42 # display "wins" or "loses" message
43 if game_status == 'WON':
44     print('Player wins')
45 else:
46     print('Player loses')
```

lick here to view code image

```
Player rolled 2 + 5 = 7
Player wins
```

lick here to view code image

```
Player rolled 1 + 2 = 3
Player loses
```

lick here to view code image

C

```
Player rolled 5 + 4 = 9
Point is 9
Player rolled 4 + 4 = 8
Player rolled 2 + 3 = 5
Player rolled 5 + 4 = 9
Player wins
```

lick here to view code image

```
Player rolled 1 + 5 = 6
Point is 6
Player rolled 1 + 6 = 7
Player loses
```

Function `roll_dice`—Returning Multiple Values Via a Tuple

Function `roll_dice` (lines 5–9) simulates rolling two dice on each roll. The function is defined once, then called from several places in the program (lines 16 and 33). The empty parameter list indicates that `roll_dice` does not require arguments to perform its task.

The built-in and custom functions you've called so far each return one value. Sometimes it's useful to return more than one value, as in `roll_dice`, which returns both die values (line 9) as a **tuple**—an **immutable** (that is, unmodifiable) sequences of values. To create a tuple, separate its values with commas, as in line 9:

```
        (die1, die2)
```

This is known as **packing a tuple**. The parentheses are optional, but we recommend using them for clarity. We discuss tuples in depth in the next chapter.

Function `display_dice`

To use a tuple's values, you can assign them to a comma-separated list of variables, which **unpacks** the tuple. To display each roll of the dice, the function `display_dice` (defined in lines 11–14 and called in lines 17 and 34) unpacks the tuple argument it receives (line 13). The number of variables to the left of = must match the number of elements in the tuple; otherwise, a `ValueError` occurs. Line 14 `print`s a formatted string containing both die values and their sum. We calculate the sum of the dice by

C

passing the tuple to the built-in `sum` function—like a list, a tuple is a sequence.

Note that functions `roll_dice` and `display_dice` each begin their blocks with a docstring that states what the function does. Also, both functions contain local variables `die1` and `die2`. These variables do not "collide," because they belong to different functions' blocks. Each local variable is accessible only in the block that defined it.

First Roll

When the script begins executing, lines 16–17 roll the dice and display the results. Line 20 calculates the sum of the dice for use in lines 22–29. You can win or lose on the first roll or any subsequent roll. The variable `game_status` keeps track of the win/loss status.

The **`in` operator** in line 22

```
        sum_of_dice in (7, 11)
```

tests whether the tuple `(7, 11)` contains `sum_of_dice`'s value. If this condition is `True`, you rolled a `7` or an `11`. In this case, you won on the first roll, so the script sets `game_status` to `'WON'`. The operator's right operand can be any iterable. There's also a **`not in`** operator to determine whether a value is *not* in an iterable. The preceding concise condition is equivalent to

lick here to view code image

```
        (sum_of_dice == 7) or (sum_of_dice == 11)
```

Similarly, the condition in line 24

```
        sum_of_dice in (2, 3, 12)
```

tests whether the tuple `(2, 3, 12)` contains `sum_of_dice`'s value. If so, you lost on the first roll, so the script sets `game_status` to `'LOST'`.

For any other sum of the dice (`4`, `5`, `6`, `8`, `9` or `10`):

- line 27 sets `game_status` to `'CONTINUE'` so you can continue rolling

C

- line 28 stores the sum of the dice in `my_point` to keep track of what you must roll to win and
- line 29 displays `my_point`.

Subsequent Rolls

If `game_status` is equal to `'CONTINUE'` (line 32), you did not win or lose, so the `while` statement's suite (lines 33–40) executes. Each loop iteration calls `roll_dice`, displays the die values and calculates their sum. If `sum_of_dice` is equal to `my_point` (line 37) or `7` (line 39), the script sets `game_status` to `'WON' or 'LOST'`, respectively, and the loop terminates. Otherwise, the `while` loop continues executing with the next roll.

Displaying the Final Results

When the loop terminates, the script proceeds to the `if else` statement (lines 43–46), which prints `'Player wins'` if `game_status` is `'WON'`, or `'Player loses'` otherwise.

4.6 PYTHON STANDARD LIBRARY

Typically, you write Python programs by combining functions and classes (that is, custom types) that you create with preexisting functions and classes defined in modules, such as those in the Python Standard Library and other libraries. A key programming goal is to avoid "reinventing the wheel."

A module is a file that groups related functions, data and classes. The type `Decimal` from the Python Standard Library's `decimal` module is actually a class. We introduced classes briefly in hapter 1 and discuss them in detail in the "Object-Oriented Programming" chapter. A **package** groups related modules. In this book, you'll work with many preexisting modules and packages, and you'll create your own modules—in fact, every Python source-code (`.py`) file you create is a module. Creating packages is beyond this book's scope. They're typically used to organize a large library's functionality into smaller subsets that are easier to maintain and can be imported separately for convenience. For example, the `matplotlib` visualization library that we use in ection 5.17 has extensive functionality (its documentation is over 2300 pages), so we'll import only the subsets we need in our examples (`pyplot` and `animation`).

The Python Standard Library is provided with the core Python language. Its packages and modules contain capabilities for a wide variety of everyday programming tasks. [2]

S

ou can see a complete list of the standard library modules at

[2] The Python Tutorial refers to this as the batteries included approach.

```
ttps://docs.python.org/3/library/
```

You've already used capabilities from the `decimal`, `statistics` and `random` modules. In the next section, you'll use mathematics capabilities from the `math` module. You'll see many other Python Standard Library modules throughout the book's examples, including many of those in the following table:

Some popular Python Standard Library modules	
`collections`—Data structures beyond lists, tuples, dictionaries and sets.	`math`—Common math constants and operations.
Cryptography modules—Encrypting data for secure transmission.	`os`—Interacting with the operating system.
`csv`—Processing comma-separated value files (like those in Excel).	`profile`, `pstats`, `timeit`—Performance analysis.
`datetime`—Date and time manipulations. Also modules `time` and `calendar`.	`random`—Pseudorandom numbers.
`decimal`—Fixed-point and floating-point arithmetic, including monetary calculations.	`re`—Regular expressions for pattern matching.
`doctest`—Embed validation tests and expected results in docstrings for simple unit testing.	`sqlite3`—SQLite relational database access.
	`statistics`—Mathematical statistics functions such as `mean`, `median`, `mode` and `variance`.
	`string`—String processing.
	`sys`—Command-line argument

`gettext` and `locale`—Internationalization and localization modules. `json`—JavaScript Object Notation (JSON) processing used with web services and NoSQL document databases.	processing; standard input, standard output and standard error streams. `tkinter`—Graphical user interfaces (GUIs) and canvas-based graphics. `turtle`—Turtle graphics. `webbrowser`—For conveniently displaying web pages in Python apps.

4.7 `math` MODULE FUNCTIONS

The **`math` module** defines functions for performing various common mathematical calculations. Recall from the previous chapter that an `import` statement of the following form enables you to use a module's definitions via the module's name and a dot (`.`):

```
In [1]: import math
```

For example, the following snippet calculates the square root of 900 by calling the `math` module's **`sqrt` function**, which returns its result as a `float` value:

```
In [2]: math.sqrt(900)
Out[2]: 30.0
```

Similarly, the following snippet calculates the absolute value of `-10` by calling the `math` module's **`fabs` function**, which returns its result as a `float` value:

```
In [3]: math.fabs(-10)
Out[3]: 10.0
```

Some `math` module functions are summarized below—you can view the complete list at

ttps://docs.python.org/3/library/math.html

Function	Description	Example
`ceil(x)`	Rounds *x* to the smallest integer not less than *x*	`ceil(9.2)` is `10.0` `ceil(-9.8)` is `-9.0`
`floor(x)`	Rounds *x* to the largest integer not greater than *x*	`floor(9.2)` is `9.0` `floor(-9.8)` is `-10.0`
`sin(x)`	Trigonometric sine of *x* (*x* in radians)	`sin(0.0)` is `0.0`
`cos(x)`	Trigonometric cosine of *x* (*x* in radians)	`cos(0.0)` is `1.0`
`tan(x)`	Trigonometric tangent of *x* (*x* in radians)	`tan(0.0)` is `0.0`
`exp(x)`	Exponential function e^x	`exp(1.0)` is `2.718282` `exp(2.0)` is `7.389056`
		`log(2.718282)` is `1.0`

h

`log(x)`	Natural logarithm of x (base e)	`log(7.389056)` is `2.0`

`log10(x)`	Logarithm of x (base 10)	`log10(10.0)` is `1.0`
		
		`log10(100.0)` is `2.0`

`pow(x, y)`	x raised to power y (x^y)	`pow(2.0, 7.0)` is `128.0`
		
		`pow(9.0, .5)` is `3.0`

`sqrt(x)`	square root of x	`sqrt(900.0)` is `30.0`
		
		`sqrt(9.0)` is `3.0`

`fabs(x)`	Absolute value of x—always returns a float. Python also has the built-in function `abs`, which returns an `int` or a `float`, based on its argument.	`fabs(5.1)` is `5.1`
		
		`fabs(-5.1)` is `5.1`

`fmod(x, y)`	Remainder of x/y as a floating-point number	`fmod(9.8, 4.0)` is `1.8`

u

4.8 USING IPYTHON TAB COMPLETION FOR DISCOVERY

You can view a module's documentation in IPython interactive mode via **tab completion**—a **discovery** feature that speeds your coding and discovery processes. After you type a portion of an identifier and press *Tab*, IPython completes the identifier for you or provides a list of identifiers that begin with what you've typed so far. This may vary based on your operating system platform and what you have imported into your IPython session:

lick here to view code image

```
In [1]: import math

In [2]: ma<Tab>
            map           %macro       %%markdown
            math          %magic       %matplotlib
            max()         %man
```

You can scroll through the identifiers with the up and down arrow keys. As you do, IPython highlights an identifier and shows it to the right of the `In []` prompt.

Viewing Identifiers in a Module

To view a list of identifiers defined in a module, type the module's name and a dot (`.`), then press *Tab*:

lick here to view code image

```
In [3]: math.<Tab>
          acos()       atan()       copysign()   e            expm1()
          acosh()      atan2()      cos()        erf()        fabs()
          asin()       atanh()      cosh()       erfc()       factorial() >
          asinh()      ceil()       degrees()    exp()        floor()
```

If there are more identifiers to display than are currently shown, IPython displays the > symbol (on some platforms) at the right edge, in this case to the right of `factorial()`. You can use the up and down arrow keys to scroll through the list. In the list of identifiers:

- Those followed by parentheses are functions (or methods, as you'll see later).
- Single-word identifiers (such as `Employee`) that begin with an uppercase letter and

C

multiword identifiers in which each word begins with an uppercase letter (such as `CommissionEmployee`) represent class names (there are none in the preceding list). This naming convention, which the *Style Guide for Python Code* recommends, is known as **CamelCase** because the uppercase letters stand out like a camel's humps.

- Lowercase identifiers without parentheses, such as `pi` (not shown in the preceding list) and `e`, are variables. The identifier `pi` evaluates to `3.141592653589793`, and the identifier `e` evaluates to `2.718281828459045`. In the `math` module, `pi` and `e` represent the mathematical constants π and *e*, respectively.

Python does not have *constants*, although many objects in Python are immutable (nonmodifiable). So even though `pi` and `e` are real-world constants, *you must not assign new values to them*, because that would change their values. To help distinguish constants from other variables, the style guide recommends naming your custom constants with all capital letters.

Using the Currently Highlighted Function

As you navigate through the identifiers, if you wish to use a currently highlighted function, simply start typing its arguments in parentheses. IPython then hides the autocompletion list. If you need more information about the currently highlighted item, you can view its docstring by typing a question mark (`?`) following the name and pressing *Enter* to view the help documentation. The following shows the `fabs` function's docstring:

lick here to view code image

```
In [4]: math.fabs?
Docstring:
fabs(x)

Return the absolute value of the float x.
Type:      builtin_function_or_method
```

The `builtin_function_or_method` shown above indicates that `fabs` is part of a Python Standard Library module. Such modules are considered to be built into Python. In this case, `fabs` is a built-in function from the `math` module.

4.9 DEFAULT PARAMETER VALUES

C

When defining a function, you can specify that a parameter has a **default parameter value**. When calling the function, if you omit the argument for a parameter with a default parameter value, the default value for that parameter is automatically passed. Let's define a function `rectangle_area` with default parameter values:

lick here to view code image

```
In [1]: def   rectangle_area(length=2, width=3):
   ...:     """Return   a rectangle's area."""
   ...:     return length *   width
   ...:
```

You specify a default parameter value by following a parameter's name with an = and a value—in this case, the default parameter values are `2` and `3` for `length` and `width`, respectively. Any parameters with default parameter values must appear in the parameter list to the *right* of parameters that do not have defaults.

The following call to `rectangle_area` has no arguments, so IPython uses both default parameter values as if you had called `rectangle_area(2, 3)`:

```
In [2]: rectangle_area()
Out[2]: 6
```

The following call to `rectangle_area` has only one argument. Arguments are assigned to parameters from left to right, so `10` is used as the `length`. The interpreter passes the default parameter value 3 for the `width` as if you had called `rectangle_area(10, 3)`:

```
In [3]: rectangle_area(10)
Out[3]: 30
```

The following call to `rectangle_area` has arguments for both `length` and `width`, so IPython- ignores the default parameter values:

```
In [4]: rectangle_area(10, 5)
Out[4]: 50
```

4.10 KEYWORD ARGUMENTS

C

When calling functions, you can use **keyword arguments** to pass arguments in *any* order. To demonstrate keyword arguments, we redefine the `rectangle_area` function—this time without default parameter values:

lick here to view code image

```
In [1]: def   rectangle_area(length, width):
   ...:      """Return   a rectangle's area."""
   ...:      return length *   width
   ...:
```

Each keyword *argument in a call* has the form *parametername=value*. The following call shows that the order of keyword arguments does not matter—they do not need to match the corresponding parameters' positions in the function definition:

lick here to view code image

```
In [2]: rectangle_area(width=5,   length=10)
Out[3]: 50
```

In each function call, you must place keyword arguments *after* a function's positional arguments—that is, any arguments for which you do not specify the parameter name. Such arguments are assigned to the function's parameters left-to-right, based on the argument's positions in the argument list. Keyword arguments are also helpful for improving the readability of function calls, especially for functions with many arguments.

4.11 ARBITRARY ARGUMENT LISTS

Functions with **arbitrary argument lists**, such as built-in functions `min` and `max`, can receive *any* number of arguments. Consider the following `min` call:

```
min(88, 75, 96, 55, 83)
```

The function's documentation states that `min` has two *required* parameters (named `arg1` and `arg2`) and an optional third parameter of the form **`*args`**, indicating that the function can receive any number of additional arguments. The `*` before the parameter name tells Python to pack any remaining arguments into a tuple that's passed to the `args` parameter. In the call above, parameter `arg1` receives `88`,

C

parameter `arg2` receives `75` and parameter `args` receives the tuple `(96, 55, 83)`.

Defining a Function with an Arbitrary Argument List

Let's define an `average` function that can receive any number of arguments:

lick here to view code image

```
In [1]: def average(*args):
   ...:     return sum(args) /   len(args)
   ...:
```

The parameter name `args` is used by convention, but you may use any identifier. If the function has multiple parameters, the `*args` parameter must be the *rightmost* parameter.

Now, let's call `average` several times with arbitrary argument lists of different lengths:

lick here to view code image

```
In [2]: average(5, 10)
Out[2]: 7.5

In [3]: average(5, 10, 15)
Out[3]: 10.0

In [4]: average(5, 10, 15, 20)
Out[4]: 12.5
```

To calculate the average, divide the sum of the `args` tuple's elements (returned by built-in function `sum`) by the tuple's number of elements (returned by built-in function `len`). Note in our `average` definition that if the length of `args` is 0, a `ZeroDivisionError` occurs. In the next chapter, you'll see how to access a tuple's elements without unpacking them.

Passing an Iterable's Individual Elements as Function Arguments

You can unpack a tuple's, list's or other iterable's elements to pass them as individual function arguments. The **`*` operator**, when applied to an iterable argument in a function call, unpacks its elements. The following code creates a five-element `grades` list, then uses the expression `*grades` to unpack its elements as `average`'s arguments:

C

lick here to view code image

```
In [5]: grades = [88, 75, 96, 55, 83]

In [6]: average(*grades)
Out[6]: 79.4
```

The call shown above is equivalent to `average(88, 75, 96, 55, 83)`.

4.12 METHODS: FUNCTIONS THAT BELONG TO OBJECTS

A **method** is simply a function that you call on an object using the form

```
object_name.method_name(arguments)
```

For example, the following session creates the string variable `s` and assigns it the string object `'Hello'`. Then the session calls the object's **`lower`** and **`upper`** methods, which produce *new* strings containing all-lowercase and all-uppercase versions of the original string, leaving `s` unchanged:

lick here to view code image

```
In [1]: s = 'Hello'

In [2]: s.lower()  # call lower   method on string object s
Out[2]: 'hello'

In [3]: s.upper()
Out[3]: 'HELLO'

In [4]: s
Out[4]: 'Hello'
```

The *Python Standard Library* reference at

```
ttps://docs.python.org/3/library/index.html
```

describes the methods of built-in types and the types in the Python Standard Library. In the "Object-Oriented Programming" chapter, you'll create *custom* types called classes and define custom methods that you can call on objects of those classes.

€

4.13 SCOPE RULES

Each identifier has a **scope** that determines where you can use it in your program. For that portion of the program, the identifier is said to be "in scope."

Local Scope

A local variable's identifier has **local scope**. It's "in scope" only from its definition to the end of the function's block. It "goes out of scope" when the function returns to its caller. So, a local variable can be used only inside the function that defines it.

Global Scope

Identifiers defined outside any function (or class) have **global scope**—these may include functions, variables and classes. Variables with global scope are known as **global variables**. Identifiers with global scope can be used in a `.py` file or interactive session anywhere after they're defined.

Accessing a Global Variable from a Function

You can access a global variable's value inside a function:

lick here to view code image

```
In [1]: x = 7

In [2]: def access_global():
   ...:     print('x printed from   access_global:', x)
   ...:

In [3]: access_global()
x printed from access_global: 7
```

However, by default, you cannot *modify* a global variable in a function—when you first assign a value to a variable in a function's block, Python creates a *new* local variable:

lick here to view code image

```
In [4]: def   try_to_modify_global():
   ...:     x = 3.5
   ...:     print('x printed from   try_to_modify_global:', x)
   ...:

In [5]: try_to_modify_global()
x printed from try_to_modify_global: 3.5
```

C

```
In [6]: x
Out[6]: 7
```

In function `try_to_modify_global`'s block, the local `x` **shadows** the global `x`, making it inaccessible in the scope of the function's block. Snippet `[6]` shows that global variable `x` still exists and has its original value (`7`) after function `try_to_modify_global` executes.

To modify a global variable in a function's block, you must use a **`global`** statement to declare that the variable is defined in the global scope:

lick here to view code image

```
In [7]: def modify_global():
   ...:     global x
   ...:     x = 'hello'
   ...:     print('x printed from modify_global:',    x)
   ...:

In [8]: modify_global()
x printed from modify_global: hello

In [9]: x
Out[9]: 'hello'
```

Blocks vs. Suites

You've now defined function *blocks* and control statement *suites*. When you create a variable in a block, it's *local* to that block. However, when you create a variable in a control statement's suite, the variable's scope depends on where the control statement is defined:

- If the control statement is in the global scope, then any variables defined in the control statement have global scope.
- If the control statement is in a function's block, then any variables defined in the control statement have local scope.

We'll continue our scope discussion in the "Object-Oriented Programming" chapter when we introduce custom classes.

Shadowing Functions

C

In the preceding chapters, when summing values, we stored the sum in a variable named `total`. The reason we did this is that `sum` is a built-in function. If you define a variable named `sum`, it *shadows* the built-in function, making it inaccessible in your code. When you execute the following assignment, Python binds the identifier `sum` to the `int` object containing `15`. At this point, the identifier `sum` no longer references the built-in function. So, when you try to use `sum` as a function, a `TypeError` occurs:

lick here to view code image

```
In [10]: sum = 10 + 5

In [11]: sum
Out[11]: 15

In [12]: sum([10, 5])
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last
 ipython-input-12-1237d97a65fb> in <module>()
----> 1 sum([10, 5])

TypeError: 'int' object is   not callable
```

Statements at Global Scope

In the scripts you've seen so far, we've written some statements outside functions at the global scope and some statements inside function blocks. Script statements at global scope execute as soon as they're encountered by the interpreter, whereas statements in a block execute only when the function is called.

4.14 `IMPORT`: A DEEPER LOOK

You've imported modules (such as `math` and `random`) with a statement like:

```
import module_name
```

then accessed their features via each module's name and a dot (`.`). Also, you've imported a specific identifier from a module (such as the `decimal` module's `Decimal` type) with a statement like:

```
from module_name import identifier
```

€

then used that identifier without having to precede it with the module name and a dot (`.`).

Importing Multiple Identifiers from a Module

Using the `from import` statement you can import a comma-separated list of identifiers from a module then use them in your code without having to precede them with the module name and a dot (`.`):

lick here to view code image

```
In [1]: from math import ceil, floor

In [2]: ceil(10.3)
Out[2]: 11

In [3]: floor(10.7)
Out[3]: 10
```

Trying to use a function that's not imported causes a `NameError`, indicating that the name is not defined.

Caution: Avoid Wildcard Imports

You can import *all* identifiers defined in a module with a **wildcard import** of the form

```
from modulename import *
```

This makes all of the module's identifiers available for use in your code. Importing a module's identifiers with a wildcard `import` can lead to subtle errors—it's considered a dangerous practice that you should avoid. Consider the following snippets:

```
In [4]: e = 'hello'

In [5]: from math import *

In [6]: e
Out[6]: 2.718281828459045
```

Initially, we assign the string `'hello'` to a variable named `e`. After executing snippet `[5]` though, the variable `e` is replaced, possibly by accident, with the `math` module's constant `e`, representing the mathematical floating-point value *e*.

C

Binding Names for Modules and Module Identifiers

Sometimes it's helpful to import a module and use an abbreviation for it to simplify your code. The `import` statement's **`as`** clause allows you to specify the name used to reference the module's identifiers. For example, in ection 3.14 we could have imported the `statistics` module and accessed its `mean` function as follows:

lick here to view code image

```
In [7]: import statistics as stats

In [8]: grades = [85, 93, 45, 87, 93]

In [9]: stats.mean(grades)
Out[9]: 80.6
```

As you'll see in later chapters, `import as` is frequently used to import Python libraries with convenient abbreviations, like `stats` for the `statistics` module. As another example, we'll use the `numpy` module which typically is imported with

```
import numpy as np
```

Library documentation often mentions popular shorthand names.

Typically, when importing a module, you should use `import` or `import as` statements, then access the module through the module name or the abbreviation following the `as` keyword, respectively. This ensures that you do not accidentally import an identifier that conflicts with one in your code.

4.15 PASSING ARGUMENTS TO FUNCTIONS: A DEEPER LOOK

Let's take a closer look at how arguments are passed to functions. In many programming languages, there are two ways to pass arguments—**pass-by-value** and **pass-by-reference** (sometimes called **call-by-value** and **call-by-reference**, respectively):

- With pass-by-value, the called function receives a *copy* of the argument's *value* and works exclusively with that copy. Changes to the function's copy do *not* affect the original variable's value in the caller.

S

- With pass-by-reference, the called function can access the argument's value in the caller directly and modify the value if it's mutable.

Python arguments are always passed by reference. Some people call this **pass-by-object-reference**, because "everything in Python is an object." [3] When a function call provides an argument, Python copies the argument object's *reference*—not the object itself—into the corresponding parameter. This is important for performance. Functions often manipulate large objects—frequently copying them would consume large amounts of computer memory and significantly slow program performance.

[3] Even the functions you defined in this chapter and the classes (custom types) youll define in later chapters are objects in Python.

Memory Addresses, References and "Pointers"

You interact with an object via a reference, which behind the scenes is that object's address (or location) in the computer's memory—sometimes called a "pointer" in other languages. After an assignment like

```
x = 7
```

the variable `x` does not actually contain the value `7`. Rather, it contains a reference to an *object* containing `7` stored *elsewhere* in memory. You might say that `x` "points to" (that is, references) the object containing `7`, as in the diagram below:

Built-In Function `id` and Object Identities

Let's consider how we pass arguments to functions. First, let's create the integer variable `x` mentioned above—shortly we'll use `x` as a function argument:

```
In [1]: x = 7
```

Now `x` refers to (or "points to") the integer object containing `7`. No two separate objects can reside at the same address in memory, so every object in memory has a *unique address*. Though we can't see an object's address, we can use the built-in **`id` function**

to obtain a *unique* `int` value which identifies only that object while it remains in memory (you'll likely get a different value when you run this on your computer):

```
In [2]: id(x)
Out[2]: 4350477840
```

The integer result of calling `id` is known as the object's **identity**. [4] No two objects in memory can have the same *identity*. We'll use object identities to demonstrate that objects are passed by reference.

[4] According to the Python documentation, depending on the Python implementation youre using, an objects identity may be the objects actual memory address, but this is not required.

Passing an Object to a Function

Let's define a `cube` function that displays its parameter's identity, then returns the parameter's value cubed:

lick here to view code image

```
In [3]: def cube(number):
   ...:     print('id(number):',   id(number))
   ...:     return number ** 3
   ...:
```

Next, let's call `cube` with the argument `x`, which refers to the integer object containing `7`:

```
In [4]: cube(x)
id(number): 4350477840
Out[4]: 343
```

The identity displayed for `cube`'s parameter `number`—`4350477840`—is the *same* as that displayed for `x` previously. Since every object has a unique identity, both the *argument* `x` and the *parameter* `number` refer to the *same object* while `cube` executes. So when function `cube` uses its parameter `number` in its calculation, it gets the value of `number` from the original object in the caller.

Testing Object Identities with the `is` Operator

C

You also can prove that the argument and the parameter refer to the same object with Python's **`is` operator**, which returns `True` if its two operands have the *same identity*:

lick here to view code image

```
In [5]: def cube(number):
   ...:     print('number is x:',   number is x)  # x is a   global variab
   ...:     return number ** 3
   ...:

In [6]: cube(x)
number is x: True
Out[6]: 343
```

Immutable Objects as Arguments

When a function receives as an argument a reference to an *immutable* (unmodifiable) object—such as an `int`, `float`, `string` or `tuple`—even though you have direct access to the original object in the caller, you cannot modify the original immutable object's value. To prove this, first let's have `cube` display `id(number)` before and after assigning a new object to the parameter `number` via an augmented assignment:

lick here to view code image

```
In [7]: def cube(number):
   ...:     print('id(number) before   modifying number:', id(number))
   ...:     number **= 3
   ...:     print('id(number) after   modifying number:', id(number))
   ...:     return number
   ...:

In [8]: cube(x)
id(number) before modifying number: 4350477840
id(number) after modifying number: 4396653744
Out[8]: 343
```

When we call `cube(x)`, the first `print` statement shows that `id(number)` initially is the same as `id(x)` in snippet `[2]`. Numeric values are immutable, so the statement

```
number **= 3
```

actually creates a *new object* containing the cubed value, then assigns that object's reference to parameter `number`. Recall that if there are no more references to the

C

original object, it will be *garbage collected*. Function `cube`'s second `print` statement shows the *new* object's identity. Object identities must be unique, so `number` must refer to a *different* object. To show that `x` was not modified, we display its value and identity again:

lick here to view code image

```
In [9]: print(f'x = {x}; id(x) = {id(x)}')
x = 7; id(x) = 4350477840
```

Mutable Objects as Arguments

In the next chapter, we'll show that when a reference to a *mutable* object like a list is passed to a function, the function *can* modify the original object in the caller.

4.16 RECURSION

Let's write a program to perform a famous mathematical calculation. Consider the *factorial* of a positive integer *n*, which is written *n*! and pronounced "*n* factorial." This is the product

$$n \cdot (n - 1) \cdot (n - 2) \cdots 1$$

with 1! equal to 1 and 0! defined to be 1. For example, 5! is the product $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$, which is equal to 120.

Iterative Factorial Approach

You can calculate 5! *iteratively* with a `for` statement, as in:

lick here to view code image

```
In [1]: factorial = 1

In [2]: for number in range(5, 0, -1):
   ...:     factorial *= number
   ...:

In [3]: factorial
Out[3]: 120
```

Recursive Problem Solving

C

Recursive problem-solving approaches have several elements in common. When you call a recursive function to solve a problem, it's actually capable of solving only the *simplest case(s)*, or **base case(s)**. If you call the function with a *base case*, it immediately returns a result. If you call the function with a more complex problem, it typically divides the problem into two pieces—one that the function knows how to do and one that it does not know how to do. To make recursion feasible, this latter piece must be a slightly simpler or smaller version of the original problem. Because this new problem resembles the original problem, the function calls a fresh *copy* of itself to work on the smaller problem—this is referred to as a **recursive call** and is also called the **recursion step**. This concept of separating the problem into two smaller portions is a form of the *divide-and-conquer* approach introduced earlier in the book.

The recursion step executes while the original function call is still active (i.e., it has not finished executing). It can result in many more recursive calls as the function divides each new subproblem into two conceptual pieces. For the recursion to eventually terminate, each time the function calls itself with a simpler version of the original problem, the sequence of smaller and smaller problems must *converge on a base case*. When the function recognizes the base case, it returns a result to the previous copy of the function. A sequence of returns ensues until the original function call returns the final result to the caller.

Recursive Factorial Approach

You can arrive at a recursive factorial representation by observing that $n!$ can be written as:

$$n! = n \cdot (n - 1)!$$

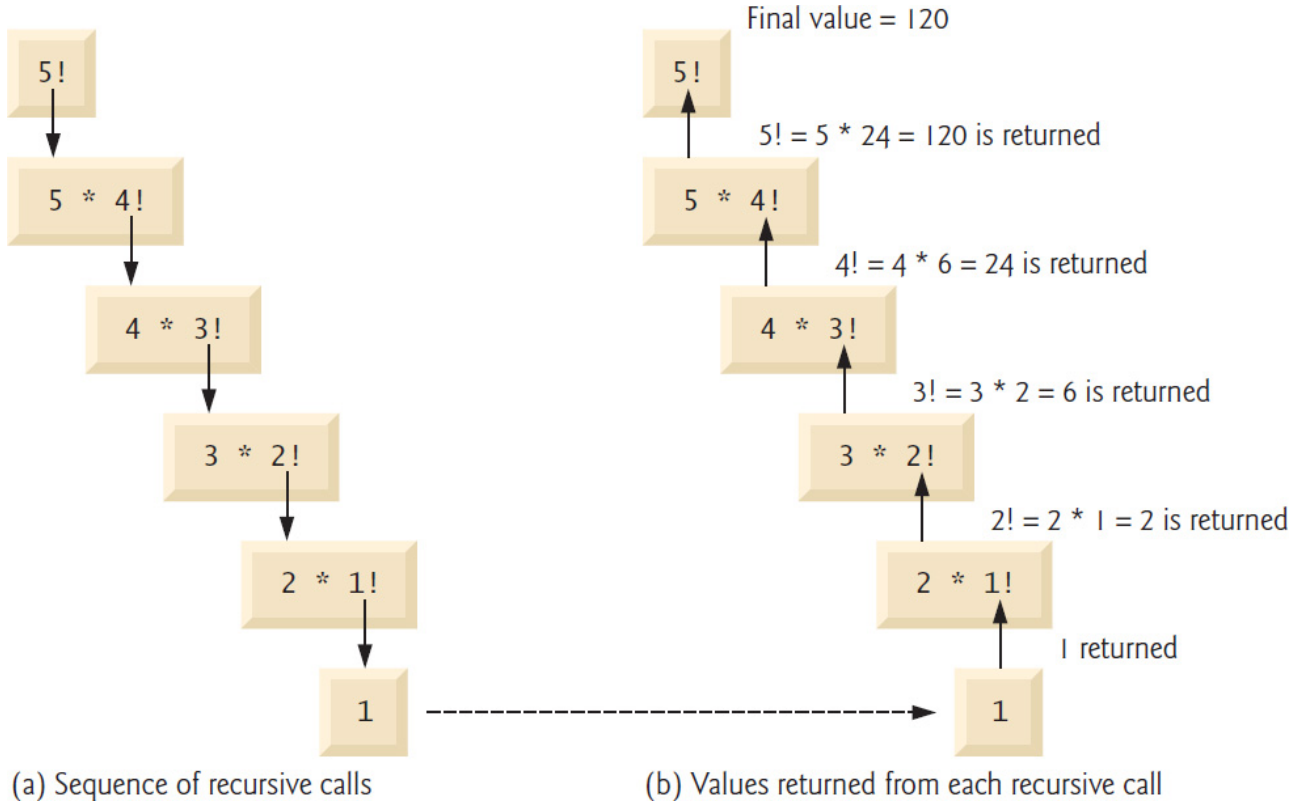
For example, 5! is equal to $5 \cdot 4!$, as in:

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$
$$5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1)$$
$$5! = 5 \cdot (4!)$$

Visualizing Recursion

The evaluation of 5! would proceed as shown below. The left column shows how the succession of recursive calls proceeds until 1! (the base case) is evaluated to be 1, which terminates the recursion. The right column shows from bottom to top the values returned from each recursive call to its caller until the final value is calculated and

returned.



(a) Sequence of recursive calls

(b) Values returned from each recursive call

Implementing a Recursive Factorial Function

The following session uses recursion to calculate and display the factorials of the integers 0 through 10:

lick here to view code image

```
In [4]: def factorial(number):
   ...:     """Return   factorial of number."""
   ...:     if number <= 1:
   ...:         return 1
   ...:     return number *   factorial(number - 1)   #   recursive call
   ...:

In [5]: for i in range(11):
   ...:     print(f'{i}! = {factorial(i)}')
   ...:
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

C

Snippet [4]'s recursive function factorial first determines whether the *terminating condition* number <= 1 is True. If this condition *is* True (the base case), factorial returns 1 and no further recursion is necessary. If number is greater than 1, the second return statement expresses the problem as the product of number and a *recursive call* to factorial that evaluates factorial(number - 1). This is a slightly smaller problem than the original calculation, factorial(number). Note that function factorial must receive a *nonnegative* argument. We do not test for this case.

The loop in snippet [5] calls the factorial function for the values from 0 through 10. The output shows that factorial values grow quickly. *Python does not limit the size of an integer*, unlike many other programming languages.

Indirect Recursion

A recursive function may call another function, which may, in turn, make a call back to the recursive function. This is known as an **indirect recursive call** or **indirect recursion**. For example, function A calls function B, which makes a call back to function A. This is still recursion because the second call to function A is made while the first call to function A is active. That is, the first call to function A has not yet finished executing (because it is waiting on function B to return a result to it) and has not returned to function A's original caller.

Stack Overflow and Infinite Recursion

Of course, the amount of memory in a computer is finite, so only a certain amount of memory can be used to store activation records on the function-call stack. If more recursive function calls occur than can have their activation records stored on the stack, a fatal error known as **stack overflow** occurs. This typically is the result of **infinite recursion**, which can be caused by omitting the base case or writing the recursion step incorrectly so that it does not converge on the base case. This error is analogous to the problem of an *infinite loop* in an *iterative* (nonrecursive) solution.

4.17 FUNCTIONAL-STYLE PROGRAMMING

Like other popular languages, such as Java and C#, Python is not a purely functional language. Rather, it offers "functional-style" features that help you write code which is less likely to contain errors, more concise and easier to read, debug and modify. Functional-style programs also can be easier to parallelize to get better performance on

today's multi-core processors. The chart below lists most of Python's key functional-style programming capabilities and shows in parentheses the chapters in which we initially cover many of them.

Functional-style programming topics

- avoiding side effects (4)
- closures
- declarative programming (4)
- decorators (10)
- dictionary comprehensions (6)
- `filter`/`map`/`reduce` (5)
- `functools` module
- generator expressions (5)
- generator functions
- higher-order functions (5)
- immutability (4)
- internal iteration (4)
- iterators (3)
- `itertools` module (16)
- `lambda` expressions (5)
- lazy evaluation (5)
- list comprehensions (5)
- `operator` module (5, 11, 16)
- pure functions (4)
- `range` function (3, 4)
- reductions (3, 5)
- set comprehensions (6)

We cover most of these features throughout the book—many with code examples and others from a literacy perspective. You've already used list, string and built-in function `range` *iterators* with the `for` statement, and several *reductions* (functions `sum`, `len`, `min` and `max`). We discuss declarative programming, immutability and internal iteration below.

What vs. How

As the tasks you perform get more complicated, your code can become harder to read, debug and modify, and more likely to contain errors. Specifying *how* the code works can become complex.

Functional-style programming lets you simply say *what* you want to do. It hides many

etails of *how* to perform each task. Typically, library code handles the *how* for you. As you'll see, this can eliminate many errors.

Consider the `for` statement in many other programming languages. Typically, *you* must specify all the details of counter-controlled iteration: a control variable, its initial value, how to increment it and a loop-continuation condition that uses the control variable to determine whether to continue iterating. This style of iteration is known as **external iteration** and is error-prone. For example, you might provide an incorrect initializer, increment or loop-continuation condition. External iteration **mutates** (that is, modifies) the control variable, and the `for` statement's suite often mutates other variables as well. Every time you modify variables you could introduce errors. Functional-style programming emphasizes **immutability**. That is, it avoids operations that modify variables' values. We'll say more in the next chapter.

Python's `for` statement and `range` function *hide* most counter-controlled iteration details. You specify *what* values `range` should produce and the variable that should receive each value as it's produced. Function `range` *knows how* to produce those values. Similarly, the `for` statement *knows how* to get each value from `range` and *how* to stop iterating when there are no more values. Specifying *what*, but not *how*, is an important aspect of **internal iteration**—a key functional-style programming concept.

The Python built-in functions `sum`, `min` and `max` each use internal iteration. To total the elements of the list `grades`, you simply declare *what* you want to do—that is, `sum(grades)`. Function `sum` *knows how* to iterate through the list and add each element to the running total. Stating what you *want* done rather than programming *how* to do it is known as **declarative programming**.

Pure Functions

In pure functional programming language you focus on writing pure functions. A **pure function's** result depends only on the argument(s) you pass to it. Also, given a particular argument (or arguments), a pure function always produces the same result. For example, built-in function `sum`'s return value depends only on the iterable you pass to it. Given a list `[1, 2, 3]`, `sum` *always* returns 6 no matter how many times you call it. Also, a pure function does not have *side effects*. For example, even if you pass a *mutable* list to a pure function, the list will contain the same values before and after the function call. When you call the pure function `sum`, it does not modify its argument.

lick here to view code image

```
In [1]: values = [1, 2, 3]

In [2]: sum(values)
Out[2]: 6

In [3]: sum(values)  # same call   always returns same result
Out[3]: 6

In [4]: values
Out[5]: [1, 2, 3]
```

In the next chapter, we'll continue using functional-style programming concepts. Also, you'll see that *functions are objects* that you can pass to other functions as data.

4.18 INTRO TO DATA SCIENCE: MEASURES OF DISPERSION

In our discussion of descriptive statistics, we've considered the measures of central tendency—mean, median and mode. These help us categorize typical values in a group—such as the mean height of your classmates or the most frequently purchased car brand (the mode) in a given country.

When we're talking about a group, the entire group is called the **population**. Sometimes a population is quite large, such as the people likely to vote in the next U.S. presidential election, which is a number in excess of 100,000,000 people. For practical reasons, the polling organizations trying to predict who will become the next president work with carefully selected small subsets of the population known as **samples**. Many of the polls in the 2016 election had sample sizes of about 1000 people.

In this section, we continue discussing basic descriptive statistics. We introduce **measures of dispersion** (also called **measures of variability**) that help you understand how spread out the values are. For example, in a class of students, there may be a bunch of students whose height is close to the average, with smaller numbers of students who are considerably shorter or taller.

For our purposes, we'll calculate each measure of dispersion both by hand and with functions from the module `statistics`, using the following population of 10 six-sided die rolls:

```
1, 3, 4, 2, 6, 5, 3, 4, 5, 2
```

Variance

To determine the **variance**, [5] we begin with the mean of these values—`3.5`. You obtain this result by dividing the sum of the face values, `35`, by the number of rolls, `10`. Next, we subtract the mean from every die value (this produces some negative results):

[5] For simplicity, were calculating the *population variance*. There is a subtle difference between the *population variance* and the *sample variance*. Instead of dividing by *n* (the number of die rolls in our example), sample variance divides by *n* `1`. The difference is pronounced for small samples and becomes insignificant as the sample size increases. The `statistics` module provides the functions `pvariance` and `variance` to calculate the population variance and sample variance, respectively. Similarly, the `statistics` module provides the functions `pstdev` and `stdev` to calculate the population standard deviation and sample standard deviation, respectively.

```
-2.5, -0.5, 0.5, -1.5, 2.5, 1.5, -0.5, 0.5, 1.5, -1.5
```

Then, we square each of these results (yielding only positives):

```
6.25, 0.25, 0.25, 2.25, 6.25, 2.25, 0.25, 0.25, 2.25, 2.25
```

Finally, we calculate the mean of these squares, which is `2.25` (`22.5 / 10`)—this is the **population variance**. Squaring the difference between each die value and the mean of all die values emphasizes **outliers**—the values that are farthest from the mean. As we get deeper into data analytics, sometimes we'll want to pay careful attention to outliers, and sometimes we'll want to ignore them. The following code uses the `statistics` module's **`pvariance`** function to confirm our manual result:

lick here to view code image

```
In [1]: import statistics

In [2]: statistics.pvariance([1,   3, 4, 2, 6, 5, 3, 4, 5, 2])
Out[2]: 2.25
```

Standard Deviation

The **standard deviation** is the square root of the variance (in this case, `1.5`), which tones down the effect of the outliers. The smaller the variance and standard deviation

C

are, the closer the data values are to the mean and the less overall **dispersion** (that is, **spread**) there is between the values and the mean. The following code calculates the **population standard deviation** with the statistics module's `pstdev` function, confirming our manual result:

lick here to view code image

```
In [3]: statistics.pstdev([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
Out[3]: 1.5
```

Passing the `pvariance` function's result to the `math` module's `sqrt` function confirms our result of `1.5`:

lick here to view code image

```
In [4]: import math

In [5]: math.sqrt(statistics.pvariance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2]))
Out[5]: 1.5
```

Advantage of Population Standard Deviation vs. Population Variance

Suppose you've recorded the March Fahrenheit temperatures in your area. You might have 31 numbers such as 19, 32, 28 and 35. The units for these numbers are degrees. When you square your temperatures to calculate the population variance, the units of the population variance become "degrees squared." When you take the square root of the population variance to calculate the population standard deviation, the units once again become degrees, which are the *same* units as your temperatures.

4.19 WRAP-UP

In this chapter, we created custom functions. We imported capabilities from the `random` and `math` modules. We introduced random-number generation and used it to simulate rolling a six-sided die. We packed multiple values into tuples to return more than one value from a function. We also unpacked a tuple to access its values. We discussed using the Python Standard Library's modules to avoid "reinventing the wheel."

We created functions with default parameter values and called functions with keyword arguments. We also defined functions with arbitrary argument lists. We called methods of objects. We discussed how an identifier's scope determines where in your program

C

ou can use it.

We presented more about importing modules. You saw that arguments are passed-by-reference to functions, and how the function-call stack and stack frames support the function-call-and-return mechanism. We also presented a recursive function and began introducing Python's functional-style programming capabilities. We've introduced basic list and tuple capabilities over the last two chapters—in the next chapter, we'll discuss them in detail.

Finally, we continued our discussion of descriptive statistics by introducing measures of dispersion—variance and standard deviation—and calculating them with functions from the Python Standard Library's `statistics` module.

For some types of problems, it's useful to have functions call themselves. A **recursive function** calls itself, either directly or indirectly through another function.

5. Sequences: Lists and Tuples

Objectives

In this chapter, you'll:

- Create and initialize lists and tuples.
- Refer to elements of lists, tuples and strings.
- Sort and search lists, and search tuples.
- Pass lists and tuples to functions and methods.
- Use list methods to perform common manipulations, such as searching for items, sorting a list, inserting items and removing items.
- Use additional Python functional-style programming capabilities, including lambdas and the functional-style programming operations filter, map and reduce.
- Use functional-style list comprehensions to create lists quickly and easily, and use generator expressions to generate values on demand.
- Use two-dimensional lists.
- Enhance your analysis and presentation skills with the Seaborn and Matplotlib visualization libraries.

Outline

5.1 INTRODUCTION

In the last two chapters, we briefly introduced the list and tuple sequence types for representing ordered collections of items. **Collections** are prepackaged data structures consisting of related data items. Examples of collections include your favorite songs on your smartphone, your contacts list, a library's books, your cards in a card game, your favorite sports team's players, the stocks in an investment portfolio, patients in a cancer study and a shopping list. Python's built-in collections enable you to store and access

data conveniently and efficiently. In this chapter, we discuss lists and tuples in more detail.

We'll demonstrate common list and tuple manipulations. You'll see that lists (which are modifiable) and tuples (which are not) have many common capabilities. Each can hold items of the same or different types. Lists can **dynamically resize** as necessary, growing and shrinking at execution time. We discuss one-dimensional and two-dimensional lists.

In the preceding chapter, we demonstrated random-number generation and simulated rolling a six-sided die. We conclude this chapter with our next Intro to Data Science section, which uses the visualization libraries Seaborn and Matplotlib to interactively develop static bar charts containing the die frequencies. In the next chapter's Intro to Data Science section, we'll present an animated visualization in which the bar chart changes *dynamically* as the number of die rolls increases—you'll see the law of large numbers "in action."

5.2 LISTS

Here, we discuss lists in more detail and explain how to refer to particular list **elements**. Many of the capabilities shown in this section apply to all sequence types.

Creating a List

Lists typically store **homogeneous data**, that is, values of the *same* data type. Consider the list `c`, which contains five integer elements:

lick here to view code image

```
In [1]: c = [-45, 6, 0, 72, 1543]

In [2]: c
Out[2]: [-45, 6, 0, 72, 1543]
```

They also may store **heterogeneous data**, that is, data of many different types. For example, the following list contains a student's first name (a string), last name (a string), grade point average (a `float`) and graduation year (an `int`):

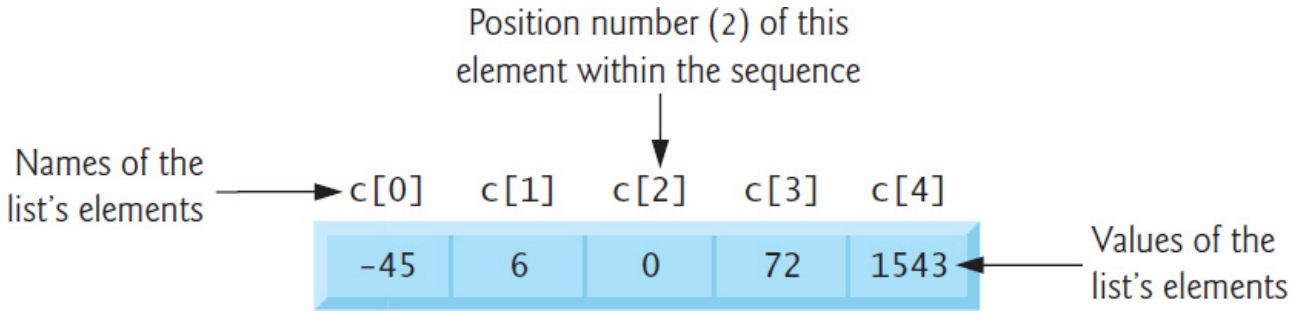
lick here to view code image

```
['Mary', 'Smith', 3.57, 2022]
```

C

Accessing Elements of a List

You reference a list element by writing the list's name followed by the element's **index** (that is, its **position number**) enclosed in square brackets (`[]`, known as the **subscription operator**). The following diagram shows the list `c` labeled with its element names:



The first element in a list has the index `0`. So, in the five-element list `c`, the first element is named `c[0]` and the last is `c[4]`:

```
In [3]: c[0]
Out[3]: -45

In [4]: c[4]
Out[4]: 1543
```

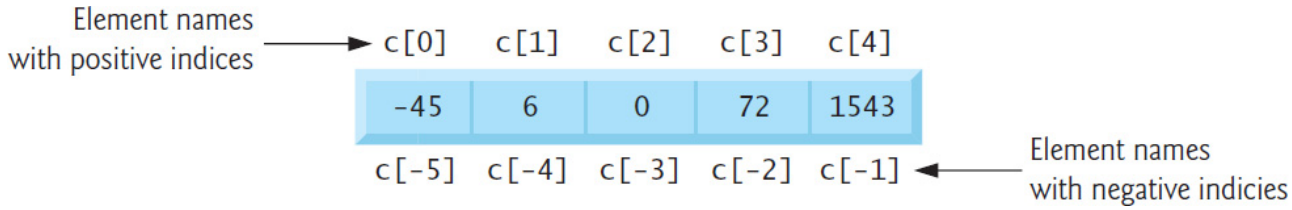
Determining a List's Length

To get a list's length, use the built-in **`len` function**:

```
In [5]: len(c)
Out[5]: 5
```

Accessing Elements from the End of the List with Negative Indices

Lists also can be accessed from the end by using *negative indices*:



So, list `c`'s last element (`c[4]`), can be accessed with `c[-1]` and its first element with `c[-5]`:

```
In [6]: c[-1]
Out[6]: 1543
```

```
In [7]: c[-5]
Out[7]: -45
```

Indices Must Be Integers or Integer Expressions

An index must be an integer or integer expression (or a *slice*, as we'll soon see):

```
In [8]: a = 1

In [9]: b = 2

In [10]: c[a + b]
Out[10]: 72
```

Using a non-integer index value causes a `TypeError`.

Lists Are Mutable

Lists are mutable—their elements can be modified:

lick here to view code image

```
In [11]: c[4] = 17

In [12]: c
Out[12]: [-45, 6, 0, 72, 17]
```

You'll soon see that you also can insert and delete elements, changing the list's length.

Some Sequences Are Immutable

Python's string and tuple sequences are immutable—they cannot be modified. You can get the individual characters in a string, but attempting to assign a new value to one of the characters causes a `TypeError`:

lick here to view code image

```
 n [13]: s = 'hello'

In [14]: s[0]
Out[14]: 'h'

In [15]: s[0] = 'H'
---------------------------------------------------------------------
```

C

```
TypeError                                 Traceback (most recent call last
<ipython-input-15-812ef2514689> in <module>()
----> 1 s[0] = 'H'

TypeError: 'str' object does   not support item assignment
```

ttempting to Access a Nonexistent Element

Using an out-of-range list, tuple or string index causes an `IndexError`:

lick here to view code image

```
In [16]: c[100]
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last
 ipython-input-16-9a31ea1e1a13> in <module>()
----> 1 c[100]

IndexError: list index out   of range
```

Using List Elements in Expressions

List elements may be used as variables in expressions:

```
In [17]: c[0] + c[1] + c[2]
Out[17]: -39
```

Appending to a List with +=

Let's start with an *empty* list `[]`, then use a `for` statement and += to append the values 1 through 5 to the list—the list grows dynamically to accommodate each item:

lick here to view code image

```
In [18]: a_list = []

In [19]: for number in range(1, 6):
    ...:     a_list += [number]
    ...:

In [20]: a_list
Out[20]: [1, 2, 3, 4, 5]
```

When the left operand of += is a list, the right operand must be an *iterable*; otherwise, a `TypeError` occurs. In snippet `[19]`'s suite, the square brackets around `number` create a one-element list, which we append to `a_list`. If the right operand contains multiple elements, += appends them all. The following appends the characters of `'Python'` to the list `letters`:

lick here to view code image

```
In [21]: letters = []

In [22]: letters += 'Python'

In [23]: letters
Out[23]: ['P', 'y', 't', 'h', 'o', 'n']
```

If the right operand of += is a tuple, its elements also are appended to the list. Later in the chapter, we'll use the list method `append` to add items to a list.

Concatenating Lists with +

You can **concatenate** two lists, two tuples or two strings using the + operator. The result is a *new* sequence of the same type containing the left operand's elements followed by the right operand's elements. The original sequences are unchanged:

lick here to view code image

```
In [24]: list1 = [10, 20, 30]

In [25]: list2 = [40, 50]

In [26]: concatenated_list = list1 + list2

In [27]: concatenated_list
Out[27]: [10, 20, 30, 40, 50]
```

A `TypeError` occurs if the + operator's operands are difference sequence types—for example, concatenating a list and a tuple is an error.

Using `for` and `range` to Access List Indices and Values

List elements also can be accessed via their indices and the subscription operator (`[]`):

lick here to view code image

C

```
In [28]: for i in range(len(concatenated_list)):
    ...:     print(f'{i}: {concatenated_list[i]}')
    ...:
0: 10
1: 20
2: 30
3: 40
4: 50
```

The function call `range(len(concatenated_list))` produces a sequence of integers representing `concatenated_list`'s indices (in this case, `0` through `4`). When looping in this manner, you must ensure that indices remain in range. Soon, we'll show a safer way to access element indices and values using built-in function `enumerate`.

Comparison Operators

You can compare entire lists element-by-element using comparison operators:

lick here to view code image

```
In [29]: a = [1, 2, 3]

In [30]: b = [1, 2, 3]

In [31]: c = [1, 2, 3, 4]

In [32]: a == b  # True:   corresponding elements in both are equal
Out[32]: True

In [33]: a == c  # False: a and c   have different elements and lengths
Out[33]: False

In [34]: a < c  # True: a has   fewer elements than c
Out[34]: True

In [35]: c >= b  # True:   elements 0-2 are equal but c has more elements
Out[35]: True
```

5.3 TUPLES

As discussed in the preceding chapter, tuples are immutable and typically store heterogeneous data, but the data can be homogeneous. A tuple's length is its number of elements and cannot change during program execution.

Creating Tuples

C

o create an empty tuple, use empty parentheses:

```
In [1]: student_tuple = ()

In [2]: student_tuple
Out[2]: ()

In [3]: len(student_tuple)
Out[3]: 0
```

Recall that you can pack a tuple by separating its values with commas:

lick here to view code image

```
In [4]: student_tuple = 'John',   'Green', 3.3

In [5]: student_tuple
Out[5]: ('John', 'Green', 3.3)

In [6]: len(student_tuple)
Out[6]: 3
```

When you output a tuple, Python always displays its contents in parentheses. You may surround a tuple's comma-separated list of values with optional parentheses:

lick here to view code image

```
In [7]: another_student_tuple = ('Mary',   'Red', 3.3)

In [8]: another_student_tuple
Out[8]: ('Mary', 'Red', 3.3)
```

The following code creates a one-element tuple:

lick here to view code image

```
In [9]: a_singleton_tuple = ('red',)    # note the comma

In [10]: a_singleton_tuple
Out[10]: ('red',)
```

The comma (`,`) that follows the string `'red'` identifies `a_singleton_tuple` as a tuple—the parentheses are optional. If the comma were omitted, the parentheses would

be redundant, and `a_singleton_tuple` would simply refer to the string `'red'` rather than a tuple.

Accessing Tuple Elements

A tuple's elements, though related, are often of multiple types. Usually, you do not iterate over them. Rather, you access each individually. Like list indices, tuple indices start at 0. The following code creates `time_tuple` representing an hour, minute and second, displays the tuple, then uses its elements to calculate the number of seconds since midnight—note that we perform a *different* operation with each value in the tuple:

lick here to view code image

```
In [11]: time_tuple = (9, 16, 1)

In [12]: time_tuple
Out[12]: (9, 16, 1)

In [13]: time_tuple[0] * 3600 + time_tuple[1] * 60 + time_tuple[2]
Out[13]: 33361
```

Assigning a value to a tuple element causes a `TypeError`.

Adding Items to a String or Tuple

As with lists, the += augmented assignment statement can be used with strings and tuples, even though they're *immutable*. In the following code, after the two assignments, `tuple1` and `tuple2` refer to the *same* tuple object:

```
In [14]: tuple1 = (10, 20, 30)

In [15]: tuple2 = tuple1

In [16]: tuple2
Out[16]: (10, 20, 30)
```

Concatenating the tuple `(40, 50)` to `tuple1` creates a *new* tuple, then assigns a reference to it to the variable `tuple1`—`tuple2` still refers to the original tuple:

lick here to view code image

```
In [17]: tuple1 += (40, 50)
```

C

```
In [18]: tuple1
Out[18]: (10, 20, 30, 40, 50)

In [19]: tuple2
Out[19]: (10, 20, 30)
```

For a string or tuple, the item to the right of += must be a string or tuple, respectively—mixing types causes a `TypeError`.

Appending Tuples to Lists

You can use += to append a tuple to a list:

lick here to view code image

```
In [20]: numbers = [1, 2, 3, 4, 5]

In [21]: numbers += (6, 7)

In [22]: numbers
Out[22]: [1, 2, 3, 4, 5, 6, 7]
```

Tuples May Contain Mutable Objects

Let's create a `student_tuple` with a first name, last name and list of grades:

lick here to view code image

```
In [23]: student_tuple = ('Amanda',   'Blue', [98, 75, 87])
```

Even though the tuple is immutable, its list element is mutable:

lick here to view code image

```
In [24]: student_tuple[2][1] = 85

In [25]: student_tuple
Out[25]: ('Amanda', 'Blue', [98, 85, 87])
```

In the *double-subscripted name* `student_tuple[2][1]`, Python views `student_tuple[2]` as the element of the tuple containing the list `[98, 75, 87]`, then uses `[1]` to access the list element containing `75`. The assignment in snippet `[24]`

C

replaces that grade with 85.

5.4 UNPACKING SEQUENCES

The previous chapter introduced tuple unpacking. You can unpack any sequence's elements by assigning the sequence to a comma-separated list of variables. A `ValueError` occurs if the number of variables to the left of the assignment symbol is not identical to the number of elements in the sequence on the right:

lick here to view code image

```
In [1]: student_tuple = ('Amanda',    [98, 85, 87])

In [2]: first_name, grades = student_tuple

In [3]: first_name
Out[3]: 'Amanda'

In [4]: grades
Out[4]: [98, 85, 87]
```

The following code unpacks a string, a list and a sequence produced by `range`:

lick here to view code image

```
In [5]: first, second = 'hi'

In [6]: print(f'{first}    {second}')
h  i

In [7]: number1, number2, number3 = [2,    3, 5]

In [8]: print(f'{number1}    {number2}  {number3}')
2  3  5

In [9]: number1, number2, number3 = range(10, 40, 10)

In [10]: print(f'{number1}    {number2}  {number3}')
10  20  30
```

Swapping Values Via Packing and Unpacking

You can swap two variables' values using sequence packing and unpacking:

lick here to view code image

C

```
In [11]: number1 = 99

In [12]: number2 = 22

In [13]: number1, number2 = (number2, number1)

In [14]: print(f'number1 = {number1}; number2 = {number2}')
number1 = 22; number2 = 99
```

Accessing Indices and Values Safely with Built-in Function `enumerate`

Earlier, we called `range` to produce a sequence of index values, then accessed list elements in a `for` loop using the index values and the subscription operator (`[]`). This is error-prone because you could pass the wrong arguments to `range`. If any value produced by `range` is an out-of-bounds index, using it as an index causes an `IndexError`.

The preferred mechanism for accessing an element's index *and* value is the built-in function **`enumerate`**. This function receives an iterable and creates an iterator that, for each element, returns a tuple containing the element's index and value. The following code uses the built-in function **`list`** to create a list containing `enumerate`'s results:

lick here to view code image

```
In [15]: colors = ['red', 'orange', 'yellow']

In [16]: list(enumerate(colors))
Out[16]: [(0, 'red'), (1, 'orange'), (2, 'yellow')]
```

Similarly the built-in function **`tuple`** creates a tuple from a sequence:

lick here to view code image

```
In [17]: tuple(enumerate(colors))
Out[17]: ((0, 'red'), (1, 'orange'), (2, 'yellow'))
```

The following `for` loop unpacks each tuple returned by `enumerate` into the variables `index` and `value` and displays them:

lick here to view code image

```
In [18]: for index, value in enumerate(colors):
```

C

```
...:     print(f'{index}: {value}')
...:
0: red
1: orange
2: yellow
```

Creating a Primitive Bar Chart

The following script creates a primitive **bar chart** where each bar's length is made of asterisks (*) and is proportional to the list's corresponding element value. We use the function `enumerate` to access the list's indices and values safely. To run this example, change to this chapter's `ch05` examples folder, then enter:

```
ipython fig05_01.py
```

or, if you're in IPython already, use the command:

```
run fig05_01.py
```

lick here to view code image

```
# fig05_01.py
"""Displaying a bar chart"""
numbers = [19, 3, 15, 7, 11]

print('\nCreating a bar chart from numbers:')
print(f'Index{"Value":>8}   Bar')

for index, value in   enumerate(numbers):
    print(f'{index:>5}{value:>8}   {"*" *   value}')
```

lick here to view code image

```
Creating a bar chart from numbers:
Index   Value   Bar
    0      19   *******************
    1       3   ***
    2      15   ***************
    3       7   *******
    4      11   ***********
```

The `for` statement uses `enumerate` to get each element's index and value, then

C

displays a formatted line containing the index, the element value and the corresponding bar of asterisks. The expression

```
"*" * value
```

creates a string consisting of `value` asterisks. When used with a sequence, the multiplication operator (`*`) *repeats* the sequence—in this case, the string `"*"`—`value` times. Later in this chapter, we'll use the open-source Seaborn and Matplotlib libraries to display a publication--quality bar chart visualization.

5.5 SEQUENCE SLICING

You can **slice** sequences to create new sequences of the same type containing *subsets* of the original elements. Slice operations can modify mutable sequences—those that do *not* modify a sequence work identically for lists, tuples and strings.

Specifying a Slice with Starting and Ending Indices

Let's create a slice consisting of the elements at indices 2 through 5 of a list:

lick here to view code image

```
In [1]: numbers = [2, 3, 5, 7, 11, 13, 17, 19]

In [2]: numbers[2:6]
Out[2]: [5, 7, 11, 13]
```

The slice *copies* elements from the *starting index* to the left of the colon (`2`) up to, but not including, the *ending index* to the right of the colon (`6`). The original list is not modified.

Specifying a Slice with Only an Ending Index

If you omit the starting index, `0` is assumed. So, the slice `numbers[:6]` is equivalent to the slice `numbers[0:6]`:

lick here to view code image

```
In [3]: numbers[:6]
Out[3]: [2, 3, 5, 7, 11, 13]

In [4]: numbers[0:6]
```

C

```
Out[4]: [2, 3, 5, 7, 11, 13]
```

Specifying a Slice with Only a Starting Index

If you omit the ending index, Python assumes the sequence's length (`8` here), so snippet `[5]`'s slice contains the elements of `numbers` at indices `6` and `7`:

lick here to view code image

```
In [5]: numbers[6:]
Out[5]: [17, 19]

In [6]: numbers[6:len(numbers)]
Out[6]: [17, 19]
```

Specifying a Slice with No Indices

Omitting both the start and end indices copies the entire sequence:

lick here to view code image

```
In [7]: numbers[:]
Out[7]: [2, 3, 5, 7, 11, 13, 17, 19]
```

Though slices create new objects, slices make **shallow copies** of the elements—that is, they copy the elements' references but not the objects they point to. So, in the snippet above, the new list's elements refer to the *same objects* as the original list's elements, rather than to separate copies. In the "Array-Oriented Programming with NumPy" chapter, we'll explain *deep* copying, which actually copies the referenced objects themselves, and we'll point out when deep copying is preferred.

Slicing with Steps

The following code uses a *step* of `2` to create a slice with every other element of `numbers`:

```
In [8]: numbers[::2]
Out[8]: [2, 5, 11, 17]
```

We omitted the start and end indices, so `0` and `len(numbers)` are assumed, respectively.

C

Slicing with Negative Indices and Steps

You can use a negative step to select slices in *reverse* order. The following code concisely creates a new list in reverse order:

lick here to view code image

```
In [9]: numbers[::-1]
Out[9]: [19, 17, 13, 11, 7, 5, 3, 2]
```

This is equivalent to:

lick here to view code image

```
In [10]: numbers[-1:-9:-1]
Out[10]: [19, 17, 13, 11, 7, 5, 3, 2]
```

Modifying Lists Via Slices

You can modify a list by assigning to a slice of it—the rest of the list is unchanged. The following code replaces `numbers`' first three elements, leaving the rest unchanged:

lick here to view code image

```
In [11]: numbers[0:3] = ['two', 'three', 'five']

In [12]: numbers
Out[12]: ['two', 'three', 'five', 7, 11, 13, 17, 19]
```

The following deletes only the first three elements of `numbers` by assigning an *empty* list to the three-element slice:

lick here to view code image

```
In [13]: numbers[0:3] = []

In [14]: numbers
Out[14]: [7, 11, 13, 17, 19]
```

The following assigns a list's elements to a slice of every other element of `numbers`:

lick here to view code image

C

```
In [15]: numbers = [2, 3, 5, 7, 11, 13, 17, 19]

In [16]: numbers[::2] = [100, 100, 100, 100]

In [17]: numbers
Out[17]: [100, 3, 100, 7, 100, 13, 100, 19]

In [18]: id(numbers)
Out[18]: 4434456648
```

Let's delete all the elements in `numbers`, leaving the *existing* list empty:

lick here to view code image

```
In [19]: numbers[:] = []

In [20]: numbers
Out[20]: []

In [21]: id(numbers)
Out[21]: 4434456648
```

Deleting `numbers`' contents (snippet `[19]`) is different from assigning `numbers` a *new* empty list `[]` (snippet `[22]`). To prove this, we display `numbers`' identity after each operation. The identities are different, so they represent separate objects in memory:

```
In [22]: numbers = []

In [23]: numbers
Out[23]: []

In [24]: id(numbers)
Out[24]: 4406030920
```

When you assign a new object to a variable (as in snippet `[21]`), the original object will be garbage collected if no other variables refer to it.

5.6 `DEL` STATEMENT

The **`del` statement** also can be used to remove elements from a list and to delete variables from the interactive session. You can remove the element at any valid index or the element(s) from any valid slice.

Deleting the Element at a Specific List Index

C

et's create a list, then use `del` to remove its last element:

lick here to view code image

```
In [1]: numbers = list(range(0,   10))

In [2]: numbers
Out[2]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

In [3]: del numbers[-1]

In [4]: numbers
Out[4]: [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Deleting a Slice from a List

The following deletes the list's first two elements:

```
In [5]: del numbers[0:2]

In [6]: numbers
Out[6]: [2, 3, 4, 5, 6, 7, 8]
```

The following uses a step in the slice to delete every other element from the entire list:

```
In [7]: del numbers[::2]

In [8]: numbers
Out[8]: [3, 5, 7]
```

Deleting a Slice Representing the Entire List

The following code deletes all of the list's elements:

```
In [9]: del numbers[:]

In [10]: numbers
Out[10]: []
```

Deleting a Variable from the Current Session

The `del` statement can delete any variable. Let's delete `numbers` from the interactive session, then attempt to display the variable's value, causing a `NameError`:

C

lick here to view code image

```
In [11]: del numbers

In [12]: numbers
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last
 ipython-input-12-426f8401232b> in <module>()
----> 1 numbers

NameError: name 'numbers' is   not defined
```

5.7 PASSING LISTS TO FUNCTIONS

In the last chapter, we mentioned that all objects are passed by reference and demonstrated passing an immutable object as a function argument. Here, we discuss references further by examining what happens when a program passes a mutable list object to a function.

Passing an Entire List to a Function

Consider the function `modify_elements`, which receives a reference to a list and multiplies each of the list's element values by `2`:

lick here to view code image

```
In [1]: def modify_elements(items):
   ...:     """Multiplies   all element values in items by 2."""
   ...:     for i in range(len(items)):
   ...:         items[i] *= 2
   ...:

In [2]: numbers = [10, 3, 7, 1, 9]

In [3]: modify_elements(numbers)

In [4]: numbers
Out[4]: [20, 6, 14, 2, 18]
```

Function `modify_elements`' `items` parameter receives a reference to the *original* list, so the statement in the loop's suite modifies each element in the original list object.

Passing a Tuple to a Function

e

When you pass a tuple to a function, attempting to modify the tuple's immutable elements results in a `TypeError`:

lick here to view code image

```
In [5]: numbers_tuple = (10, 20, 30)

In [6]: numbers_tuple
Out[6]: (10, 20, 30)

In [7]: modify_elements(numbers_tuple)
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last
 ipython-input-7-9339741cd595> in <module>()
----> 1 modify_elements(numbers_tuple)

<ipython-input-1-27acb8f8f44c> in   modify_elements(items)
      2     """Multiplies   all element values in items by 2."""
      3     for i in range(len(items)):
----> 4         items[i] *= 2
      5
      6

TypeError: 'tuple' object   does not support item assignment
```

Recall that tuples may contain mutable objects, such as lists. Those objects still can be modified when a tuple is passed to a function.

A Note Regarding Tracebacks

The previous traceback shows the *two* snippets that led to the `TypeError`. The first is snippet `[7]`'s function call. The second is snippet `[1]`'s function definition. Line numbers precede each snippet's code. We've demonstrated mostly single-line snippets. When an exception occurs in such a snippet, it's always preceded by `----> 1`, indicating that line 1 (the snippet's only line) caused the exception. Multiline snippets like the definition of `modify_elements` show consecutive line numbers starting at 1. The notation `----> 4` above indicates that the exception occurred in line 4 of `modify_elements`. No matter how long the traceback is, the last line of code with `---->` caused the exception.

5.8 SORTING LISTS

Sorting enables you to arrange data either in ascending or descending order.

Sorting a List in Ascending Order

List method **`sort`** *modifies* a list to arrange its elements in ascending order:

lick here to view code image

```
In [1]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [2]: numbers.sort()

In [3]: numbers
Out[3]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Sorting a List in Descending Order

To sort a list in descending order, call list method `sort` with the optional keyword argument **`reverse-`** set to `True` (`False` is the default):

lick here to view code image

```
In [4]: numbers.sort(reverse=True)

In [5]: numbers
Out[5]: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Built-In Function `sorted`

Built-in function **`sorted`** *returns a new list* containing the sorted elements of its argument *sequence*—the original sequence is *unmodified*. The following code demonstrates function `sorted` for a list, a string and a tuple:

lick here to view code image

```
In [6]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [7]: ascending_numbers = sorted(numbers)

In [8]: ascending_numbers
Out[8]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

In [9]: numbers
Out[9]: [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [10]: letters = 'fadgchjebi'

In [11]: ascending_letters = sorted(letters)
```

C

```
In [12]: ascending_letters
Out[12]: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

In [13]: letters
Out[13]: 'fadgchjebi'

In [14]: colors = ('red', 'orange', 'yellow', 'green', 'blue')

In [15]: ascending_colors = sorted(colors)

In [16]: ascending_colors
Out[16]: ['blue', 'green', 'orange', 'red', 'yellow']

In [17]: colors
Out[17]: ('red', 'orange', 'yellow', 'green', 'blue')
```

Use the optional keyword argument `reverse` with the value `True` to sort the elements in descending order.

5.9 SEARCHING SEQUENCES

Often, you'll want to determine whether a sequence (such as a list, tuple or string) contains a value that matches a particular **key** value. **Searching** is the process of locating a key.

List Method `index`

List method **`index`** takes as an argument a search key—the value to locate in the list—then searches through the list from index 0 and returns the index of the *first* element that matches the search key:

lick here to view code image

```
In [1]: numbers = [3, 7, 1, 4, 2, 8, 5, 6]

In [2]: numbers.index(5)
Out[2]: 6
```

A `ValueError` occurs if the value you're searching for is not in the list.

Specifying the Starting Index of a Search

Using method `index`'s optional arguments, you can search a subset of a list's elements. You can use `*=` to *multiply a sequence*—that is, append a sequence to itself multiple

C

times. After the following snippet, `numbers` contains two copies of the original list's contents:

lick here to view code image

```
In [3]: numbers *= 2

In [4]: numbers
Out[4]: [3, 7, 1, 4, 2, 8, 5, 6, 3, 7, 1, 4, 2, 8, 5, 6]
```

The following code searches the updated list for the value `5` starting from index `7` and continuing through the end of the list:

```
In [5]: numbers.index(5, 7)
Out[5]: 14
```

Specifying the Starting and Ending Indices of a Search

Specifying the starting and ending indices causes `index` to search from the starting index up to but not including the ending index location. The call to `index` in snippet `[5]`:

```
numbers.index(5, 7)
```

assumes the length of `numbers` as its optional third argument and is equivalent to:

lick here to view code image

```
numbers.index(5, 7, len(numbers))
```

The following looks for the value `7` in the range of elements with indices `0` through `3`:

lick here to view code image

```
In [6]: numbers.index(7, 0, 4)
Out[6]: 1
```

Operators `in` and `not in`

Operator `in` tests whether its right operand's iterable contains the left operand's value:

C

```
In [7]: 1000 in numbers
Out[7]: False

In [8]: 5 in   numbers
Out[8]: True
```

Similarly, operator `not in` tests whether its right operand's iterable does *not* contain the left operand's value:

```
In [9]: 1000 not in numbers
Out[9]: True

In [10]: 5 not in numbers
Out[10]: False
```

Using Operator `in` to Prevent a `ValueError`

You can use the operator `in` to ensure that calls to method `index` do not result in `ValueError`s for search keys that are not in the corresponding sequence:

lick here to view code image

```
In [11]: key = 1000

In [12]: if key in numbers:
    ...:     print(f'found {key}   at index {numbers.index(search_key)}')
    ...: else:
    ...:     print(f'{key} not found')
    ...:
1000 not found
```

Built-In Functions `any` and `all`

Sometimes you simply need to know whether *any* item in an iterable is `True` or whether *all* the items are `True`. The built-in function **`any`** returns `True` if any item in its iterable argument is `True`. The built-in function **`all`** returns `True` if all items in its iterable argument are `True`. Recall that nonzero values are `True` and 0 is `False`. Non-empty iterable objects also evaluate to `True`, whereas any empty iterable evaluates to `False`. Functions `any` and `all` are additional examples of internal iteration in functional-style programming.

C

5.10 OTHER LIST METHODS

Lists also have methods that add and remove elements. Consider the list `color_names`:

lick here to view code image

```
In [1]: color_names = ['orange', 'yellow', 'green']
```

Inserting an Element at a Specific List Index

Method **`insert`** adds a new item at a specified index. The following inserts `'red'` at index `0`:

lick here to view code image

```
In [2]: color_names.insert(0, 'red')

In [3]: color_names
Out[3]: ['red', 'orange', 'yellow', 'green']
```

Adding an Element to the End of a List

You can add a new item to the end of a list with method **`append`**:

lick here to view code image

```
In [4]: color_names.append('blue')

In [5]: color_names
Out[5]: ['red', 'orange', 'yellow', 'green', 'blue']
```

Adding All the Elements of a Sequence to the End of a List

Use list method **`extend`** to add all the elements of another sequence to the end of a list:

lick here to view code image

```
In [6]: color_names.extend(['indigo', 'violet'])

In [7]: color_names
Out[7]: ['red', 'orange', 'yellow', 'green', 'blue',   'indigo', 'violet']
```

C

This is the equivalent of using +=. The following code adds all the characters of a string then all the elements of a tuple to a list:

lick here to view code image

```
In [8]: sample_list = []

In [9]: s = 'abc'

In [10]: sample_list.extend(s)

In [11]: sample_list
Out[11]: ['a', 'b', 'c']

In [12]: t = (1, 2, 3)

In [13]: sample_list.extend(t)

In [14]: sample_list
Out[14]: ['a', 'b', 'c', 1, 2, 3]
```

Rather than creating a temporary variable, like `t`, to store a tuple before appending it to a list, you might want to pass a tuple directly to `extend`. In this case, the tuple's parentheses are required, because `extend` expects one iterable argument:

lick here to view code image

```
In [15]: sample_list.extend((4,   5, 6))  # note the extra parentheses

In [16]: sample_list
Out[16]: ['a', 'b', 'c', 1, 2, 3, 4, 5, 6]
```

A `TypeError` occurs if you omit the required parentheses.

Removing the First Occurrence of an Element in a List

Method **`remove`** deletes the first element with a specified value—a `ValueError` occurs if `remove`'s argument is not in the list:

lick here to view code image

```
In [17]: color_names.remove('green')

In [18]: color_names
Out[18]: ['red', 'orange', 'yellow', 'blue', 'indigo',   'violet']
```

C

Emptying a List

To delete all the elements in a list, call method **clear**:

```
In [19]: color_names.clear()

In [20]: color_names
Out[20]: []
```

This is the equivalent of the previously shown slice assignment

```
color_names[:] = []
```

Counting the Number of Occurrences of an Item

List method **count** searches for its argument and returns the number of times it is found:

lick here to view code image

```
In [21]: responses = [1, 2, 5, 4, 3, 5, 2, 1, 3, 3,
    ...:              1, 4, 3, 3, 3, 2, 3, 3, 2, 2]
    ...:

In [22]: for i in range(1, 6):
   ...:     print(f'{i} appears {responses.count(i)} times   in responses'
   ...:
1 appears 3 times in responses
2 appears 5 times in responses
3 appears 8 times in responses
4 appears 2 times in responses
5 appears 2 times in responses
```

Reversing a List's Elements

List method **reverse** reverses the contents of a list in place, rather than creating a reversed copy, as we did with a slice previously:

lick here to view code image

```
In [23]: color_names = ['red', 'orange', 'yellow', 'green', 'blue']
```

C

```
In [24]: color_names.reverse()

In [25]: color_names
Out[25]: ['blue', 'green', 'yellow', 'orange', 'red']
```

Copying a List

List method `copy` returns a *new* list containing a *shallow* copy of the original list:

lick here to view code image

```
In [26]: copied_list = color_names.copy()

In [27]: copied_list
Out[27]: ['blue', 'green', 'yellow', 'orange', 'red']
```

This is equivalent to the previously demonstrated slice operation:

```
copied_list = color_names[:]
```

5.11 SIMULATING STACKS WITH LISTS

The preceding chapter introduced the function-call stack. Python does not have a built-in stack type, but you can think of a stack as a constrained list. You *push* using list method `append`, which adds a new element to the *end* of the list. You *pop* using list method **pop** with no arguments, which removes and returns the item at the *end* of the list.

Let's create an empty list called `stack`, push (`append`) two strings onto it, then `pop` the strings to confirm they're retrieved in last-in, first-out (LIFO) order:

lick here to view code image

```
 n [1]: stack = []

In [2]: stack.append('red')

In [3]: stack
Out[3]: ['red']

In [4]: stack.append('green')

In [5]: stack
Out[5]: ['red', 'green']
```

C

```
In [6]: stack.pop()
Out[6]: 'green'

In [7]: stack
Out[7]: ['red']

In [8]: stack.pop()
Out[8]: 'red'

In [9]: stack
Out[9]: []

In [10]: stack.pop()
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last
<ipython-input-10-50ea7ec13fbe> in <module>()
----> 1 stack.pop()

IndexError: pop from empty   list
```

or each `pop` snippet, the value that `pop` removes and returns is displayed. Popping from an empty stack causes an `IndexError`, just like accessing a nonexistent list element with `[]`. To prevent an `IndexError`, ensure that `len(stack)` is greater than `0` before calling `pop`. You can run out of memory if you keep pushing items faster than you pop them.

You also can use a list to simulate another popular collection called a **queue** in which you insert at the back and delete from the front. Items are retrieved from queues in **first-in, first-out (FIFO) order**.

5.12 LIST COMPREHENSIONS

Here, we continue discussing *functional-style* features with **list comprehensions**—a concise and convenient notation for creating new lists. List comprehensions can replace many `for` statements that iterate over existing sequences and create new lists, such as:

lick here to view code image

```
In [1]: list1 = []

In [2]: for item in range(1, 6):
   ...:     list1.append(item)
   ...:

In [3]: list1
```

E

```
Out[3]: [1, 2, 3, 4, 5]
```

Using a List Comprehension to Create a List of Integers

We can accomplish the same task in a single line of code with a list comprehension:

lick here to view code image

```
In [4]: list2 = [item for item in range(1, 6)]

In [5]: list2
Out[5]: [1, 2, 3, 4, 5]
```

Like snippet `[2]`'s `for` statement, the list comprehension's **`for` clause**

```
for item in range(1, 6)
```

iterates over the sequence produced by `range(1, 6)`. For each `item`, the list comprehension evaluates the expression to the left of the `for` clause and places the expression's value (in this case, the `item` itself) in the new list. Snippet `[4]`'s particular comprehension could have been expressed more concisely using the function `list`:

```
list2 = list(range(1, 6))
```

Mapping: Performing Operations in a List Comprehension's Expression

A list comprehension's expression can perform tasks, such as calculations, that **map** elements to new values (possibly of different types). Mapping is a common functional-style programming operation that produces a result with the *same* number of elements as the original data being mapped. The following comprehension maps each value to its cube with the expression `item ** 3`:

lick here to view code image

```
In [6]: list3 = [item ** 3 for item in range(1, 6)]

In [7]: list3
Out[7]: [1, 8, 27, 64, 125]
```

Filtering: List Comprehensions with `if` Clauses

C

Another common functional-style programming operation is **filtering** elements to select only those that match a condition. This typically produces a list with *fewer* elements than the data being filtered. To do this in a list comprehension, use the **`if` clause**. The following includes in `list4` only the even values produced by the `for` clause:

lick here to view code image

```
In [8]: list4 = [item for item in range(1, 11) if item % 2 == 0]

In [9]: list4
Out[9]: [2, 4, 6, 8, 10]
```

List Comprehension That Processes Another List's Elements

The `for` clause can process any iterable. Let's create a list of lowercase strings and use a list comprehension to create a new list containing their uppercase versions:

lick here to view code image

```
In [10]: colors = ['red', 'orange', 'yellow', 'green', 'blue']

In [11]: colors2 = [item.upper() for   item in colors]

In [12]: colors2
Out[12]: ['RED', 'ORANGE', 'YELLOW', 'GREEN', 'BLUE']

In [13]: colors
Out[13]: ['red', 'orange', 'yellow', 'green', 'blue']
```

5.13 GENERATOR EXPRESSIONS

A **generator expression** is similar to a list comprehension, but creates an iterable **generator object** that produces values *on demand*. This is known as **lazy evaluation**. List comprehensions use **greedy evaluation**—they create lists *immediately* when you execute them. For large numbers of items, creating a list can take substantial memory and time. So generator expressions can reduce your program's memory consumption and improve performance if the whole list is not needed at once.

Generator expressions have the same capabilities as list comprehensions, but you define them in parentheses instead of square brackets. The generator expression in snippet `[2]` squares and returns only the odd values in `numbers`:

C

lick here to view code image

```
In [1]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [2]: for value in (x ** 2 for x in numbers if x % 2 != 0):
   ...:     print(value, end='  ')
   ...:
9  49  1  81  25
```

To show that a generator expression does not create a list, let's assign the preceding snippet's generator expression to a variable and evaluate the variable:

lick here to view code image

```
In [3]: squares_of_odds = (x ** 2   for x in numbers if x % 2 != 0)

In [3]: squares_of_odds
Out[3]: <generator object <genexpr> at   0x1085e84c0>
```

The text `"generator object <genexpr>"` indicates that `square_of_odds` is a generator object that was created from a generator expression (`genexpr`).

5.14 FILTER, MAP AND REDUCE

The preceding section introduced several functional-style features—list comprehensions, filtering and mapping. Here we demonstrate the built-in `filter` and `map` functions for filtering and mapping, respectively. We continue discussing reductions in which you process a collection of elements into a *single* value, such as their count, total, product, average, minimum or maximum.

Filtering a Sequence's Values with the Built-In `filter` Function

Let's use built-in function **`filter`** to obtain the odd values in `numbers`:

lick here to view code image

```
In [1]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [2]: def is_odd(x):
   ...:     """Returns   True only if x is odd."""
   ...:     return x % 2 != 0
   ...:
```

C

```
In [3]: list(filter(is_odd, numbers))
Out[3]: [3, 7, 1, 9, 5]
```

Like data, Python functions are objects that you can assign to variables, pass to other functions and return from functions. Functions that receive other functions as arguments are a functional-style capability called **higher-order functions**. For example, `filter`'s first argument must be a function that receives one argument and returns `True` if the value should be included in the result. The function `is_odd` returns `True` if its argument is odd. The `filter` function calls `is_odd` once for each value in its second argument's iterable (`numbers`). Higher-order functions may also return a function as a result.

Function `filter` returns an iterator, so `filter`'s results are not produced until you iterate through them. This is another example of lazy evaluation. In snippet `[3]`, function `list` iterates through the results and creates a list containing them. We can obtain the same results as above by using a list comprehension with an `if` clause:

lick here to view code image

```
In [4]: [item for item in numbers if is_odd(item)]
Out[4]: [3, 7, 1, 9, 5]
```

Using a `lambda` Rather than a Function

For simple functions like `is_odd` that `return` only a *single expression's value*, you can use a **lambda expression** (or simply a **lambda**) to define the function inline where it's needed—typically as it's passed to another function:

lick here to view code image

```
In [5]: list(filter(lambda x: x   % 2 != 0, numbers))
Out[5]: [3, 7, 1, 9, 5]
```

We pass `filter`'s return value (an iterator) to function `list` here to convert the results to a list and display them.

A lambda expression is an *anonymous function*—that is, a *function without a name*. In the `filter` call

lick here to view code image

C

```
filter(lambda x: x % 2 != 0, numbers)
```

the first argument is the lambda

```
lambda x: x % 2 != 0
```

A lambda begins with the **`lambda`** keyword followed by a comma-separated parameter list, a colon (`:`) and an expression. In this case, the parameter list has one parameter named `x`. A `lambda` *implicitly* returns its expression's value. So any simple function of the form

lick here to view code image

```
def function_name(parameter_list):
    return expression
```

may be expressed as a more concise `lambda` of the form

lick here to view code image

```
lambda parameter_list: expression
```

Mapping a Sequence's Values to New Values

Let's use built-in function **`map`** with a `lambda` to square each value in `numbers`:

lick here to view code image

```
In [6]: numbers
Out[6]: [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [7]: list(map(lambda x: x ** 2, numbers))
Out[7]: [100, 9, 49, 1, 81, 16, 4, 64, 25, 36]
```

Function `map`'s first argument is a function that receives one value and returns a new value—in this case, a `lambda` that squares its argument. The second argument is an iterable of values to map. Function `map` uses lazy evaluation. So, we pass to the `list` function the iterator that `map` returns. This enables us to iterate through and create a list of the mapped values. Here's an equivalent list comprehension:

C

lick here to view code image

```
In [8]: [item ** 2 for item in numbers]
Out[8]: [100, 9, 49, 1, 81, 16, 4, 64, 25, 36]
```

Combining `filter` and `map`

You can combine the preceding `filter` and `map` operations as follows:

lick here to view code image

```
In [9]: list(map(lambda x: x ** 2,
    ...:             filter(lambda   x: x % 2 != 0, numbers)))
    ...:
Out[9]: [9, 49, 1, 81, 25]
```

There is a lot going on in snippet `[9]`, so let's take a closer look at it. First, `filter` returns an iterable representing only the odd values of `numbers`. Then `map` returns an iterable representing the squares of the filtered values. Finally, `list` uses `map`'s iterable to create the list. You might prefer the following list comprehension to the preceding snippet:

lick here to view code image

```
In [10]: [x ** 2 for x in numbers if x % 2 != 0]
Out[10]: [9, 49, 1, 81, 25]
```

For each value of `x` in `numbers`, the expression `x ** 2` is performed only if the condition `x % 2 != 0` is `True`.

Reduction: Totaling the Elements of a Sequence with `sum`

As you know reductions process a sequence's elements into a single value. You've performed reductions with the built-in functions `len`, `sum`, `min` and `max`. You also can create custom reductions using the `functools` module's `reduce` function. See `ttps://docs.python.org/3/library/functools.html` for a code example. When we investigate big data and Hadoop in hapter 16, we'll demonstrate MapReduce programming, which is based on the filter, map and reduce operations in functional-style programming.

5.15 OTHER SEQUENCE PROCESSING FUNCTIONS

Python provides other built-in functions for manipulating sequences.

Finding the Minimum and Maximum Values Using a Key Function

We've previously shown the built-in reduction functions `min` and `max` using arguments, such as `int`s or lists of `int`s. Sometimes you'll need to find the minimum and maximum of more complex objects, such as strings. Consider the following comparison:

```
In [1]: 'Red' < 'orange'
Out[1]: True
```

The letter `'R'` "comes after" `'o'` in the alphabet, so you might expect `'Red'` to be less than `'orange'` and the condition above to be `False`. However, strings are compared by their characters' underlying *numerical values*, and lowercase letters have *higher* numerical values than uppercase letters. You can confirm this with built-in function **`ord`**, which returns the numerical value of a character:

```
In [2]: ord('R')
Out[2]: 82

In [3]: ord('o')
Out[3]: 111
```

Consider the list `colors`, which contains strings with uppercase and lowercase letters:

lick here to view code image

```
In [4]: colors = ['Red', 'orange', 'Yellow', 'green', 'Blue']
```

Let's assume that we'd like to determine the minimum and maximum strings using *alphabetical* order, not *numerical* (lexicographical) order. If we arrange colors alphabetically

lick here to view code image

```
'Blue', 'green', 'orange', 'Red', 'Yellow'
```

you can see that `'Blue'` is the minimum (that is, closest to the beginning of the

C

alphabet), and `'Yellow'` is the maximum (that is, closest to the end of the alphabet).

Since Python compares strings using numerical values, you must first convert each string to all lowercase or all uppercase letters. Then their numerical values will also represent *alphabetical* ordering. The following snippets enable `min` and `max` to determine the minimum and maximum strings alphabetically:

lick here to view code image

```
In [5]: min(colors, key=lambda   s: s.lower())
Out[5]: 'Blue'

In [6]: max(colors, key=lambda   s: s.lower())
Out[6]: 'Yellow'
```

The `key` keyword argument must be a one-parameter function that returns a value. In this case, it's a `lambda` that calls string method **`lower`** to get a string's lowercase version. Functions `min` and `max` call the `key` argument's function for each element and use the results to compare the elements.

Iterating Backward Through a Sequence

Built-in function **`reversed`** returns an iterator that enables you to iterate over a sequence's values backward. The following list comprehension creates a new list containing the squares of `numbers`' values in reverse order:

lick here to view code image

```
In [7]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [7]: reversed_numbers = [item for   item in reversed(numbers)]

In [8]: reversed_numbers
Out[8]: [36, 25, 64, 4, 16, 81, 1, 49, 9, 100]
```

Combining Iterables into Tuples of Corresponding Elements

Built-in function **`zip`** enables you to iterate over *multiple* iterables of data at the *same* time. The function receives as arguments any number of iterables and returns an iterator that produces tuples containing the elements at the same index in each. For example, snippet `[11]`'s call to `zip` produces the tuples `('Bob', 3.5)`, `('Sue', 4.0)` and `('Amanda', 3.75)` consisting of the elements at index 0, 1 and 2 of each

C

list, respectively:

lick here to view code image

```
In [9]: names = ['Bob', 'Sue', 'Amanda']

In [10]: grade_point_averages = [3.5,   4.0, 3.75]

In [11]: for name, gpa in zip(names, grade_point_averages):
    ...:     print(f'Name={name}; GPA={gpa}')
    ...:
Name=Bob; GPA=3.5
Name=Sue; GPA=4.0
Name=Amanda; GPA=3.75
```

We unpack each tuple into `name` and `gpa` and display them. Function `zip`'s shortest argument determines the number of tuples produced. Here both have the same length.

5.16 TWO-DIMENSIONAL LISTS

Lists can contain other lists as elements. A typical use of such nested (or multidimensional) lists is to represent **tables** of values consisting of information arranged in **rows** and **columns**. To identify a particular table element, we specify *two* indices—by convention, the first identifies the element's row, the second the element's column.

Lists that require two indices to identify an element are called **two-dimensional lists** (or **double-indexed lists** or **double-subscripted lists**). Multidimensional lists can have more than two indices. Here, we introduce two-dimensional lists.

Creating a Two-Dimensional List

Consider a two-dimensional list with three rows and four columns (i.e., a 3-by-4 list) that might represent the grades of three students who each took four exams in a course:

lick here to view code image

```
In [1]: a = [[77, 68, 86, 73], [96, 87, 89, 81], [70, 90, 86, 81]]
```

Writing the list as follows makes its row and column tabular structure clearer:

lick here to view code image

C

```
a = [[77, 68, 86, 73],  # first student's grades
     [96, 87, 89, 81],  # second student's grades
     [70, 90, 86, 81]]  # third student's grades
```

Illustrating a Two-Dimensional List

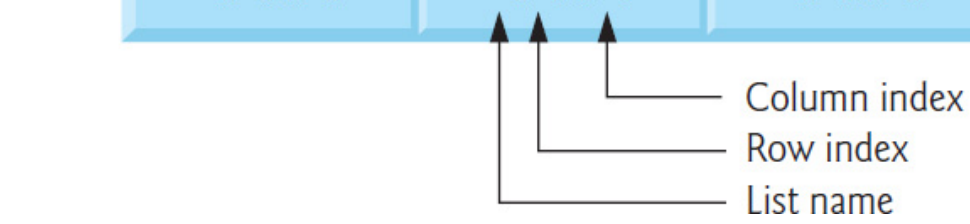
The diagram below shows the list `a`, with its rows and columns of exam grade values:

	Column 0	Column 1	Column 2	Column 3
Row 0	77	68	86	73
Row 1	96	87	89	81
Row 2	70	90	86	81

Identifying the Elements in a Two-Dimensional List

The following diagram shows the names of list `a`'s elements:

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]



Every element is identified by a name of the form `a[i][j]`—`a` is the list's name, and *i* and *j* are the indices that uniquely identify each element's row and column, respectively. The element names in row 0 all have 0 as the first index. The element names in column 3 all have 3 as the second index.

In the two-dimensional list `a`:

- `77`, `68`, `86` and `73` initialize `a[0][0]`, `a[0][1]`, `a[0][2]` and `a[0][3]`, respectively,
- `96`, `87`, `89` and `81` initialize `a[1][0]`, `a[1][1]`, `a[1][2]` and `a[1][3]`,

respectively, and

- `70`, `90`, `86` and `81` initialize `a[2][0]`, `a[2][1]`, `a[2][2]` and `a[2][3]`, respectively.

A list with *m* rows and *n* columns is called an **m-by-n list** and has *m* × *n* elements.

The following nested `for` statement outputs the rows of the preceding two-dimensional list one row at a time:

lick here to view code image

```
In [2]: for row in a:
   ...:     for item in row:
   ...:         print(item, end=' ')
   ...:     print()
   ...:
77 68 86 73
96 87 89 81
70 90 86 81
```

How the Nested Loops Execute

Let's modify the nested loop to display the list's name and the row and column indices and value of each element:

lick here to view code image

```
In [3]: for i, row in enumerate(a):
   ...:     for j, item in enumerate(row):
   ...:         print(f'a[{i}][{j}]={item} ', end=' ')
   ...:     print()
   ...:
a[0][0]=77  a[0][1]=68  a[0][2]=86  a[0][3]=73
a[1][0]=96  a[1][1]=87  a[1][2]=89  a[1][3]=81
a[2][0]=70  a[2][1]=90  a[2][2]=86  a[2][3]=81
```

The outer `for` statement iterates over the two-dimensional list's rows one row at a time. During each iteration of the outer `for` statement, the inner `for` statement iterates over *each* column in the current row. So in the first iteration of the outer loop, `row` 0 is

```
[77, 68, 86, 73]
```

C

and the nested loop iterates through this list's four elements `a[0][0]=77`, `a[0][1]=68`, `a[0][2]=86` and `a[0][3]=73`.

In the second iteration of the outer loop, `row` 1 is

```
[96, 87, 89, 81]
```

and the nested loop iterates through this list's four elements `a[1][0]=96`, `a[1][1]=87`, `a[1][2]=89` and `a[1][3]=81`.

In the third iteration of the outer loop, `row` 2 is

```
[70, 90, 86, 81]
```

and the nested loop iterates through this list's four elements `a[2][0]=70`, `a[2][1]=90`, `a[2][2]=86` and `a[2][3]=81`.

In the "Array-Oriented Programming with NumPy" chapter, we'll cover the NumPy library's `ndarray` collection and the Pandas library's `DataFrame` collection. These enable you to manipulate multidimensional collections more concisely and conveniently than the two-dimensional list manipulations you've seen in this section.

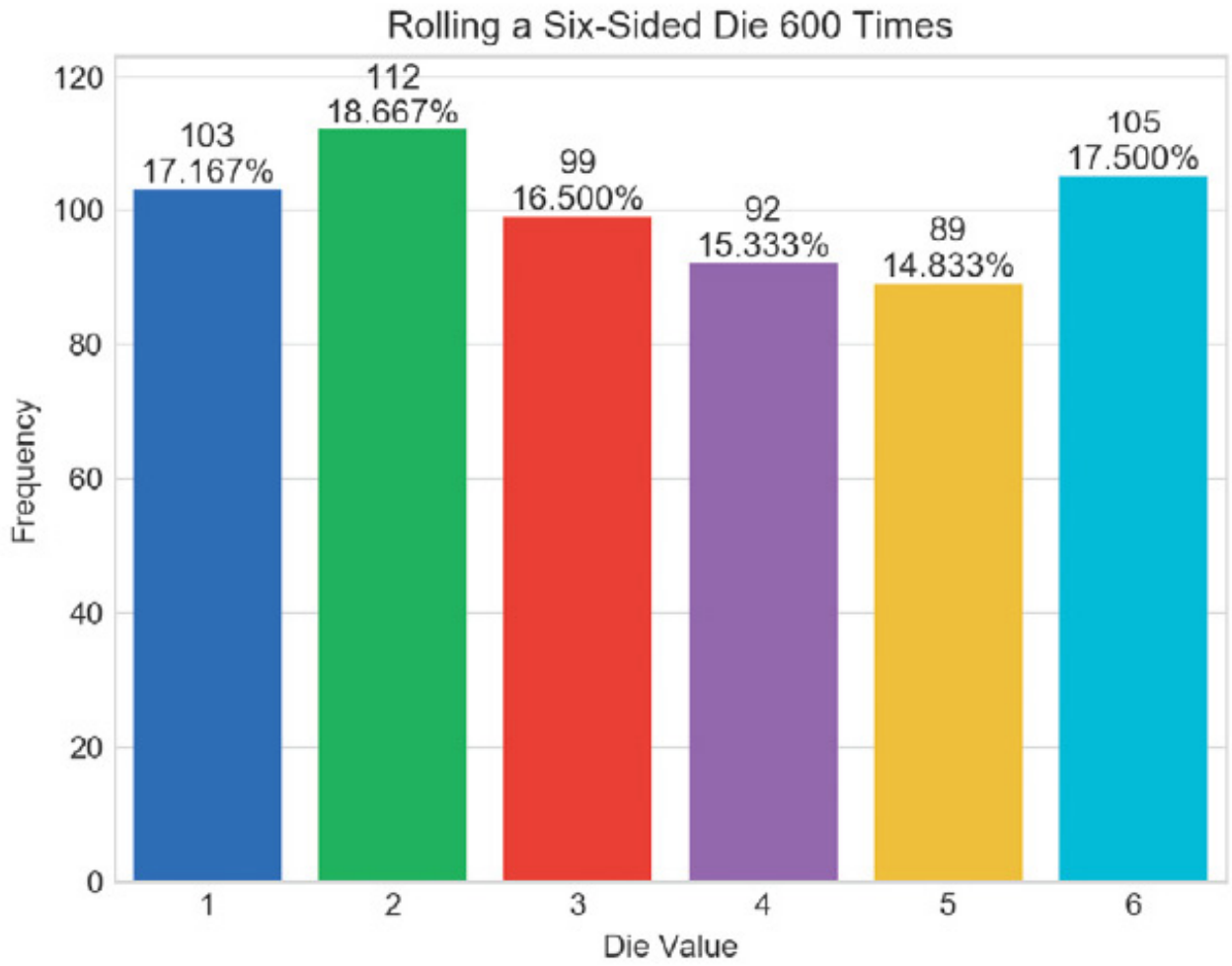
5.17 INTRO TO DATA SCIENCE: SIMULATION AND STATIC VISUALIZATIONS

The last few chapters' Intro to Data Science sections discussed basic descriptive statistics. Here, we focus on visualizations, which help you "get to know" your data. Visualizations give you a powerful way to understand data that goes beyond simply looking at raw data.

We use two open-source visualization libraries—Seaborn and Matplotlib—to display *static* bar charts showing the final results of a six-sided-die-rolling simulation. The **Seaborn visualization library** is built over the **Matplotlib visualization library** and simplifies many Matplotlib operations. We'll use aspects of both libraries, because some of the Seaborn operations return objects from the Matplotlib library. In the next chapter's Intro to Data Science section, we'll make things "come alive" with *dynamic visualizations*.

5.17.1 Sample Graphs for 600, 60,000 and 6,000,000 Die Rolls

he screen capture below shows a vertical bar chart that for 600 die rolls summarizes the frequencies with which each of the six faces appear, and their percentages of the total. Seaborn refers to this type of graph as a **bar plot**:



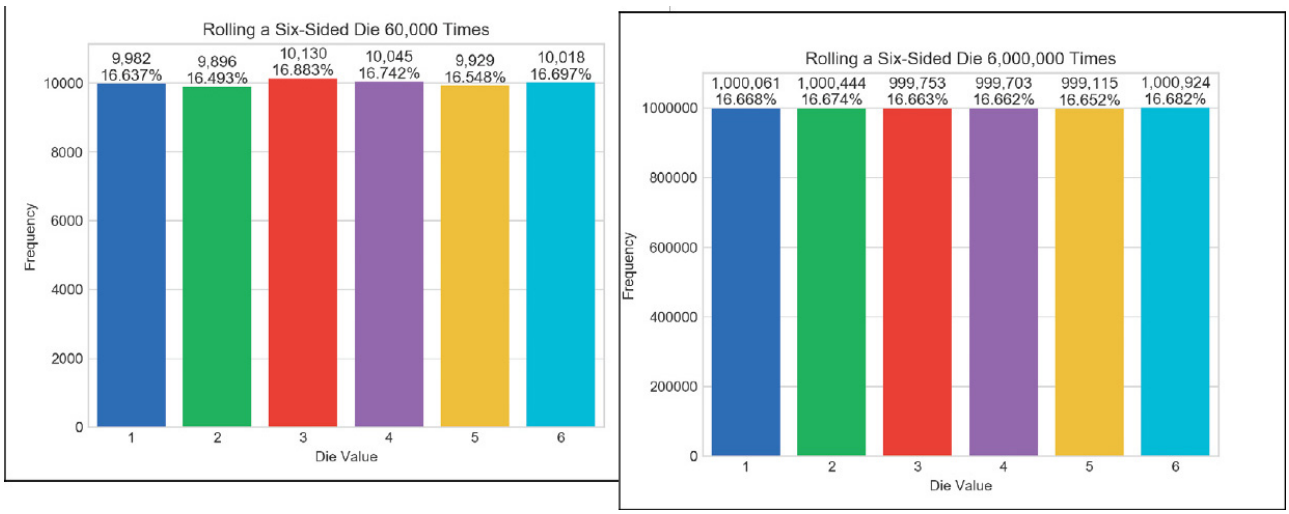
Here we expect about 100 occurrences of each die face. However, with such a small number of rolls, none of the frequencies is exactly 100 (though several are close) and most of the percentages are not close to 16.667% (about 1/6th). As we run the simulation for 60,000 die rolls, the bars will become much closer in size. At 6,000,000 die rolls, they'll appear to be exactly the same size. This is the " aw of large numbers" at work. The next chapter will show the lengths of the bars changing dynamically.

We'll discuss how to control the plot's appearance and contents, including:

- the graph title inside the window (**Rolling a Six-Sided Die 600 Times**),
- the descriptive labels **Die Value** for the *x*-axis and **Frequency** for the *y*-axis,
- the text displayed above each bar, representing the *frequency* and *percentage* of the total rolls, and
- the bar colors.

We'll use various Seaborn default options. For example, Seaborn determines the text labels along the *x*-axis from the die face values 1–6 and the text labels along the *y*-axis from the actual die frequencies. Behind the scenes, Matplotlib determines the positions and sizes of the bars, based on the window size and the magnitudes of the values the bars represent. It also positions the **Frequency** axis's numeric labels based on the actual die frequencies that the bars represent. There are many more features you can customize. You should tweak these attributes to your personal preferences.

The first screen capture below shows the results for 60,000 die rolls—imagine trying to do this by hand. In this case, we expect about 10,000 of each face. The second screen capture below shows the results for 6,000,000 rolls—surely something you'd never do by hand! In this case, we expect about 1,000,000 of each face, and the frequency bars appear to be identical in length (they're close but not exactly the same length). Note that with more die rolls, the frequency percentages are much closer to the expected 16.667%.



5.17.2 Visualizing Die-Roll Frequencies and Percentages

In this section, you'll interactively develop the bar plots shown in the preceding section.

Launching IPython for Interactive Matplotlib Development

IPython has built-in support for interactively developing Matplotlib graphs, which you also need to develop Seaborn graphs. Simply launch IPython with the command:

```
ipython --matplotlib
```

Importing the Libraries

First, let's import the libraries we'll use:

lick here to view code image

```
In [1]: import matplotlib.pyplot   as plt

In [2]: import numpy as np

In [3]: import random

In [4]: import seaborn as sns
```

1. The **`matplotlib.pyplot` module** contains the Matplotlib library's graphing capabilities that we use. This module typically is imported with the name `plt`.
2. The NumPy (Numerical Python) library includes the function `unique` that we'll use to summarize the die rolls. The **`numpy` module** typically is imported as `np`.
3. The `random` module contains Python's random-number- generation functions.
4. The **`seaborn` module** contains the Seaborn library's graphing capabilities we use. This module typically is imported with the name `sns`. Search for why this curious abbreviation was chosen.

Rolling the Die and Calculating Die Frequencies

Next, let's use a *list comprehension* to create a list of 600 random die values, then use NumPy's **`unique`** function to determine the unique roll values (most likely all six possible face values) and their frequencies:

lick here to view code image

```
In [5]: rolls = [random.randrange(1,   7) for i in range(600)]

In [6]: values, frequencies = np.unique(rolls,   return_counts=True)
```

The NumPy library provides the high-performance **`ndarray`** collection, which is typically much faster than lists. [1] Though we do not use `ndarray` directly here, the NumPy `unique` function expects an `ndarray` argument and returns an `ndarray`. If you pass a list (like `rolls`), NumPy converts it to an `ndarray` for better performance. The `ndarray` that `unique` returns we'll simply assign to a variable for use by a Seaborn plotting function.

[1] Well run a performance comparison in hapter 7 where we discuss `ndarray` in

C

depth.

Specifying the keyword argument **`return_counts`**=`True` tells `unique` to count each unique value's number of occurrences. In this case, `unique` returns a tuple of two one-dimensional `ndarray`s containing the sorted unique values and the corresponding frequencies, respectively. We unpack the tuple's `ndarray`s into the variables `values` and `frequencies`. If `return_counts` is `False`, only the list of unique values is returned.

Creating the Initial Bar Plot

Let's create the bar plot's title, set its style, then graph the die faces and frequencies:

lick here to view code image

```
In [7]: title = f'Rolling a Six-Sided   Die {len(rolls):,} Times'

In [8]: sns.set_style('whitegrid')

In [9]: axes = sns.barplot(x=values, y=frequencies,   palette='bright')
```

Snippet `[7]`'s f-string includes the number of die rolls in the bar plot's title. The comma (,) format specifier in

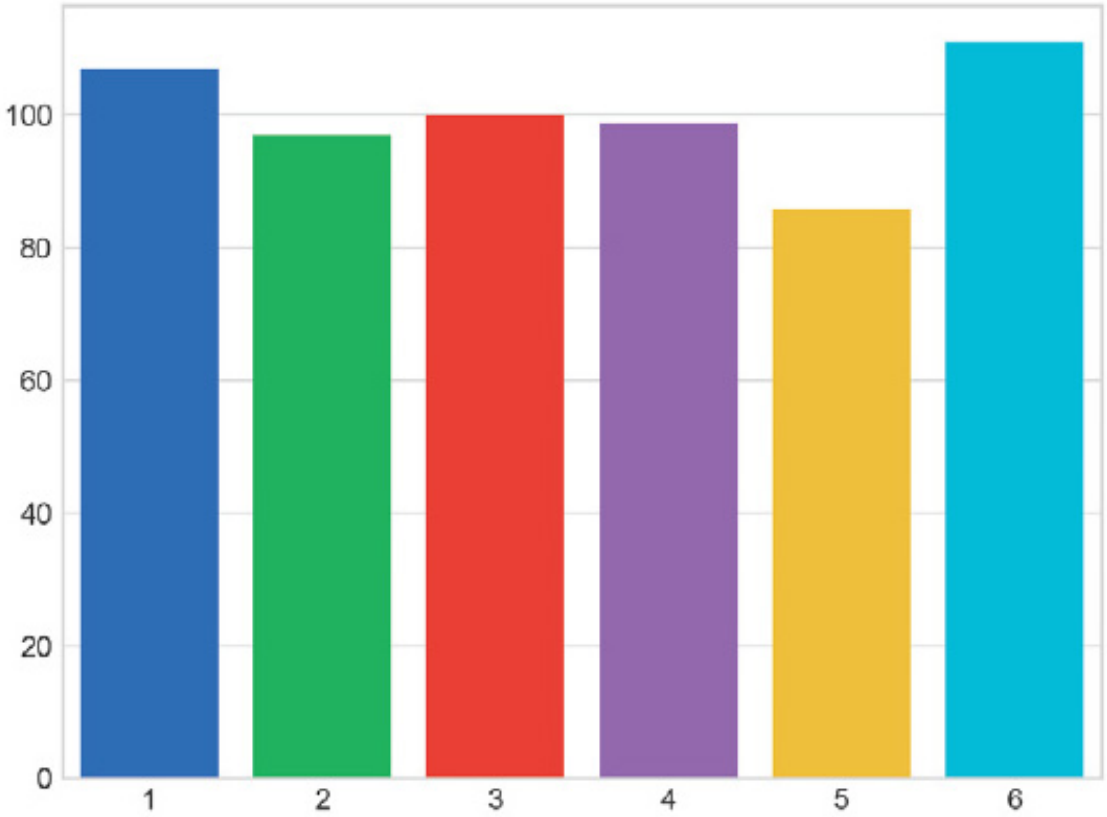
```
{len(rolls):,}
```

displays the number with *thousands separators*—so, 60000 would be displayed as 60,000.

By default, Seaborn plots graphs on a plain white background, but it provides several styles to choose from (`'darkgrid'`, `'whitegrid'`, `'dark'`, `'white'` and `'ticks'`). Snippet `[8]` specifies the `'whitegrid'` style, which displays light-gray horizontal lines in the vertical bar plot. These help you see more easily how each bar's height corresponds to the numeric frequency labels at the bar plot's left side.

Snippet `[9]` graphs the die frequencies using Seaborn's **`barplot` function**. When you execute this snippet, the following window appears (because you launched IPython with the `--matplotlib` option):

C



Seaborn interacts with Matplotlib to display the bars by creating a Matplotlib **`Axes`** object, which manages the content that appears in the window. Behind the scenes, Seaborn uses a Matplotlib **`Figure`** object to manage the window in which the `Axes` will appear. Function `barplot`'s first two arguments are `ndarray`s containing the *x*-axis and *y*-axis values, respectively. We used the optional `palette` keyword argument to choose Seaborn's predefined color palette `'bright'`. You can view the palette options at:

```
ttps://seaborn.pydata.org/tutorial/color_palettes.html
```

Function `barplot` returns the `Axes` object that it configured. We assign this to the variable `axes` so we can use it to configure other aspects of our final plot. Any changes you make to the bar plot after this point will appear *immediately* when you execute the corresponding snippet.

Setting the Window Title and Labeling the *x*- and *y*-Axes

The next two snippets add some descriptive text to the bar plot:

lick here to view code image

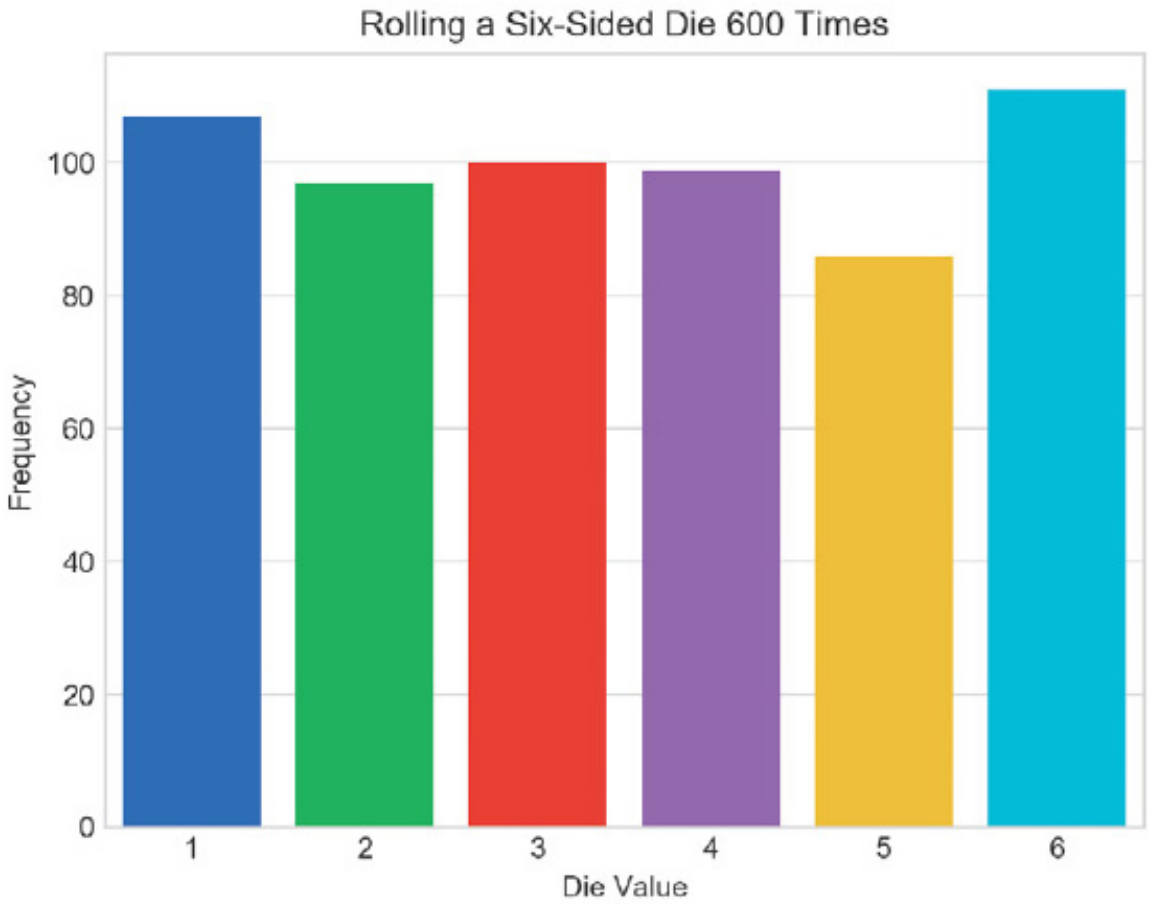
```
In [10]: axes.set_title(title)
Out[10]: Text(0.5,1,'Rolling a Six-Sided Die 600 Times')
```

€

```
In [11]: axes.set(xlabel='Die Value',   ylabel='Frequency')
Out[11]: [Text(92.6667,0.5,'Frequency'), Text(0.5,58.7667,'Die   Value')]
```

Snippet `[10]` uses the `axes` object's **`set_title`** method to display the `title` string centered above the plot. This method returns a `Text` object containing the title and its *location* in the window, which IPython simply displays as output for confirmation. You can ignore the `Out[]`s in the snippets above.

Snippet `[11]` add labels to each axis. The **`set`** method receives keyword arguments for the `Axes` object's properties to set. The method displays the `xlabel` text along the *x*-axis, and the `ylabel`– text along the *y*-axis, and returns a list of `Text` objects containing the labels and their locations. The bar plot now appears as follows:



Finalizing the Bar Plot

The next two snippets complete the graph by making room for the text above each bar, then displaying it:

lick here to view code image

```
In [12]: axes.set_ylim(top=max(frequencies) * 1.10)
```

C

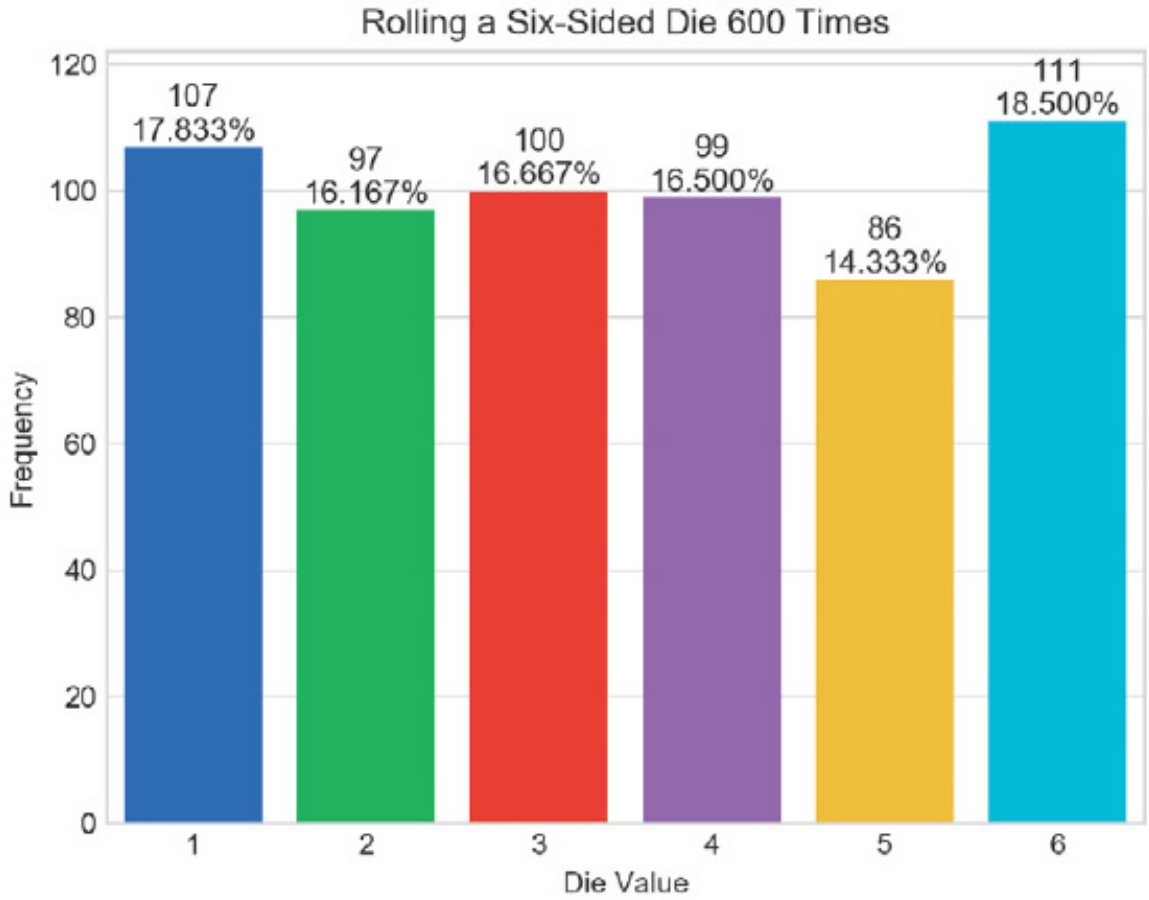
```
Out[12]: (0.0, 122.10000000000001)

In [13]: for bar, frequency in zip(axes.patches, frequencies):
    ...:     text_x = bar.get_x() + bar.get_width() / 2.0
    ...:     text_y = bar.get_height()
    ...:     text = f'{frequency:,}\n{frequency / len(rolls):.3%}'
    ...:     axes.text(text_x, text_y, text,
    ...:               fontsize=11,   ha='center', va='bottom')
    ...:
```

To make room for the text above the bars, snippet `[12]` scales the *y*-axis by 10%. We chose this value via experimentation. The `Axes` object's **`set_ylim`** method has many optional keyword arguments. Here, we use only `top` to change the maximum value represented by the *y*-axis. We multiplied the largest frequency by 1.10 to ensure that the *y*-axis is 10% taller than the tallest bar.

Finally, snippet `[13]` displays each bar's frequency value and percentage of the total rolls. The `axes` object's `patches` collection contains two-dimensional colored shapes that represent the plot's bars. The `for` statement uses `zip` to iterate through the `patches` and their corresponding `frequency` values. Each iteration unpacks into `bar` and `frequency` one of the tuples `zip` returns. The `for` statement's suite operates as follows:

- The first statement calculates the center *x*-coordinate where the text will appear. We calculate this as the sum of the bar's left-edge *x*-coordinate (`bar.get_x()`) and half of the bar's width (`bar.get_width() / 2.0`).
- The second statement gets the *y*-coordinate where the text will appear —`bar.get_y()` represents the bar's top.
- The third statement creates a two-line string containing that bar's frequency and the corresponding percentage of the total die rolls.
- The last statement calls the `Axes` object's **`text`** method to display the text above the bar. This method's first two arguments specify the text's *x*–*y* position, and the third argument is the text to display. The keyword argument `ha` specifies the *horizontal alignment*—we centered text horizontally around the *x*-coordinate. The keyword argument `va` specifies the *vertical alignment*—we aligned the bottom of the text with at the *y*-coordinate. The final bar plot is shown below:



Rolling Again and Updating the Bar Plot—Introducing IPython Magics

Now that you've created a nice bar plot, you probably want to try a different number of die rolls. First, clear the existing graph by calling Matplotlib's **`cla`** (clear axes) function:

```
In [14]: plt.cla()
```

IPython provides special commands called **magics** for conveniently performing various tasks. Let's use the **`%recall` magic** to get snippet `[5]`, which created the `rolls` list, and place the code at the next `In []` prompt:

lick here to view code image

```
In [15]: %recall 5

In [16]: rolls = [random.randrange(1,   7) for i in range(600)]
```

You can now edit the snippet to change the number of rolls to `60000`, then press *Enter* to create a new list:

lick here to view code image

C

```
In [16]: rolls = [random.randrange(1,   7) for i in range(60000)]
```

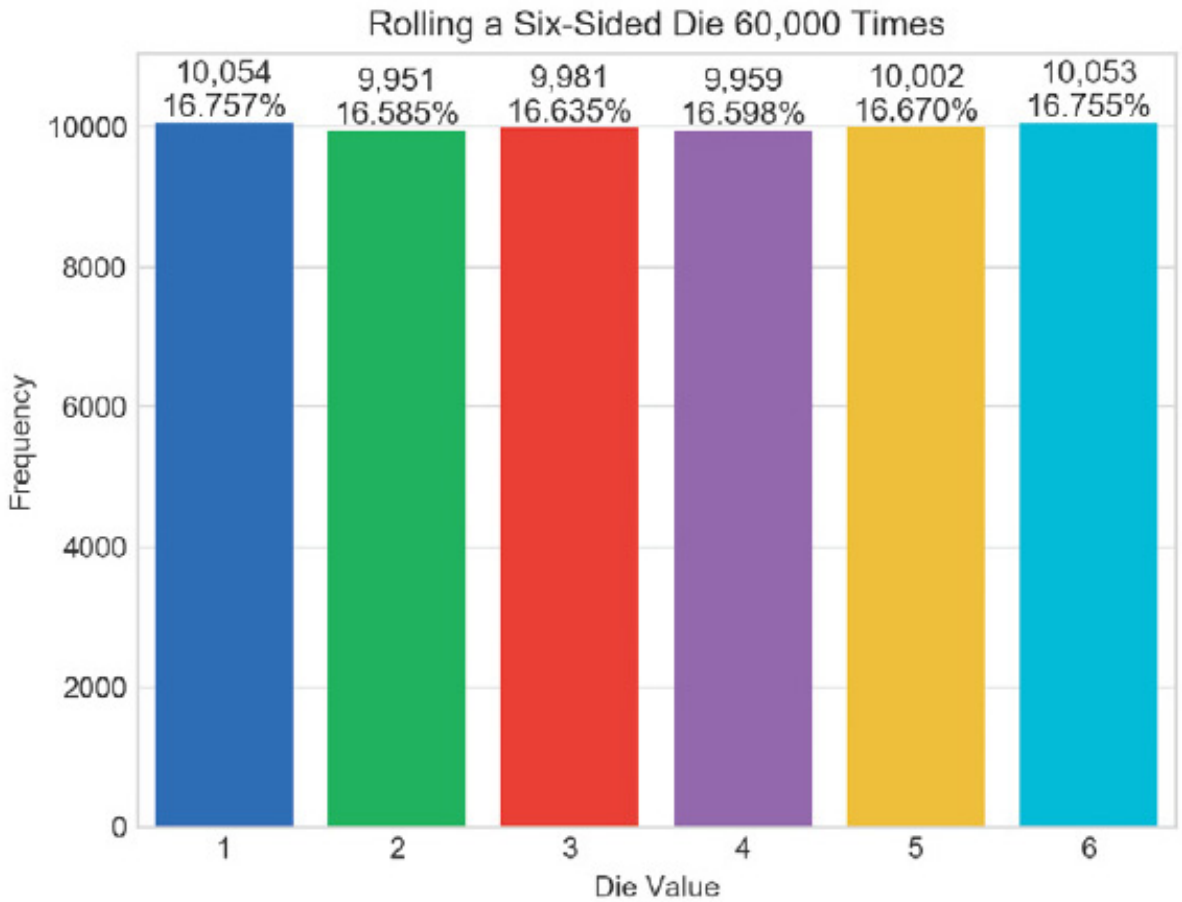
Next, recall snippets `[6]` through `[13]`. This displays all the snippets in the specified range in the next `In []` prompt. Press *Enter* to re-execute these snippets:

lick here to view code image

```
In [17]: %recall 6-13

In [18]: values, frequencies = np.unique(rolls,   return_counts=True)
    ...: title = f'Rolling a Six-Sided   Die {len(rolls):,} Times'
    ...: sns.set_style('whitegrid')
    ...: axes = sns.barplot(x=values, y=frequencies,   palette='bright')
    ...: axes.set_title(title)
    ...: axes.set(xlabel='Die Value',   ylabel='Frequency')
    ...: axes.set_ylim(top=max(frequencies) * 1.10)
    ...: for bar, frequency in zip(axes.patches, frequencies):
    ...:     text_x = bar.get_x() + bar.get_width() / 2.0
    ...:     text_y = bar.get_height()
    ...:     text = f'{frequency:,}\n{frequency / len(rolls):.3%}'
    ...:     axes.text(text_x, text_y, text,
    ...:               fontsize=11,   ha='center', va='bottom')
    ...:
```

The updated bar plot is shown below:



C

Saving Snippets to a File with the `%save` Magic

Once you've interactively created a plot, you may want to save the code to a file so you can turn it into a script and run it in the future. Let's use the **`%save` magic** to save snippets 1 through 13 to a file named `RollDie.py`. IPython indicates the file to which the lines were written, then displays the lines that it saved:

lick here to view code image

```
In [19]: %save RollDie.py 1-13
The following commands were written to file `RollDie.py`:
import matplotlib.pyplot as plt
import numpy as np
import random
import seaborn as sns
rolls = [random.randrange(1, 7) for i in range(600)]
values, frequencies = np.unique(rolls, return_counts=True)
title = f'Rolling a Six-Sided Die {len(rolls):,} Times'
sns.set_style("whitegrid")
axes = sns.barplot(values, frequencies, palette='bright')
axes.set_title(title)
axes.set(xlabel='Die Value', ylabel='Frequency')
axes.set_ylim(top=max(frequencies) * 1.10)
for bar, frequency in zip(axes.patches, frequencies):
    text_x = bar.get_x() + bar.get_width() / 2.0
    text_y = bar.get_height()
    text = f'{frequency:,}\n{frequency / len(rolls):.3%}'
    axes.text(text_x, text_y, text,
              fontsize=11, ha='center', va='bottom')
```

Command-Line Arguments; Displaying a Plot from a Script

Provided with this chapter's examples is an edited version of the `RollDie.py` file you saved above. We added comments and a two modifications so you can run the script with an argument that specifies the number of die rolls, as in:

```
ipython RollDie.py 600
```

The Python Standard Library's **`sys` module** enables a script to receive *command-line arguments* that are passed into the program. These include the script's name and any values that appear to the right of it when you execute the script. The `sys` module's **`argv`** list contains the arguments. In the command above, `argv[0]` is the *string* `'RollDie.py'` and `argv[1]` is the *string* `'600'`. To control the number of die rolls with the command-line argument's value, we modified the statement that creates the `rolls` list as follows:

C

lick here to view code image

```
        rolls = [random.randrange(1, 7) for i in range(int(sys.argv[1]))]
```

Note that we converted the `argv[1]` string to an `int`.

Matplotlib and Seaborn do not automatically display the plot for you when you create it in a script. So at the end of the script we added the following call to Matplotlib's **`show`** function, which displays the window containing the graph:

```
        plt.show()
```

5.18 WRAP-UP

This chapter presented more details of the list and tuple sequences. You created lists, accessed their elements and determined their length. You saw that lists are mutable, so you can modify their contents, including growing and shrinking the lists as your programs execute. You saw that accessing a nonexistent element causes an `IndexError`. You used `for` statements to iterate through list elements.

We discussed tuples, which like lists are sequences, but are immutable. You unpacked a tuple's elements into separate variables. You used `enumerate` to create an iterable of tuples, each with a list index and corresponding element value.

You learned that all sequences support slicing, which creates new sequences with subsets of the original elements. You used the `del` statement to remove elements from lists and delete variables from interactive sessions. We passed lists, list elements and slices of lists to functions. You saw how to search and sort lists, and how to search tuples. We used list methods to insert, append and remove elements, and to reverse a list's elements and copy lists.

We showed how to simulate stacks with lists. We used the concise list-comprehension notation to create new lists. We used additional built-in methods to sum list elements, iterate backward through a list, find the minimum and maximum values, filter values and map values to new values. We showed how nested lists can represent two-dimensional tables in which data is arranged in rows and columns. You saw how nested `for` loops process two-dimensional lists.

The chapter concluded with an Intro to Data Science section that presented a die-

C

olling simulation and static visualizations. A detailed code example used the Seaborn and Matplotlib visualization libraries to create a *static* bar plot visualization of the simulation's final results. In the next Intro to Data Science section, we use a die-rolling simulation with a *dynamic* bar plot visualization to make the plot "come alive."

In the next chapter, "Dictionaries and Sets," we'll continue our discussion of Python's built-in collections. We'll use dictionaries to store unordered collections of key–value pairs that map immutable keys to values, just as a conventional dictionary maps words to definitions. We'll use sets to store unordered collections of unique elements.

In the "Array-Oriented Programming with NumPy" chapter, we'll discuss NumPy's `ndarray` collection in more detail. You'll see that while lists are fine for small amounts of data, they are not efficient for the large amounts of data you'll encounter in big data analytics applications. For such cases, the NumPy library's highly optimized `ndarray` collection should be used. `ndarray` (*n*-dimensional array) can be much faster than lists. We'll run Python profiling tests to see just how much faster. As you'll see, NumPy also includes many capabilities for conveniently and efficiently manipulating arrays of *many* dimensions. In big data analytics applications, the processing demands can be humongous, so everything we can do to improve performance significantly matters. In our " ig Data: Hadoop, Spark, NoSQL and IoT" chapter, you'll use one of the most popular high-performance big-data databases—MongoDB. [2]

[2] The databases name is rooted in the word humongous.

6. Dictionaries and Sets

Objectives

In this chapter, you'll:

- Use dictionaries to represent unordered collections of key–value pairs.
- Use sets to represent unordered collections of unique values.
- Create, initialize and refer to elements of dictionaries and sets.
- Iterate through a dictionary's keys, values and key–value pairs.
- Add, remove and update a dictionary's key–value pairs.
- Use dictionary and set comparison operators.
- Combine sets with set operators and methods.
- Use operators `in` and `not in` to determine if a dictionary contains a key or a set contains a value.
- Use the mutable set operations to modify a set's contents.
- Use comprehensions to create dictionaries and sets quickly and conveniently.
- Learn how to build dynamic visualizations.
- Enhance your understanding of mutability and immutability.

Outline

6.1 INTRODUCTION

We've discussed three built-in sequence collections—strings, lists and tuples. Now, we consider the built-in non-sequence collections—dictionaries and sets. A **dictionary** is an *unordered* collection which stores **key–value pairs** that map immutable keys to values, just as a conventional dictionary maps words to definitions. A **set** is an

unordered collection of *unique* immutable elements.

6.2 DICTIONARIES

A dictionary *associates* keys with values. Each key *maps* to a specific value. The following table contains examples of dictionaries with their keys, key types, values and value types:

Keys	Key type	Values	Value type
Country names	`str`	Internet country codes	`str`
Decimal numbers	`int`	Roman numerals	`str`
States	`str`	Agricultural products	list of `str`
Hospital patients	`str`	Vital signs	tuple of `int`s and `float`s
Baseball players	`str`	Batting averages	`float`
Metric measurements	`str`	Abbreviations	`str`
Inventory codes	`str`	Quantity in stock	`int`

nique Keys

A dictionary's keys must be *immutable* (such as strings, numbers or tuples) and *unique* (that is, no duplicates). Multiple keys can have the same value, such as two different inventory codes that have the same quantity in stock.

6.2.1 Creating a Dictionary

You can create a dictionary by enclosing in curly braces, `{}`, a comma-separated list of key–value pairs, each of the form *key*`:` *value*. You can create an empty dictionary with `{}`.

Let's create a dictionary with the country-name keys `'Finland'`, `'South Africa'` and `'Nepal'` and their corresponding Internet country code values `'fi'`, `'za'` and `'np'`:

lick here to view code image

```
In [1]: country_codes = {'Finland':   'fi', 'South Africa':   'za',
   ...:                  'Nepal':   'np'}
   ...:

In [2]: country_codes
Out[2]: {'Finland': 'fi', 'South Africa': 'za', 'Nepal':   'np'}
```

When you output a dictionary, its comma-separated list of key–value pairs is always enclosed in curly braces. Because dictionaries are *unordered* collections, the display order can differ from the order in which the key–value pairs were added to the dictionary. In snippet `[2]`'s output the key–value pairs are displayed in the order they were inserted, but do *not* write code that depends on the order of the key–value pairs.

Determining if a Dictionary Is Empty

The built-in function `len` returns the number of key–value pairs in a dictionary:

```
In [3]: len(country_codes)
Out[3]: 3
```

You can use a dictionary as a condition to determine if it's empty—a non-empty dictionary evaluates to `True`:

lick here to view code image

```
In [4]: if country_codes:
   ...:     print('country_codes is   not empty')
   ...: else:
   ...:     print('country_codes is   empty')
   ...:
country_codes is not empty
```

An empty dictionary evaluates to `False`. To demonstrate this, in the following code we call method **`clear`** to delete the dictionary's key–value pairs, then in snippet `[6]` we recall and re-execute snippet `[4]`:

lick here to view code image

```
In [5]: country_codes.clear()

In [6]: if country_codes:
   ...:     print('country_codes is   not empty')
   ...: else:
   ...:     print('country_codes is   empty')
   ...:
country_codes is empty
```

6.2.2 Iterating through a Dictionary

The following dictionary maps month-name strings to `int` values representing the numbers of days in the corresponding month. Note that *multiple* keys can have the *same* value:

lick here to view code image

```
In [1]: days_per_month = {'January':   31, 'February': 28, 'March': 31}

In [2]: days_per_month
Out[2]: {'January': 31, 'February': 28, 'March': 31}
```

Again, the dictionary's string representation shows the key–value pairs in their insertion order, but this is not guaranteed because dictionaries are *unordered*. We'll show how to process keys in *sorted* order later in this chapter.

The following `for` statement iterates through `days_per_month`'s key–value pairs. Dictionary method **`items`** returns each key–value pair as a tuple, which we unpack into `month` and `days`:

C

lick here to view code image

```
In [3]: for month, days in days_per_month.items():
   ...:     print(f'{month} has {days} days')
   ...:
January has 31 days
February has 28 days
March has 31 days
```

6.2.3 Basic Dictionary Operations

For this section, let's begin by creating and displaying the dictionary `roman_numerals`. We intentionally provide the incorrect value `100` for the key `'X'`, which we'll correct shortly:

lick here to view code image

```
In [1]: roman_numerals = {'I':   1, 'II': 2, 'III': 3, 'V': 5, 'X': 100}

In [2]: roman_numerals
Out[2]: {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 100}
```

Accessing the Value Associated with a Key

Let's get the value associated with the key `'V'`:

```
In [3]: roman_numerals['V']
Out[3]: 5
```

Updating the Value of an Existing Key–Value Pair

You can update a key's associated value in an assignment statement, which we do here to replace the incorrect value associated with the key `'X'`:

lick here to view code image

```
In [4]: roman_numerals['X'] = 10

In [5]: roman_numerals
Out[5]: {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 10}
```

Adding a New Key–Value Pair

C

Assigning a value to a nonexistent key inserts the key–value pair in the dictionary:

lick here to view code image

```
In [6]: roman_numerals['L'] = 50

In [7]: roman_numerals
Out[7]: {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 10, 'L':   50}
```

String keys are case sensitive. Assigning to a nonexistent key inserts a new key–value pair. This may be what you intend, or it could be a logic error.

Removing a Key–Value Pair

You can delete a key–value pair from a dictionary with the `del` statement:

lick here to view code image

```
In [8]: del roman_numerals['III']

In [9]: roman_numerals
Out[9]: {'I': 1, 'II': 2, 'V': 5, 'X': 10, 'L': 50}
```

You also can remove a key–value pair with the dictionary method **pop**, which returns the value for the removed key:

lick here to view code image

```
In [10]: roman_numerals.pop('X')
Out[10]: 10

In [11]: roman_numerals
Out[11]: {'I': 1, 'II': 2, 'V': 5, 'L': 50}
```

Attempting to Access a Nonexistent Key

Accessing a nonexistent key results in a `KeyError`:

lick here to view code image

```
 n [12]: roman_numerals['III']
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last
```

C

```
<ipython-input-12-ccd50c7f0c8b> in <module>()
----> 1 roman_numerals['III']

KeyError: 'III'
```

ou can prevent this error by using dictionary method **get**, which normally returns its argument's corresponding value. If that key is not found, `get` returns `None`. IPython does not display anything when `None` is returned in snippet `[13]`. If you specify a second argument to `get`, it returns that value if the key is not found:

lick here to view code image

```
In [13]: roman_numerals.get('III')

In [14]: roman_numerals.get('III',   'III not in dictionary')
Out[14]: 'III not in dictionary'

In [15]: roman_numerals.get('V')
Out[15]: 5
```

Testing Whether a Dictionary Contains a Specified Key

Operators `in` and `not in` can determine whether a dictionary contains a specified key:

lick here to view code image

```
In [16]: 'V' in roman_numerals
Out[16]: True

In [17]: 'III' in roman_numerals
Out[17]: False

In [18]: 'III' not in roman_numerals
Out[18]: True
```

6.2.4 Dictionary Methods `keys` and `values`

Earlier, we used dictionary method `items` to iterate through tuples of a dictionary's key–value pairs. Similarly, methods **keys** and **values** can be used to iterate through only a dictionary's keys or values, respectively:

lick here to view code image

```
In [1]: months = {'January': 1, 'February': 2, 'March': 3}

In [2]: for month_name in months.keys():
   ...:     print(month_name, end='    ')
   ...:
January  February  March

In [3]: for month_number in months.values():
   ...:     print(month_number, end='    ')
   ...:
1  2  3
```

Dictionary Views

Dictionary methods `items`, `keys` and `values` each return a view of a dictionary's data. When you iterate over a **view**, it "sees" the dictionary's current contents—it does *not* have its own copy of the data.

To show that views do *not* maintain their own copies of a dictionary's data, let's first save the view returned by `keys` into the variable `months_view`, then iterate through it:

lick here to view code image

```
In [4]: months_view = months.keys()

In [5]: for key in months_view:
   ...:     print(key, end='  ')
   ...:
January  February  March
```

Next, let's add a new key–value pair to `months` and display the updated dictionary:

lick here to view code image

```
In [6]: months['December'] = 12

In [7]: months
Out[7]: {'January': 1, 'February': 2, 'March': 3,   'December': 12}
```

Now, let's iterate through `months_view` again. The key we added above is indeed displayed:

lick here to view code image

C

```
In [8]: for key in months_view:
   ...:     print(key, end='  ')
   ...:
January  February  March  December
```

Do not modify a dictionary while iterating through a view. According to Section 4.10.1 of the Python Standard Library documentation, [1] either you'll get a `RuntimeError` or the loop might not process all of the view's values.

[1] ttps://docs.python.org/3/library/stdtypes.html#dictionary-iew-objects.

Converting Dictionary Keys, Values and Key–Value Pairs to Lists

You might occasionally need *lists* of a dictionary's keys, values or key–value pairs. To obtain such a list, pass the view returned by `keys`, `values` or `items` to the built-in `list` function. Modifying these lists does *not* modify the corresponding dictionary:

lick here to view code image

```
In [9]: list(months.keys())
Out[9]: ['January', 'February', 'March', 'December']

In [10]: list(months.values())
Out[10]: [1, 2, 3, 12]

In [11]: list(months.items())
Out[11]: [('January',   1), ('February', 2), ('March', 3), ('December', 12
```

rocessing Keys in Sorted Order

To process keys in *sorted* order, you can use built-in function `sorted` as follows:

lick here to view code image

```
In [12]: for month_name in sorted(months.keys()):
    ...:     print(month_name, end='    ')
    ...:
February  December  January  March
```

6.2.5 Dictionary Comparisons

The comparison operators == and != can be used to determine whether two dictionaries have identical or different contents. An equals (==) comparison evaluates to `True` if both dictionaries have the same key–value pairs, *regardless* of the order in which those key–value pairs were added to each dictionary:

lick here to view code image

```
In [1]: country_capitals1 = {'Belgium':   'Brussels',
   ...:                      'Haiti':   'Port-au-Prince'}
   ...:

In [2]: country_capitals2 = {'Nepal':   'Kathmandu',
   ...:                      'Uruguay':   'Montevideo'}
   ...:

In [3]: country_capitals3 = {'Haiti':   'Port-au-Prince',
   ...:                      'Belgium':   'Brussels'}
   ...:

In [4]: country_capitals1 == country_capitals2
Out[4]: False

In [5]: country_capitals1 == country_capitals3
Out[5]: True

In [6]: country_capitals1 != country_capitals2
Out[6]: True
```

6.2.6 Example: Dictionary of Student Grades

The following script represents an instructor's grade book as a dictionary that maps each student's name (a string) to a list of integers containing that student's grades on three exams. In each iteration of the loop that displays the data (lines 13–17), we unpack a key–value pair into the variables `name` and `grades` containing one student's name and the corresponding list of three grades. Line 14 uses built-in function `sum` to total a given student's grades, then line 15 calculates and displays that student's average by dividing `total` by the number of grades for that student (`len(grades)`). Lines 16–17 keep track of the total of all four students' grades and the number of grades for all the students, respectively. Line 19 prints the class average of all the students' grades on all the exams.

lick here to view code image

```
1 # fig06_01.py
2 """Using a dictionary to represent an   instructor's grade book."""
```

C

```
3 grade_book =   {
4     'Susan': [92, 85, 100],
5     'Eduardo': [83, 95, 79],
6     'Azizi': [91, 89, 82],
7     'Pantipa': [97, 91, 92]
8 }
9
10 all_grades_total   = 0
11 all_grades_count   = 0
12
13 for name, grades in   grade_book.items():
14     total =   sum(grades)
15     print(f'Average for {name} is {total/len(grades):.2f}')
16     all_grades_total   += total
17     all_grades_count   += len(grades)
18
19 print(f"Class's average is: {all_grades_total /   all_grades_count:.2f
```

lick here to view code image

```
Average for Susan is 92.33
Average for Eduardo is 85.67
Average for Azizi is 87.33
Average for Pantipa is 93.33
Class's average is: 89.67
```

6.2.7 Example: Word Counts [2]

[2] Techniques like word frequency counting are often used to analyze published works. For example, some people believe that the works of William Shakespeare actually might have been written by Sir Francis Bacon, Christopher Marlowe or others. Comparing the word frequencies of their works with those of Shakespeare can reveal writing-style similarities. Well look at other document-analysis techniques in the Natural Language Processing (NLP) chapter.

The following script builds a dictionary to count the number of occurrences of each word in a string. Lines 4–5 create a string `text` that we'll break into words—a process known as **tokenizing a string**. Python automatically concatenates strings separated by whitespace in parentheses. Line 7 creates an empty dictionary. The dictionary's keys will be the unique words, and its values will be integer counts of how many times each word appears in `text`.

C

lick here to view code image

```
1 # fig06_02.py
2 """Tokenizing a string and counting unique   words."""
3
4 text = ('this is sample text with several words '
5         'this is more sample text with some different words')
6
7 word_counts =   {}
8
9 # count occurrences of each unique word
10 for word in text.split():
11     if word in word_counts:
12         word_counts[word]   += 1  # update existing   key-value pair
13     else:
14         word_counts[word]   = 1  # insert new   key-value pair
15
16 print(f'{"WORD":<12}COUNT')
17
18 for word, count in   sorted(word_counts.items()):
19     print(f'{word:<12}{count}')
20
21 print('\nNumber of unique words:', len(word_counts))
```

lick here to view code image

```
WORD        COUNT
different   1
is          2
more        1
sample      2
several     1
some        1
text        2
this        2
with        2
words       2
Number of unique words: 10
```

Line 10 tokenizes `text` by calling string method **`split`**, which separates the words using the method's delimiter string argument. If you do not provide an argument, `split` uses a space. The method returns a list of tokens (that is, the words in `text`). Lines 10–14 iterate through the list of words. For each `word`, line 11 determines whether that `word` (the key) is already in the dictionary. If so, line 12 increments that word's count; otherwise, line 14 inserts a new key–value pair for that `word` with an

C

initial count of 1.

Lines 16–21 summarize the results in a two-column table containing each `word` and its corresponding `count`. The `for` statement in lines 18 and 19 iterates through the diction-ary's key–value pairs. It unpacks each key and value into the variables `word` and `count`, then displays them in two columns. Line 21 displays the number of unique words.

Python Standard Library Module `collections`

The Python Standard Library already contains the counting functionality that we implemented using the dictionary and the loop in lines 10–14. The module **`collections`** contains the type **`Counter`**, which receives an iterable and summarizes its elements. Let's reimplement the preceding script in fewer lines of code with `Counter`:

lick here to view code image

```
In [1]: from collections import Counter

In [2]: text = ('this is sample text   with several words '
   ...:         'this is more sample   text with some different words')
   ...:

In [3]: counter = Counter(text.split())

In [4]: for word, count in sorted(counter.items()):
   ...:     print(f'{word:<12}{count}')
   ...:
different   1
is          2
more        1
sample      2
several     1
some        1
text        2
this        2
with        2
words       2

In [5]: print('Number of unique   keys:', len(counter.keys()))
Number of unique keys: 10
```

Snippet `[3]` creates the `Counter`, which summarizes the list of strings returned by `text.split()`. In snippet `[4]`, `Counter` method **`items`** returns each string and its associated count as a tuple. We use built-in function `sorted` to get a list of these tuples

C

in ascending order. By default sorted orders the tuples by their first elements. If those are identical, then it looks at the second element, and so on. The `for` statement iterates over the resulting sorted list, displaying each `word` and `count` in two columns.

6.2.8 Dictionary Method `update`

You may insert and update key–value pairs using dictionary method **`update`**. First, let's create an empty `country_codes` dictionary:

```
In [1]: country_codes = {}
```

The following `update` call receives a dictionary of key–value pairs to insert or update:

lick here to view code image

```
In [2]: country_codes.update({'South   Africa': 'za'})

In [3]: country_codes
Out[3]: {'South Africa': 'za'}
```

Method `update` can convert keyword arguments into key–value pairs to insert. The following call automatically converts the parameter name `Australia` into the string key `'Australia'` and associates the value `'ar'` with that key:

lick here to view code image

```
In [4]: country_codes.update(Australia='ar')

In [5]: country_codes
Out[5]: {'South Africa': 'za', 'Australia': 'ar'}
```

Snippet `[4]` provided an incorrect country code for `Australia`. Let's correct this by using another keyword argument to update the value associated with `'Australia'`:

lick here to view code image

```
In [6]: country_codes.update(Australia='au')

In [7]: country_codes
Out[7]: {'South Africa': 'za', 'Australia': 'au'}
```

C

Method `update` also can receive an iterable object containing key–value pairs, such as a list of two-element tuples.

6.2.9 Dictionary Comprehensions

Dictionary comprehensions provide a convenient notation for quickly generating dictionaries, often by mapping one dictionary to another. For example, in a dictionary with *unique* values, you can reverse the key–value pairs:

lick here to view code image

```
In [1]: months = {'January': 1, 'February': 2, 'March': 3}

In [2]: months2 = {number: name for   name, number in months.items()}

In [3]: months2
Out[3]: {1: 'January', 2: 'February', 3: 'March'}
```

Curly braces delimit a *dictionary comprehension*, and the expression to the left of the `for` clause specifies a key–value pair of the form *key*`:` *value*. The comprehension iterates through `months.items()`, unpacking each key–value pair tuple into the variables `name` and `number`. The expression `number: name` reverses the key and value, so the new dictionary maps the month numbers to the month names.

What if `months` contained *duplicate* values? As these become the keys in `months2`, attempting to insert a *duplicate* key simply updates the existing key's value. So if `'February'` and `'March'` both mapped to `2` originally, the preceding code would have produced

```
{1: 'January', 2: 'March'}
```

A dictionary comprehension also can map a dictionary's values to new values. The following comprehension converts a dictionary of names and lists of grades into a dictionary of names and grade-point averages. The variables `k` and `v` commonly mean *key* and *value*:

lick here to view code image

```
In [4]: grades = {'Sue': [98, 87, 94], 'Bob': [84, 95, 91]}

In [5]: grades2 = {k: sum(v) / len(v) for   k, v in grades.items()}
```

C

```
In [6]: grades2
Out[6]: {'Sue': 93.0, 'Bob': 90.0}
```

The comprehension unpacks each tuple returned by `grades.items()` into `k` (the name) and `v` (the list of grades). Then, the comprehension creates a new key–value pair with the key `k` and the value of `sum(v) / len(v)`, which averages the list's elements.

6.3 SETS

A set is an unordered collection of *unique* values. Sets may contain only immutable objects, like strings, `int`s, `float`s and tuples that contain only immutable elements. Though sets are iterable, they are not sequences and do not support indexing and slicing with square brackets, `[]`. Dictionaries also do not support slicing.

Creating a Set with Curly Braces

The following code creates a set of strings named `colors`:

lick here to view code image

```
In [1]: colors = {'red', 'orange', 'yellow', 'green', 'red', 'blue'}

In [2]: colors
Out[2]: {'blue', 'green', 'orange', 'red', 'yellow'}
```

Notice that the duplicate string `'red'` was ignored (without causing an error). An important use of sets is **duplicate elimination**, which is automatic when creating a set. Also, the resulting set's values are *not* displayed in the same order as they were listed in snippet `[1]`. Though the color names are displayed in sorted order, sets are *unordered*. You should not write code that depends on the order of their elements.

Determining a Set's Length

You can determine the number of items in a set with the built-in `len` function:

```
In [3]: len(colors)
Out[3]: 5
```

Checking Whether a Value Is in a Set

You can check whether a set contains a particular value using the `in` and `not in`

C

operators:

lick here to view code image

```
In [4]: 'red' in colors
Out[4]: True

In [5]: 'purple' in colors
Out[5]: False

In [6]: 'purple' not in colors
Out[6]: True
```

Iterating Through a Set

Sets are iterable, so you can process each set element with a `for` loop:

lick here to view code image

```
In [7]: for color in colors:
   ...:     print(color.upper(), end='   ')
   ...:
RED GREEN YELLOW BLUE ORANGE
```

Sets are *unordered*, so there's no significance to the iteration order.

Creating a Set with the Built-In `set` Function

You can create a set from another collection of values by using the built-in **`set`** function—here we create a list that contains several duplicate integer values and use that list as `set`'s argument:

lick here to view code image

```
In [8]: numbers = list(range(10))   + list(range(5))

In [9]: numbers
Out[9]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4]

In [10]: set(numbers)
Out[10]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
```

If you need to create an empty set, you must use the `set` function with empty parentheses, rather than empty braces, `{}`, which represent an empty dictionary:

C

```
In [11]: set()
Out[11]: set()
```

Python displays an empty set as `set()` to avoid confusion with Python's string representation of an empty dictionary (`{}`).

Frozenset: An Immutable Set Type

Sets are *mutable*—you can add and remove elements, but set *elements* must be *immutable*. Therefore, a set cannot have other sets as elements. A **frozenset** is an *immutable* set—it cannot be modified after you create it, so a set *can* contain frozensets as elements. The built-in function **`frozenset`** creates a frozenset from any iterable.

6.3.1 Comparing Sets

Various operators and methods can be used to compare sets. The following sets contain the same values, so == returns `True` and `!=` returns `False`.

lick here to view code image

```
In [1]: {1, 3, 5} == {3, 5, 1}
Out[1]: True

In [2]: {1, 3, 5} != {3, 5, 1}
Out[2]: False
```

The < operator tests whether the set to its left is a **proper subset** of the one to its right—that is, all the elements in the left operand are in the right operand, and the sets are not equal:

lick here to view code image

```
In [3]: {1, 3, 5} < {3, 5, 1}
Out[3]: False

In [4]: {1, 3, 5} < {7, 3, 5, 1}
Out[4]: True
```

The <= operator tests whether the set to its left is an **improper subset** of the one to its right—that is, all the elements in the left operand are in the right operand, and the sets might be equal:

C

lick here to view code image

```
In [5]: {1, 3, 5} <= {3, 5, 1}
Out[5]: True

In [6]: {1, 3} <= {3, 5, 1}
Out[6]: True
```

You may also check for an improper subset with the set method **`issubset`**:

lick here to view code image

```
In [7]: {1, 3, 5}.issubset({3, 5, 1})
Out[7]: True

In [8]: {1, 2}.issubset({3, 5, 1})
Out[8]: False
```

The > operator tests whether the set to its left is a **proper superset** of the one to its right—that is, all the elements in the right operand are in the left operand, and the left operand has more elements:

lick here to view code image

```
In [9]: {1, 3, 5} > {3, 5, 1}
Out[9]: False

In [10]: {1, 3, 5, 7} > {3, 5, 1}
Out[10]: True
```

The >= operator tests whether the set to its left is an **improper superset** of the one to its right—that is, all the elements in the right operand are in the left operand, and the sets might be equal:

lick here to view code image

```
In [11]: {1, 3, 5} >= {3, 5, 1}
Out[11]: True

In [12]: {1, 3, 5} >= {3, 1}
Out[12]: True

In [13]: {1, 3} >= {3, 1, 7}
Out[13]: False
```

C

You may also check for an improper superset with the set method **`issuperset`**:

lick here to view code image

```
In [14]: {1, 3, 5}.issuperset({3, 5, 1})
Out[14]: True

In [15]: {1, 3, 5}.issuperset({3, 2})
Out[15]: False
```

The argument to `issubset` or `issuperset` can be *any* iterable. When either of these methods receives a non-set iterable argument, it first converts the iterable to a set, then performs the operation.

6.3.2 Mathematical Set Operations

This section presents the set type's mathematical operators `|`, `&`, `-` and `^` and the corresponding methods.

Union

The **union** of two sets is a set consisting of all the unique elements from both sets. You can calculate the union with the **`|` operator** or with the set type's **`union`** method:

lick here to view code image

```
In [1]: {1, 3, 5} | {2, 3, 4}
Out[1]: {1, 2, 3, 4, 5}

In [2]: {1, 3, 5}.union([20, 20, 3, 40, 40])
Out[2]: {1, 3, 5, 20, 40}
```

The operands of the binary set operators, like `|`, must both be sets. The corresponding set methods may receive any iterable object as an argument—we passed a list. When a mathematical set method receives a non-set iterable argument, it first converts the iterable to a set, then applies the mathematical operation. Again, though the new sets' string representations show the values in ascending order, you should not write code that depends on this.

Intersection

The **intersection** of two sets is a set consisting of all the unique elements that the two

C

sets have in common. You can calculate the intersection with the **`&` operator** or with the set type's **`intersection`** method:

lick here to view code image

```
In [3]: {1, 3, 5} & {2, 3, 4}
Out[3]: {3}

In [4]: {1, 3, 5}.intersection([1, 2, 2, 3, 3, 4, 4])
Out[4]: {1, 3}
```

Difference

The **difference** between two sets is a set consisting of the elements in the left operand that are not in the right operand. You can calculate the difference with the **`-` operator** or with the set type's **`difference`** method:

lick here to view code image

```
In [5]: {1, 3, 5} - {2, 3, 4}
Out[5]: {1, 5}

In [6]: {1, 3, 5, 7}.difference([2, 2, 3, 3, 4, 4])
Out[6]: {1, 5, 7}
```

Symmetric Difference

The **symmetric difference** between two sets is a set consisting of the elements of both sets that are not in common with one another. You can calculate the symmetric difference with the **`^` operator** or with the set type's **`symmetric_difference`** method:

lick here to view code image

```
In [7]: {1, 3, 5} ^ {2, 3, 4}
Out[7]: {1, 2, 4, 5}

In [8]: {1, 3, 5, 7}.symmetric_difference([2,   2, 3, 3, 4, 4])
Out[8]: {1, 2, 4, 5, 7}
```

Disjoint

Two sets are **disjoint** if they do not have any common elements. You can determine

C

this with the set type's **`isdisjoint`** method:

lick here to view code image

```
In [9]: {1, 3, 5}.isdisjoint({2, 4, 6})
Out[9]: True

In [10]: {1, 3, 5}.isdisjoint({4, 6, 1})
Out[10]: False
```

6.3.3 Mutable Set Operators and Methods

The operators and methods presented in the preceding section each result in a *new* set. Here we discuss operators and methods that modify an *existing* set.

Mutable Mathematical Set Operations

Like operator `|`, **union augmented assignment `|=`** performs a set union operation, but `|=` modifies its left operand:

lick here to view code image

```
In [1]: numbers = {1, 3, 5}

In [2]: numbers |= {2, 3, 4}

In [3]: numbers
Out[3]: {1, 2, 3, 4, 5}
```

Similarly, the set type's **`update`** method performs a union operation modifying the set on which it's called—the argument can be any iterable:

lick here to view code image

```
In [4]: numbers.update(range(10))

In [5]: numbers
Out[5]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
```

The other mutable set methods are:

- intersection augmented assignment `&=`

C

- difference augmented assignment `-=`
- symmetric difference augmented assignment `^=`

and their corresponding methods with iterable arguments are:

- `intersection_update`
- `difference_update`
- `symmetric_difference_update`

Methods for Adding and Removing Elements

Set method **`add`** inserts its argument if the argument is *not* already in the set; otherwise, the set remains unchanged:

lick here to view code image

```
In [6]: numbers.add(17)

In [7]: numbers.add(3)

In [8]: numbers
Out[8]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 17}
```

Set method **`remove`** removes its argument from the set—a `KeyError` occurs if the value is not in the set:

lick here to view code image

```
In [9]: numbers.remove(3)

In [10]: numbers
Out[10]: {0, 1, 2, 4, 5, 6, 7, 8, 9, 17}
```

Method **`discard`** also removes its argument from the set but does not cause an exception if the value is not in the set.

You also can remove an *arbitrary* set element and return it with **`pop`**, but sets are unordered, so you do not know which element will be returned:

C

lick here to view code image

```
In [11]: numbers.pop()
Out[11]: 0

In [12]: numbers
Out[12]: {1, 2, 4, 5, 6, 7, 8, 9, 17}
```

A `KeyError` occurs if the set is empty when you call `pop`.

Finally, method **`clear`** empties the set on which it's called:

```
In [13]: numbers.clear()

In [14]: numbers
Out[14]: set()
```

6.3.4 Set Comprehensions

Like dictionary comprehensions, you define set comprehensions in curly braces. Let's create a new set containing only the unique even values in the list `numbers`:

lick here to view code image

```
In [1]: numbers = [1, 2, 2, 3, 4, 5, 6, 6, 7, 8, 9, 10, 10]

In [2]: evens = {item for item in numbers if item % 2 == 0}

In [3]: evens
Out[3]: {2, 4, 6, 8, 10}
```

6.4 INTRO TO DATA SCIENCE: DYNAMIC VISUALIZATIONS

The preceding chapter's Intro to Data Science section introduced visualization. We simulated rolling a six-sided die and used the Seaborn and Matplotlib visualization libraries to create a publication-quality *static* bar plot showing the frequencies and percentages of each roll value. In this section, we make things "come alive" with *dynamic visualizations*.

The Law of Large Numbers

C

hen we introduced random-number generation, we mentioned that if the `random` module's `randrange` function indeed produces integers at random, then every number in the specified range has an equal probability (or likelihood) of being chosen each time the function is called. For a six-sided die, each value 1 through 6 should occur one-sixth of the time, so the probability of any one of these values occurring is $1/6^{th}$ or about 16.667%.

In the next section, we create and execute a *dynamic* (that is, *animated*) die-rolling simulation script. In general, you'll see that the more rolls we attempt, the closer each die value's percentage of the total rolls gets to 16.667% and the heights of the bars gradually become about the same. This is a manifestation of the *law of large numbers*.

6.4.1 How Dynamic Visualization Works

The plots produced with Seaborn and Matplotlib in the previous chapter's Intro to Data Science section help you analyze the results for a fixed number of die rolls *after* the simulation completes. This section's enhances that code with the Matplotlib **`animation`** module's **`FuncAnimation`** function, which updates the bar plot *dynamically*. You'll see the bars, die frequencies and percentages "come alive," updating *continuously* as the rolls occur.

Animation Frames

`FuncAnimation` drives a **frame-by-frame animation**. Each **animation frame** specifies everything that should change during one plot update. Stringing together many of these updates over time creates the animation effect. You decide what each frame displays with a function you define and pass to `FuncAnimation`.

Each animation frame will:

- roll the dice a specified number of times (from 1 to as many as you'd like), updating die frequencies with each roll,
- clear the current plot,
- create a new set of bars representing the updated frequencies, and
- create new frequency and percentage text for each bar.

Generally, displaying more frames-per-second yields smoother animation. For example, video games with fast-moving elements try to display *at least* 30 frames-per-

W

second and often more. Though you'll specify the number of milliseconds between animation frames, the actual number of frames-per-second can be affected by the amount of work you perform in each frame and the speed of your computer's processor. This example displays an animation frame every 33 milliseconds—yielding approximately 30 (1000 / 33) frames-per-second. Try larger and smaller values to see how they affect the animation. Experimentation is important in developing the best visualizations.

Running `RollDieDynamic.py`

In the previous chapter's Intro to Data Science section, we developed the static visualization *interactively* so you could see how the code updates the bar plot as you execute each statement. The actual bar plot with the final frequencies and percentages was drawn only once.

For this dynamic visualization, the screen results update frequently so that you can see the animation. Many things change continuously—the lengths of the bars, the frequencies and percentages above the bars, the spacing and labels on the axes and the total number of die rolls shown in the plot's title. For this reason, we present this visualization as a script, rather than interactively developing it.

The script takes two command-line arguments:

- `number_of_frames`—The number of animation frames to display. This value determines the total number of times that `FuncAnimation` updates the graph. For each animation frame, `FuncAnimation` calls a function that you define (in this example, `update`) to specify how to change the plot.

- `rolls_per_frame`—The number of times to roll the die in each animation frame. We'll use a loop to roll the die this number of times, summarize the results, then update the graph with bars and text representing the new frequencies.

To understand how we use these two values, consider the following command:

```
ipython RollDieDynamic.py 6000 1
```

In this case, `FuncAnimation` calls our `update` function 6000 times, rolling one die per frame for a total of 6000 rolls. This enables you to see the bars, frequencies and percentages update one roll at a time. On our system, this animation took about 3.33 minutes (6000 frames / 30 frames-per-second / 60 seconds-per-minute) to show you

only 6000 die rolls.

Displaying animation frames to the screen is a relatively slow *input–output-bound* operation compared to the die rolls, which occur at the computer's super fast CPU speeds. If we roll only one die per animation frame, we won't be able to run a large number of rolls in a reasonable amount of time. Also, for small numbers of rolls, you're unlikely to see the die percentages converge on their expected 16.667% of the total rolls.

To see the law of large numbers in action, you can increase the execution speed by rolling the die more times per animation frame. Consider the following command:

```
ipython RollDieDynamic.py 10000 600
```

In this case, `FuncAnimation` will call our `update` function 10,000 times, performing 600 rolls-per-frame for a total of 6,000,000 rolls. On our system, this took about 5.55 minutes (10,000 frames / 30 frames-per-second / 60 seconds-per-minute), but displayed approximately 18,000 rolls-per-second (30 frames-per-second * 600 rolls-per-frame), so we could quickly see the frequencies and percentages converge on their expected values of about 1,000,000 rolls per face and 16.667% per face.

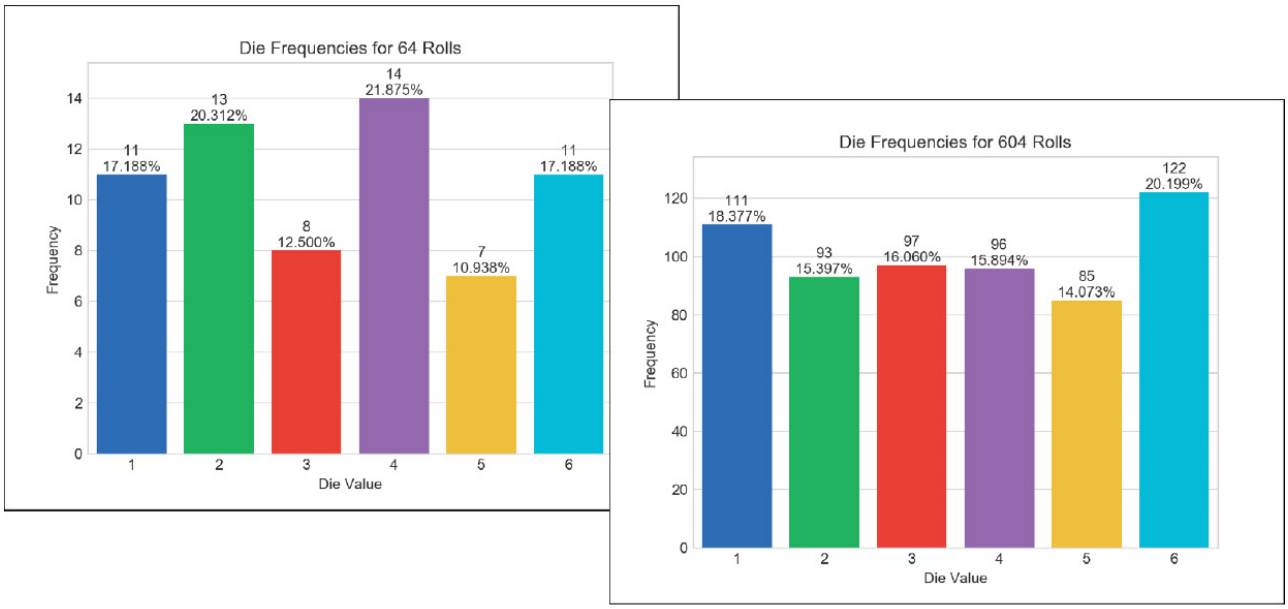
Experiment with the numbers of rolls and frames until you feel that the program is helping you visualize the results most effectively. It's fun and informative to watch it run and to tweak it until you're satisfied with the animation quality.

Sample Executions

We took the following four screen captures during each of two sample executions. In the first, the screens show the graph after just 64 die rolls, then again after 604 of the 6000 total die rolls. Run this script live to see over time how the bars update dynamically. In the second execution, the screen captures show the graph after 7200 die rolls and again after 166,200 out of the 6,000,000 rolls. With more rolls, you can see the percentages closing in on their expected values of 16.667% as predicted by the law of large numbers.

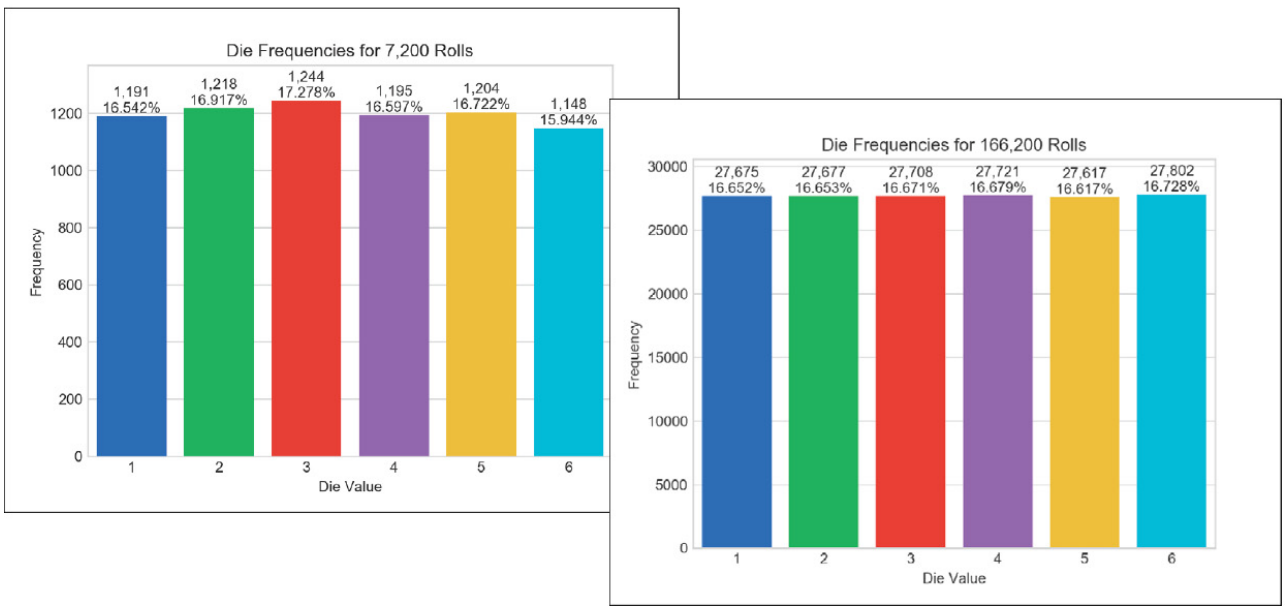
Execute 6000 animation frames rolling the die once per frame:

```
ipython RollDieDynamic.py 6000 1
```



Execute 10,000 animation frames rolling the die 600 times per frame:

```
ipython RollDieDynamic.py 10000 600
```



.4.2 Implementing a Dynamic Visualization

The script we present in this section uses the same Seaborn and Matplotlib features shown in the previous chapter's Intro to Data Science section. We reorganized the code for use with Matplotlib's *animation* capabilities.

Importing the Matplotlib `animation` Module

We focus primarily on the new features used in this example. Line 3 imports the Matplotlib `animation` module.

lick here to view code image

```
1 # RollDieDynamic.py
```

```
2 """Dynamically graphing frequencies of die   rolls."""
3 from matplotlib import   animation
4 import matplotlib.pyplot as   plt
5 import random
6 import seaborn as sns
7 import sys
8
```

Function `update`

Lines 9–27 define the `update` function that `FuncAnimation` calls once per animation frame. This function must provide at least one argument. Lines 9–10 show the beginning of the function definition. The parameters are:

- `frame_number`—The next value from `Func-Animation`'s `frames` argument, which we'll discuss momentarily. Though `FuncAnimation` requires the `update` function to have this parameter, we do not use it in this `update` function.
- `rolls`—The number of die rolls per animation frame.
- `faces`—The die face values used as labels along the graph's *x*-axis.
- `frequencies`—The list in which we summarize the die frequencies.

We discuss the rest of the function's body in the next several subsections.

lick here to view code image

```
9 def update(frame_number, rolls,   faces, frequencies):
10     """Configures bar plot contents for each   animation frame."""
```

Function `update`: Rolling the Die and Updating the `frequencies` List

Lines 12–13 roll the die `rolls` times and increment the appropriate `frequencies` element for each roll. Note that we subtract `1` from the die value (`1` through `6`) before incrementing the corresponding `frequencies` element—as you'll see, `frequencies` is a six-element list (defined in line 36), so its indices are `0` through `5`.

lick here to view code image

```
11     # roll die and update frequencies
12     for i in range(rolls):
```

C

```
13          frequencies[random.randrange(1, 7) - 1] += 1
14
```

Function `update`: Configuring the Bar Plot and Text

Line 16 in function `update` calls the `matplotlib.pyplot` module's **`cla`** (clear axes) function to remove the existing bar plot elements before drawing new ones for the current animation frame. We discussed the code in lines 17–27 in the previous chapter's Intro to Data Science section. Lines 17–20 create the bars, set the bar plot's title, set the *x*- and *y*-axis labels and scale the plot to make room for the frequency and percentage text above each bar. Lines 23–27 display the frequency and percentage text.

lick here to view code image

```
15      # reconfigure plot for updated die frequencies
16      plt.cla()  # clear old contents contents of current Figure
17      axes =   sns.barplot(faces, frequencies, palette='bright')     # ne
18      axes.set_title(f'Die Frequencies for {sum(frequencies):,} Rolls')
19      axes.set(xlabel='Die Value', ylabel='Frequency')
20      axes.set_ylim(top=max(frequencies)   * 1.10)  # scale y-axis   by
21
22      # display frequency & percentage above each patch   (bar)
23      for bar, frequency in   zip(axes.patches, frequencies):
24          text_x   = bar.get_x() + bar.get_width() / 2.0
25          text_y   = bar.get_height()
26          text =   f'{frequency:,}\n{frequency   / sum(frequencies):.3%}
27          axes.text(text_x,   text_y, text, ha='center', va='bottom')
28
```

Variables Used to Configure the Graph and Maintain State

Lines 30 and 31 use the `sys` module's `argv` list to get the script's command-line arguments. Line 33 specifies the Seaborn `'whitegrid'` style. Line 34 calls the `matplotlib.pyplot` module's **`figure`** function to get the `Figure` object in which `FuncAnimation` displays the animation. The function's argument is the window's title. As you'll soon see, this is one of `FuncAnimation`'s required arguments. Line 35 creates a list containing the die face values 1–6 to display on the plot's *x*-axis. Line 36 creates the six-element `frequencies` list with each element initialized to `0`—we update this list's counts with each die roll.

lick here to view code image

```
29 # read command-line arguments for number of frames and   rolls per fra
```

C

```
number_of_frames    = int(sys.argv[1])
rolls_per_frame    = int(sys.argv[2])

sns.set_style('whitegrid')  # white   background with gray grid lines
figure =    plt.figure('Rolling a Six-Sided Die')  # Figure for animati
values =    list(range(1, 7))  # die faces for display on x-axis
frequencies =    [0] * 6  # six-element list of die frequencies

```

alling the `animation` Module's `FuncAnimation` Function

Lines 39–41 call the Matplotlib `animation` module's `FuncAnimation` function to update the bar chart dynamically. The function returns an object representing the animation. Though this is not used explicitly, you *must* store the reference to the animation; otherwise, Python immediately terminates the animation and returns its memory to the system.

lick here to view code image

```
# configure and start animation that calls function update
die_animation    = animation.FuncAnimation(
    figure,    update, repeat=False, frames=number_of_frames,    interva
    fargs=(rolls_per_frame,    values, frequencies))

plt.show()  # display window
```

`FuncAnimation` has two required arguments:

- `figure`—the `Figure` object in which to display the animation, and
- `update`—the function to call once per animation frame.

In this case, we also pass the following optional keyword arguments:

- **`repeat`**—`False` terminates the animation after the specified number of frames. If `True` (the default), when the animation completes it restarts from the beginning.
- **`frames`**—The total number of animation frames, which controls how many times `FunctAnimation` calls `update`. Passing an integer is equivalent to passing a `range`—for example, `600` means `range(600)`. `FuncAnimation` passes one value from this range as the first argument in each call to `update`.

C

- **`interval`**—The number of milliseconds (33, in this case) between animation frames (the default is 200). After each call to `update`, `FuncAnimation` waits 33 milliseconds before making the next call.

- **`fargs`** (short for "function arguments")—A tuple of other arguments to pass to the function you specified in `FuncAnimation`'s second argument. The arguments you specify in the `fargs` tuple correspond to `update`'s parameters `rolls`, `faces` and `frequencies` (line 9).

For a list of `FuncAnimation`'s other optional arguments, see

```
ttps://matplotlib.org/api/_as_gen/matplotlib.animation.FuncAnimation.html
```

Finally, line 43 displays the window.

6.5 WRAP-UP

In this chapter, we discussed Python's dictionary and set collections. We said what a dictionary is and presented several examples. We showed the syntax of key–value pairs and showed how to use them to create dictionaries with comma-separated lists of key–value pairs in curly braces, `{}`. You also created dictionaries with dictionary comprehensions.

You used square brackets, `[]`, to retrieve the value corresponding to a key, and to insert and update key–value pairs. You also used the dictionary method `update` to change a key's associated value. You iterated through a dictionary's keys, values and items.

You created sets of unique immutable values. You compared sets with the comparison operators, combined sets with set operators and methods, changed sets' values with the mutable set operations and created sets with set comprehensions. You saw that sets are mutable. Frozensets are immutable, so they can be used as set and frozenset elements.

In the Intro to Data Science section, we continued our visualization introduction by presenting the die-rolling simulation with a *dynamic* bar plot to make the law of large numbers "come alive." In addition, to the Seaborn and Matplotlib features shown in the previous chapter's Intro to Data Science section, we used Matplotlib's `FuncAnimation` function to control a frame-by-frame animation. `FuncAnimation` called a function we defined that specified what to display in each animation frame.

h

In the next chapter, we discuss array-oriented programming with the popular NumPy library. As you'll see, NumPy's `ndarray` collection can be up to two orders of magnitude faster than performing many of the same operations with Python's built-in lists. This power will come in handy for today's big data applications.

. Array-Oriented Programming with NumPy

Objectives

In this chapter you'll:

- Learn how arrays differ from lists.
- Use the `numpy` module's high-performance `ndarray`s.
- Compare list and `ndarray` performance with the IPython `%timeit` magic.
- Use `ndarray`s to store and retrieve data efficiently.
- Create and initialize `ndarray`s.
- Refer to individual `ndarray` elements.
- Iterate through `ndarray`s.
- Create and manipulate multidimensional `ndarray`s.
- Perform common `ndarray` manipulations.
- Create and manipulate pandas one-dimensional `Series` and two-dimensional `DataFrame`s.
- Customize `Series` and `DataFrame` indices.
- Calculate basic descriptive statistics for data in a `Series` and a `DataFrame`.
- Customize floating-point number precision in pandas output formatting.

Outline

7.1 INTRODUCTION

The **NumPy (Numerical Python)** library first appeared in 2006 and is the preferred Python array implementation. It offers a high-performance, richly functional *n*-dimensional array type called **`ndarray`**, which from this point forward we'll refer to by its synonym, `array`. NumPy is one of the many open-source libraries that the Anaconda Python distribution installs. Operations on `array`s are up to two orders of

magnitude faster than those on lists. In a big-data world in which applications may do massive amounts of processing on vast amounts of array-based data, this performance advantage can be critical. According to `libraries.io`, over 450 Python libraries depend on NumPy. Many popular data science libraries such as Pandas, SciPy (Scientific Python) and Keras (for deep learning) are built on or depend on NumPy.

In this chapter, we explore `array`'s basic capabilities. Lists can have multiple dimensions. You generally process multi-dimensional lists with nested loops or list comprehensions with multiple `for` clauses. A strength of NumPy is "array-oriented programming," which uses functional-style programming with *internal* iteration to make array manipulations concise and straightforward, eliminating the kinds of bugs that can occur with the *external* iteration of explicitly programmed loops.

In this chapter's Intro to Data Science section, we begin our multi-section introduction to the *pandas* library that you'll use in many of the data science case study chapters. Big data applications often need more flexible collections than NumPy's `array`s—collections that support mixed data types, custom indexing, missing data, data that's not structured consistently and data that needs to be manipulated into forms appropriate for the databases and data analysis packages you use. We'll introduce pandas array-like one-dimensional `Series` and two-dimensional `DataFrames` and begin demonstrating their powerful capabilities. After reading this chapter, you'll be familiar with four array-like collections—lists, `array`s, `Series` and `DataFrame`s. We'll add a fifth—tensors—in the "Deep Learning" chapter.

7.2 CREATING `ARRAY`S FROM EXISTING DATA

The NumPy documentation recommends importing the **`numpy` module** as `np` so that you can access its members with `"np."`:

```
In [1]: import numpy as np
```

The `numpy` module provides various functions for creating `array`s. Here we use the **`array`** function, which receives as an argument an `array` or other collection of elements and returns a new `array` containing the argument's elements. Let's pass a list:

lick here to view code image

```
In [2]: numbers = np.array([2,  3, 5, 7, 11])
```

C

The `array` function copies its argument's contents into the `array`. Let's look at the type of object that function `array` returns and display its contents:

lick here to view code image

```
In [3]: type(numbers)
Out[3]: numpy.ndarray

In [4]: numbers
Out[4]: array([ 2,  3,  5,  7, 11])
```

Note that the *type* is `numpy.ndarray`, but all `array`s are output as "`array`." When outputting an `array`, NumPy separates each value from the next with a comma and a space and *right-aligns* all the values using the same field width. It determines the field width based on the value that occupies the *largest* number of character positions. In this case, the value `11` occupies the two character positions, so all the values are formatted in two-character fields. That's why there's a leading space between the `[` and `2`.

Multidimensional Arguments

The `array` function copies its argument's dimensions. Let's create an `array` from a two-row-by-three-column list:

lick here to view code image

```
In [5]: np.array([[1, 2, 3], [4, 5, 6]])
Out[5]:
array([[1, 2, 3],
       [4, 5, 6]])
```

NumPy auto-formats `array`s, based on their number of dimensions, aligning the columns within each row.

7.3 ARRAY ATTRIBUTES

An `array` object provides **attributes** that enable you to discover information about its structure and contents. In this section we'll use the following `array`s:

lick here to view code image

C

```
In [1]: import numpy as np

In [2]: integers = np.array([[1,   2, 3], [4, 5, 6]])

In [3]: integers
Out[3]:
array([[1, 2, 3],
       [4, 5, 6]])

In [4]: floats = np.array([0.0,   0.1, 0.2, 0.3, 0.4])

In [5]: floats
Out[5]: array([ 0. ,  0.1,  0.2,  0.3,  0.4])
```

NumPy does not display trailing 0s to the right of the decimal point in floating-point values.

Determining an `array`'s Element Type

The `array` function determines an `array`'s element type from its argument's elements. You can check the element type with an `array`'s **`dtype`** attribute:

lick here to view code image

```
In [6]: integers.dtype
Out[6]: dtype('int64')  # int32 on   some platforms

In [7]: floats.dtype
Out[7]: dtype('float64')
```

As you'll see in the next section, various `array`-creation functions receive a `dtype` keyword argument so you can specify an `array`'s element type.

For performance reasons, NumPy is written in the C programming language and uses C's data types. By default, NumPy stores integers as the NumPy type `int64` values—which correspond to 64-bit (8-byte) integers in C—and stores floating-point numbers as the NumPy type `float64` values—which correspond to 64-bit (8-byte) floating-point values in C. In our examples, most commonly you'll see the types `int64`, `float64`, `bool` (for Boolean) and `object` for non-numeric data (such as strings). The complete list of supported types is at

ttps://docs.scipy.org/doc/numpy/user/basics.types.html.

Determining an `array`'s Dimensions

The attribute **`ndim`** contains an `array`'s number of dimensions and the attribute **`shape`** contains a *tuple* specifying an `array`'s dimensions:

lick here to view code image

```
In [8]: integers.ndim
Out[8]: 2

In [9]: floats.ndim
Out[9]: 1

In [10]: integers.shape
Out[10]: (2, 3)

In [11]: floats.shape
Out[11]: (5,)
```

Here, `integers` has `2` rows and `3` columns (6 elements) and `floats` is one-dimensional, so snippet `[11]` shows a one-element tuple (indicated by the comma) containing `floats`' number of elements (5).

Determining an `array`'s Number of Elements and Element Size

You can view an `array`'s total number of elements with the attribute **`size`** and the number of bytes required to store each element with **`itemsize`**:

lick here to view code image

```
In [12]: integers.size
Out[12]: 6

In [13]: integers.itemsize  # 4 if   C compiler uses 32-bit ints
Out[13]: 8

In [14]: floats.size
Out[14]: 5

In [15]: floats.itemsize
Out[15]: 8
```

Note that `integers`' `size` is the product of the `shape` tuple's values—two rows of three elements each for a total of six elements. In each case, `itemsize` is 8 because `integers` contains `int64` values and `floats` contains `float64` values, which each occupy 8 bytes.

C

Iterating Through a Multidimensional `array`'s Elements

You'll generally manipulate `array`s using concise functional-style programming techniques. However, because `array`s are *iterable*, you can use external iteration if you'd like:

lick here to view code image

```
In [16]: for row in integers:
    ...:     for column in row:
    ...:         print(column, end='    ')
    ...:     print()
    ...:
1   2   3
4   5   6
```

You can iterate through a multidimensional `array` as if it were one-dimensional by using its **`flat`** attribute:

lick here to view code image

```
In [17]: for i in integers.flat:
    ...:     print(i, end='  ')
    ...:
1  2  3  4  5  6
```

7.4 FILLING `ARRAY`S WITH SPECIFIC VALUES

NumPy provides functions **`zeros`**, **`ones`** and **`full`** for creating `array`s containing `0`s, `1`s or a specified value, respectively. By default, `zeros` and `ones` create `array`s containing `float64` values. We'll show how to customize the element type momentarily. The first argument to these functions must be an integer or a tuple of integers specifying the desired dimensions. For an integer, each function returns a one-dimensional `array` with the specified number of elements:

lick here to view code image

```
In [1]: import numpy as np

In [2]: np.zeros(5)
Out[2]: array([ 0.,  0.,  0.,  0.,  0.])
```

C

For a tuple of integers, these functions return a multidimensional `array` with the specified dimensions. You can specify the `array`'s element type with the `zeros` and `ones` function's `dtype` keyword argument:

lick here to view code image

```
In [3]: np.ones((2, 4), dtype=int)
Out[3]:
array([[1, 1, 1, 1],
       [1, 1, 1, 1]])
```

The `array` returned by `full` contains elements with the second argument's value and type:

lick here to view code image

```
In [4]: np.full((3, 5), 13)
Out[4]:
array([[13, 13, 13, 13, 13],
       [13, 13, 13, 13, 13],
       [13, 13, 13, 13, 13]])
```

7.5 CREATING `ARRAY`S FROM RANGES

NumPy provides optimized functions for creating `array`s from ranges. We focus on simple evenly spaced integer and floating-point ranges, but NumPy also supports nonlinear ranges. [1]

[1] `ttps://docs.scipy.org/doc/numpy/reference/routines.array-reation.html`.

Creating Integer Ranges with `arange`

Let's use NumPy's **`arange`** function to create integer ranges—similar to using built-in function `range`. In each case, `arange` first determines the resulting `array`'s number of elements, allocates the memory, then stores the specified range of values in the `array`:

lick here to view code image

```
In [1]: import numpy as np
```

```
In [2]: np.arange(5)
Out[2]: array([0, 1, 2, 3, 4])

In [3]: np.arange(5, 10)
Out[3]: array([5, 6, 7, 8, 9])

In [4]: np.arange(10, 1, -2)
Out[4]: array([10,  8,  6,  4,  2])
```

Though you can create `arrays` by passing `ranges` as arguments, always use `arange` as it's optimized for `arrays`. Soon we'll show how to determine the execution time of various operations so you can compare their performance.

Creating Floating-Point Ranges with `linspace`

You can produce evenly spaced floating-point ranges with NumPy's **`linspace`** function. The function's first two arguments specify the starting and ending values in the range, and the ending value *is included* in the `array`. The optional keyword argument `num` specifies the number of evenly spaced values to produce—this argument's default value is 50:

lick here to view code image

```
In [5]: np.linspace(0.0, 1.0, num=5)
Out[5]: array([ 0.  ,  0.25,  0.5 ,  0.75,  1.  ])
```

Reshaping an `array`

You also can create an `array` from a range of elements, then use `array` method **`reshape`** to transform the one-dimensional array into a multidimensional array. Let's create an `array` containing the values from 1 through 20, then reshape it into four rows by five columns:

lick here to view code image

```
In [6]: np.arange(1, 21).reshape(4, 5)
Out[6]:
array([[ 1,  2,  3,  4,  5],
       [ 6,  7,  8,  9, 10],
       [11, 12, 13, 14, 15],
       [16, 17, 18, 19, 20]])
```

Note the *chained method calls* in the preceding snippet. First, `arange` produces an

C

`array` containing the values 1–20. Then we call `reshape` on that `array` to get the 4-by-5 `array` that was displayed.

You can reshape any `array`, provided that the new shape has the *same* number of elements as the original. So a six-element one-dimensional `array` can become a 3-by-2 or 2-by-3 `array`, and vice versa, but attempting to reshape a 15-element `array` into a 4-by-4 `array` (16 elements) causes a `ValueError`.

Displaying Large `array`s

When displaying an `array`, if there are 1000 items or more, NumPy drops the middle rows, columns or both from the output. The following snippets generate 100,000 elements. The first case shows all four rows but only the first and last three of the 25,000 columns. The notation `...` represents the missing data. The second case shows the first and last three of the 100 rows, and the first and last three of the 1000 columns:

lick here to view code image

```
In [7]: np.arange(1, 100001).reshape(4, 25000)
Out[7]:
array([[     1,      2,      3, ...,  24998,  24999,     25000],
       [ 25001,  25002,  25003, ...,  49998,  49999,     50000],
       [ 50001,  50002,  50003, ...,  74998,  74999,     75000],
       [ 75001,  75002,  75003, ...,  99998,  99999,    100000]])

In [8]: np.arange(1, 100001).reshape(100, 1000)
Out[8]:
array([[     1,      2,      3, ...,    998,    999,      1000],
       [  1001,   1002,   1003, ...,   1998,   1999,      2000],
       [  2001,   2002,   2003, ...,   2998,   2999,      3000],
       ...,
       [ 97001,  97002,  97003, ...,  97998,  97999,     98000],
       [ 98001,  98002,  98003, ...,  98998,  98999,     99000],
       [ 99001,  99002,  99003, ...,  99998,  99999,    100000]])
```

7.6 LIST VS. `array` PERFORMANCE: INTRODUCING `%timeit`

Most `array` operations execute *significantly* faster than corresponding list operations. To demonstrate, we'll use the IPython **`%timeit` magic** command, which times the *average* duration of operations. Note that the times displayed on your system may vary from what we show here.

C

Timing the Creation of a List Containing Results of 6,000,000 Die Rolls

We've demonstrated rolling a six-sided die 6,000,000 times. Here, let's use the `random` module's `randrange` function with a list comprehension to create a list of six million die rolls and time the operation using `%timeit`. Note that we used the line-continuation character (`\`) to split the statement in snippet `[2]` over two lines:

lick here to view code image

```
In [1]: import random

In [2]: %timeit rolls_list = \
   ...:     [random.randrange(1,   7) for i in range(0, 6_000_000)]
6.29 s ± 119 ms per loop (mean ± std. dev. of 7 runs, 1   loop each)
```

By default, `%timeit` executes a statement in a loop, and it runs the loop *seven* times. If you do not indicate the number of loops, `%timeit` chooses an appropriate value. In our testing, operations that on average took more than 500 milliseconds iterated only once, and operations that took fewer than 500 milliseconds iterated 10 times or more.

After executing the statement, `%timeit` displays the statement's *average* execution time, as well as the standard deviation of all the executions. On average, `%timeit` indicates that it took 6.29 seconds (`s`) to create the list with a standard deviation of 119 milliseconds (`ms`). In total, the preceding snippet took about 44 seconds to run the snippet seven times.

Timing the Creation of an `array` Containing Results of 6,000,000 Die Rolls

Now, let's use the **`randint` function** from the **`numpy.random` module** to create an `array` of 6,000,000 die rolls

lick here to view code image

```
In [3]: import numpy as np

In [4]: %timeit rolls_array = np.random.randint(1, 7, 6_000_000)
72.4 ms ± 635 µs per loop (mean ± std. dev. of 7 runs, 10   loops each)
```

On average, `%timeit` indicates that it took only 72.4 *milliseconds* with a standard deviation of 635 microseconds (`µs`) to create the `array`. In total, the preceding snippet took just under half a second to execute on our computer—about 1/100th of the time

C

snippet [2] took to execute. The operation is *two orders of magnitude faster* with array!

60,000,000 and 600,000,000 Die Rolls

Now, let's create an array of 60,000,000 die rolls:

lick here to view code image

```
In [5]: %timeit rolls_array = np.random.randint(1, 7, 60_000_000)
873 ms ± 29.4 ms per loop (mean ± std. dev. of 7 runs, 1   loop each)
```

On average, it took only 873 milliseconds to create the array.

Finally, let's do 600,000,000 million die rolls:

lick here to view code image

```
In [6]: %timeit rolls_array = np.random.randint(1, 7, 600_000_000)
10.1 s ± 232 ms per loop (mean ± std. dev. of 7 runs, 1   loop each)
```

It took about 10 seconds to create 600,000,000 elements with NumPy vs. about 6 seconds to create only 6,000,000 elements with a list comprehension.

Based on these timing studies, you can see clearly why arrays are preferred over lists for compute-intensive operations. In the data science case studies, we'll enter the performance-intensive worlds of big data and AI. We'll see how clever hardware, software, communications and algorithm designs combine to meet the often enormous computing challenges of today's applications.

Customizing the %timeit Iterations

The number of iterations within each %timeit loop and the number of loops are customizable with the -n and -r options. The following executes snippet [4]'s statement three times per loop and runs the loop twice: [2]

[2] For most readers, using %timeits default settings should be fine.

lick here to view code image

```
In [7]: %timeit -n3 -r2 rolls_array = np.random.randint(1, 7, 6_000_000)
85.5 ms ± 5.32 ms per loop (mean ± std. dev. of 2 runs, 3   loops each)
```

C

Other IPython Magics

IPython provides dozens of magics for a variety of tasks—for a complete list, see the IPython magics documentation. [3] Here are a few helpful ones:

3

ttp://ipython.readthedocs.io/en/stable/interactive/magics.html.

- `%load` to read code into IPython from a local file or URL.
- `%save` to save snippets to a file.
- `%run` to execute a .py file from IPython.
- `%precision` to change the default floating-point precision for IPython outputs.
- `%cd` to change directories without having to exit IPython first.
- `%edit` to launch an external editor—handy if you need to modify more complex snippets.
- `%history` to view a list of all snippets and commands you've executed in the current IPython session.

7.7 ARRAY OPERATORS

NumPy provides many operators which enable you to write simple expressions that perform operations on entire `array`s. Here, we demonstrate arithmetic between `array`s and numeric values and between `array`s of the same shape.

Arithmetic Operations with `array`s and Individual Numeric Values

First, let's perform *element-wise arithmetic* with `array`s and numeric values by using arithmetic operators and augmented assignments. Element-wise operations are applied to every element, so snippet `[4]` multiplies every element by `2` and snippet `[5]` cubes every element. Each returns a *new* `array` containing the result:

lick here to view code image

```
In [1]: import numpy as np
```